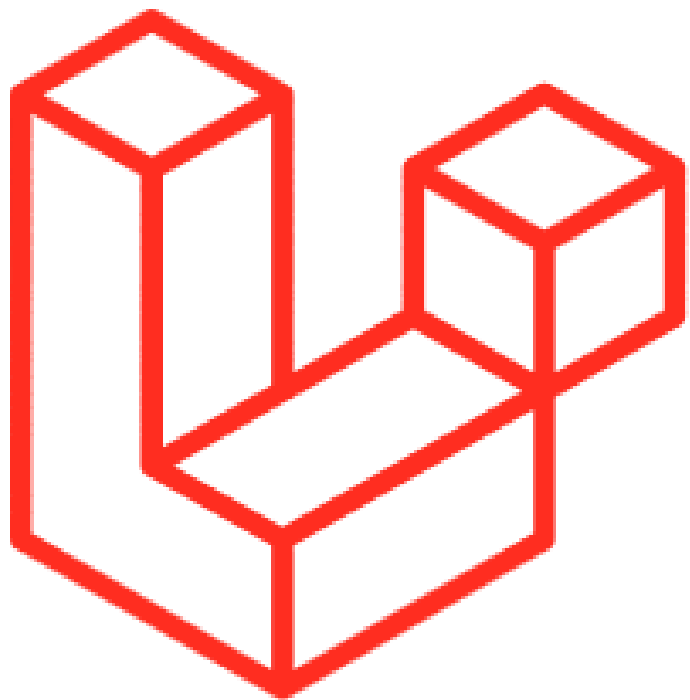




Laravel

PRIMA DI INIZIARE

REST API

API – **A**pplication **P**rogramming **I**nterface

RESTful – **R**epresentational **S**tate **T**ransfer

A livello base, un'API è un meccanismo che consente ad un'applicazione o servizio di accedere ad una risorsa all'interno di un'altra applicazione o servizio.

L'applicazione o servizio che accede alle risorse è il client.

L'applicazione o servizio che contiene la risorsa è il server.

Le API rest possono essere sviluppate in diversi linguaggi e supportano diversi formati di dati ma devono seguire i sei vincoli architetturali di progettazione REST.

VINCOLI ARCHITETTURALI REST

1. **INTERFACCIA UNIFORME:** tutte le richieste per la stessa risorsa devono avere lo stesso aspetto, indipendentemente dalla provenienza della richiesta stessa. L'API REST deve quindi garantire che lo stesso dato appartenga ad un solo URI (Uniform Resource Identifier).
2. **DISACCOPPAMENTO CLIENT-SERVER:** le applicazioni client e server devono essere totalmente indipendenti l'una dall'altra. L'unica informazione che il client deve conoscere è l'URI della risorsa, non ha bisogno di sapere come i dati sono memorizzati nel server. Non può interagire con l'applicazione server in nessun altro modo. L'applicazione server, a sua volta, non può modificare l'applicazione client se non passandole i dati richiesti via HTTP.
3. **CONDIZIONE DI STATELESS:** le REST API sono stateless. Ogni richiesta è indipendente e deve includere tutte le informazioni necessarie per elaborarla. Il server non può memorizzare dati relativi alla richiesta del client e non mantiene lo stato delle sessioni tra le richieste.
4. **POSSIBILITÀ DI MEMORIZZAZIONE NELLA CACHE:** le risorse possono essere memorizzate nella cache lato server o client. Le API devono indicare se una risposta è cacheabile o meno.
5. **ARCHITETTURA DI SISTEMA A LIVELLI:** le chiamate e risposte passano attraverso diversi livelli, non sempre le applicazioni si connettono direttamente. Possono esistere diversi intermediari e le API REST devono essere progettate in modo tale che né il client né il server sappiano se comunicano con l'app finale o con un intermediario.
6. **CODICE ON DEMAND (FACOLTATIVO):** di solito le API REST inviano risorse statiche. In alcuni casi possono contenere codice eseguibile, ma solo on-demand.

COME FUNZIONANO LE REST API

Passaggi generali di qualsiasi chiamata REST API:

1. Il **client invia una richiesta al server** formattandola in modo che il server la comprenda (ogni API ha una sua documentazione di riferimento)
2. Il **server autentica il client** e conferma che ha il diritto di effettuare la richiesta
3. Il server **riceve la richiesta** e **la elabora** internamente
4. Il **server risponde al client**. La risposta contiene informazioni circa l'esito della richiesta e ogni altra informazione richiesta dal client.

Le API REST comunicano tramite **richieste HTTP** per eseguire le funzioni di database standard come la creazione, lettura, aggiornamento e eliminazione di record (**CRUD**) all'interno di una risorsa.

Ad esempio, un API REST utilizzerà una richiesta GET per recuperare un record. Una richiesta POST crea un nuovo record.

Lo stato di una risorsa in un determinato istante è noto come **rappresentazione della risorsa**. Queste informazioni possono essere inviate al client praticamente in qualsiasi formato. Il più comune è **JSON** perché indipendente dal linguaggio di programmazione e perché leggibile sia dall'uomo che dalle macchine.

CONTENUTO DELLA RICHIESTA API

COMPONENTI FONDAMENTALI

- **ENDPOINT (URL della risorsa):** è l'indirizzo web della risorsa con cui vogliamo interagire.
Esempio: <https://api.example.com/users/123> per richiedere i dettagli dell'utente con ID 123
- **METODO HTTP:** indica l'operazione che vogliamo eseguire sulla risorsa. I principali metodi HTTP nelle REST API sono:
 - **GET:** recupera una risorsa
 - **POST:** crea una risorsa. Inviare la stessa richiesta POST più volte ha l'effetto di creare la stessa risorsa più volte.
 - **PUT:** aggiorna una risorsa esistente (sostituzione completa)
 - **PATCH:** aggiorna parzialmente una risorsa
 - **DELETE:** elimina una risorsa

- **HEADERS (intestazioni HTTP):** forniscono informazioni aggiuntive sulla richiesta (metadati), come il formato dei dati o autenticazione.

ESEMPIO DI HEADER COMUNE

Content-Type: application/json

Authorization: Bearer <token>

User-Agent: MyApp/1.0

- **BODY (corpo della richiesta):** contiene i dati che si vogliono inviare al server (solo per POST, PUT, PATCH).
- **QUERY PARAMETERS** (parametri della query): vengono aggiunti all'URL per filtrare o personalizzare la richiesta
<https://api.example.com/users?limit=10&sort=desc>
- **PATH PARAMETERS** (parametri nel percorso) – opzionale: sono parte dell'URL e servono per identificare una specifica risorsa

<https://api.example.com/users/123>

CONTENUTO DELLA RESPONSE

COMPONENTI FONDAMENTALI

- **STATUS CODE:** risultato della richiesta HTTP

HEADERS (intestazioni della risposta)

Contengono metadati sulla risposta.

Esempio:

Content-Type: application/json

Content-Length: 256

Cache-Control: no-cache

BODY (corpo della risposta)

Contiene i dati restituiti dal server (di solito JSON nelle API moderne).

Esempio:

```
{  
  "id": 123,  
  "name": "Mario Rossi",  
  "email": "mario@example.com"  
}
```

HEADER DELLA REQUEST

- Può contenere informazioni extra per la request, come:

PARAMETRO	FUNZIONE	ESEMPIO
Content-Type	Specifica formato del contenuto del body	application/json
Accept	Tipo di contenuto che il client si aspetta come risposta	application/json, text/html
Authorization	Autorizzazione tramite token	bearer eyjhbgtci0ijiuzi1.....
User-Agent	Informazioni sul client che fa la richiesta	postmanruntime/7.30.0
Cache-Control	Indica se contenuto può essere memorizzato	no-cache, max-age=0
X-Requested-With	Spesso usato per richieste AJAX	xmlhttprequest
Origin	Origine della richiesta (per CORS)	https://frontend.miosito.it
Cookie	Dati di sessione inviati con la richiesta	sessionId=abc123

BODY DELLA REQUEST

Può avere diversi formati a seconda del tipo di contenuto che si vuole inviare o ricevere. Il formato va specificato nell'header così che il server o client sappia come interpretare il body della richiesta o della risposta.

FORMATO	MIME TYPE	USO PRINCIPALE
JSON	Application/json	Dati strutturati, standard moderno
XML	Application/xml	Sistemi legacy, SOAP
URL-encoded	Application/x-www-form-urlencoded	Form HTML classici
Multipart/form-data	Multipart/form-data	Upload file
Plain Text	Text/plain	Testo semplice
HTML	Text/html	Risposte di pagine web
Binari (PDF, img...)	Application/pdf, image/png, ecc	File e media

BODY - FORMATI

JSON (il più comune)

- Usato per scambio dati tra client e server in modo leggibile e strutturato
- Leggero, leggibile, supportato ovunque

```
json

{
  "nome": "Luca",
  "email": "luca@example.com"
}
```

XML (Extensible Markup Language)

- Strutturato, formale ma più verboso del JSON

```
xml

<utente>
  <nome>Luca</nome>
  <email>luca@example.com</email>
</utente>
```

Form URL Encoded

- Usato per form HTML tradizionali, richieste leggere
- Simile ai parametri di una query string

```
ini

nome=Luca&email=luca%40example.com
```

BODY - FORMATI

Multipart/Form-Data

- Usato per Upload di file (es. immagini, documenti).
- diviso in “parti” con intestazioni per ciascuna sezione.

```
--boundary
Content-Disposition: form-data; name="foto"; filename="profilo.jpg"
Content-Type: image/jpeg
```

Plain Text

- Usato per Inviare o ricevere semplice testo, log, messaggi brevi.

```
Questo è un messaggio di testo semplice.
```

HTML

- Usato per ricevere una risposta in formato HTML da server web

```
<html><body><h1>Benvenuto</h1></body></html>
```

FILE BINARI (IMMAGINI, PDF, AUDIO, ECC.)

- Usato per scaricare file, immagini, audio.
- Contiene dati binari non leggibili da uomini.

ESEMPIO REQUEST + RESPONSE

```
POST /api/utenti HTTP/1.1
Host: api.miosito.it
Authorization: Bearer xyz123
Content-Type: application/json

{
  "nome": "Luca",
  "email": "luca@example.com"
}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 1,
  "nome": "Luca",
  "email": "luca@example.com"
}
```

METODI DI AUTENTICAZIONE DI REST API

DIFFERENZE TRA AUTENTICAZIONE ED AUTORIZZAZIONE

AUTENTICAZIONE	AUTORIZZAZIONE
Determina se gli utenti o il client sono chi dicono di essere	Determina cosa un utente o client può o non può fare una volta autenticato
Richiede all'utente di fornire delle credenziali per essere riconosciuto	Verifica se l'accesso alle risorse è consentito attraverso una serie policies e regole basate su ruoli, scopes o permessi
Solitamente avviene prima dell'autorizzazione	Generalmente avviene dopo l'autenticazione
Generalmente trasmette info attraverso un ID token	In genere trasmette info attraverso un Access Token

Le richieste devono essere autenticate prima che il server possa inviare una risposta.

Le API REST presentano quattro **metodi di autenticazione** comuni:

- **AUTENTICAZIONE HTTP**: l'HTTP definisce alcuni schemi di autenticazione che possono essere utilizzati direttamente per implementare API REST. Tra questi:
 - **Autenticazione di base (Basic Authentication)**: il client invia username e password nell'header Authorization di ogni richiesta, codificandoli con base64 (convertiti in un set di 64 caratteri. NB: codificati, non cifrati). Poco sicuro perché i dati viaggiano in chiaro, va usato su HTTPS.
 - **Autenticazione bearer**: il controllo di accesso avviene tramite token bearer. Questo è solitamente una stringa crittografata di caratteri che il server genera in risposta alla richiesta di login. Il client invia il token nell'header della richiesta per accedere alle risorse. I token hanno una durata temporale limitata.

- **CHIAVI API:** sono valori unici che vengono assegnati dal server ad ogni utente al primo accesso. Il client usa questa chiave per autenticarsi in ogni richiesta. Le chiavi API sono meno sicure perché possono essere esposte se trasmesse in chiaro o memorizzate nel client. Può essere utile per API pubbliche o interne con accesso limitato. Consentono il monitoraggio del flusso degli utenti.
- **Autenticazione con JWT (JSON Web Token):** meccanismo stateless per autenticazione delle API, non prevede la memorizzazione dei dati del client sul server. Quando l'utente accede all'applicazione, il server API crea un JWT che contiene l'identità dell'utente. Il JWT può essere firmato digitalmente o crittografato e deve essere incluso in ogni richiesta.
- **OAuth 2.0** (Autenticazione Delegata): **il client ottiene un access token dopo l'autenticazione su un provider di identità (IdP).** Protocollo di autorizzazione utilizzato da Google, Facebook, ecc. L'utente effettua il login tramite un provider – il provider restituisce un access token – il client usa il token per accedere alle API. È un sistema sicuro, usato per Single Sign-On (SSO), non richiede gestione di password ma è complesso da implementare e richiede un provider di identità.
- **HMAC (Hash-based Message Authentication Code):** il client genera una firma digitale basata su una chiave segreta. La firma viene verificata dal server per autenticare la richiesta. È un metodo sicuro perché i dati non vengono trasmessi in chiaro ed è difficile da falsificare. Richiede però più risorse computazionali.

STRUMENTI PER L'AUTENTICAZIONE API IN LARAVEL:

- **Laravel Passport** per OAuth 2.0 authentication: supporta access token con OAuth 2.0 (client credentials, authorization code, password grant, refresh tokens). Adatto per API pubbliche, permette a terzi di autenticarsi nel sistema (se, ad esempio, un'app esterna utilizza la nostra API), gestisce scopes e permessi granulari sui token.
- **Laravel Sanctum:** progettato per SPA, applicazioni mobile e API leggere. Utilizza token personali (Personal Access Token) e supporta l'autenticazione tramite cookie-based session per SPA

COSA CONTIENE LA RISPOSTA DEL SERVER – COMPONENTI PRINCIPALI

- **STATUS CODE**: contiene codice di stato che indica l'esito della richiesta. I codici 2xx indicano il buon esito della richiesta, i codici 4xx e 5xx indicano errori, i codici 3xx indicano reindirizzamenti URL.
- **HEADERS** (intestazioni HTTP): forniscono metadati relativi alla risposta stessa. Comunemente includono:
- **BODY**: contiene i dati veri e propri inviati dal server al client. Generalmente in formato JSON o XML (ma può essere in uno dei formati già presentati per la request). Solitamente contiene:
 - Dati della risorsa richiesta
 - Eventuale messaggio di errore come Invalid Parameters o Resource not found
 - **MESSAGGIO DI ERRORE** (se applicabile): se la richiesta non ha avuto successo, il corpo della risposta può contenere info dettagliate sull'errore. Generalmente si usa una struttura JSON o XML che descrive l'errore.

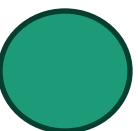
ESEMPIO DI
RISPOSTA 400

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
X-Rate-Limit: 500

{
  "status": "error",
  "message": "Missing required parameter: email"
}
```

VEDI CODICI DI STATO HTTP

>>>



HEADER DELLA RESPONSE

```
http

HTTP/1.1 200 OK
Date: Wed, 25 Jun 2025 12:30:00 GMT
Content-Type: application/json
Content-Length: 128
Cache-Control: no-cache
Access-Control-Allow-Origin: *
Set-Cookie: sessionId=xyz123; HttpOnly
```

- **Content-Type:** tipo di contenuto della risposta. Ad es.: application/json, application/xml, text/html, ecc.
- **Content-Length:** dimensioni del contenuto in bite. Es. 1287
- **Date:** data e ora della risposta. ES: Wed, 25 Jun 2025 12:30:00 GMT
- **Cache-Control:** gestisce cache della risposta. ES: no-cache, max-age=3600
- **Etag:** identificatore univoco della risorsa (per cache validation). ES: "abc123xyz"
- **Expires:** dati di scadenza della risorsa memorizzata in cache
- **Last-Modified:** data ultima modifica alla risorsa
- **Set-Cookie:** invia cookie da memorizzare nel browser
- **Access-Control-Allow-Origin:** autorizza richieste cross-domain
- **Content-Encoding:** indica la compressione del contenuto. ES. gzip, br
- **Location:** indica l'URL della risorsa appena creata nel caso di chiamate POST
- **WWW-Authenticate:** specifica il tipo di autenticazione richiesta. ES: bearer realm="Access to protected API"
- **Retry-After:** indica quanto attendere prima di rifare la richiesta. ES: 120 (secondi) o data
- **X-RateLimit-Remaining:** Quante richieste utente può ancora fare

HTTP Status Code

PIÙ COMUNI

1XX - INFORMAZIONI

- 100** Continue
- 101** Switching Protocols
- 102** Processing
- 103** Early Hints

3XX - REDIRECTION

- 300** Multiple Choices
- 301** Moved Permanently
- 302** Found
- 303** See Other
- 304** Not Modified
- 305** Use Proxy
- 306** Switch Proxy
- 307** Temporary Redirect
- 308** Permanent Redirect

2XX - SUCCESSO

- 200** OK - GET, POST, PUT,
- 201** DELETE Created - POST
- 202** Accepted
- 203** Non-Authoritative
- 204** No Content
- 205** Reset Content
- 206** Partial Content

5XX - SERVER ERRORS

- 500** Internal Server Error
- 501** Not Implemented
- 502** Bad Gateway
- 503** Service Unavailable
- 504** Gateway Timeout
- 505** HTTP Version Not Supported
- 507** Insufficient Storage
- 511** Network Authentication Required

4XX - CLIENT ERRORS

- 400** Bad Request
- 401** Unauthorized
- 402** Payment Required
- 403** Forbidden
- 404** Not Found
- 405** Method Not Allowed
- 407** Proxy Authentication Required
- 408** Request Timeout
- 409** Conflict
- 410** Gone
- 412** Precondition Failed
- 416** Request Range Not Satisfiable
- 417** Expectation Failed
- 422** Unprocessable Entity
- 423** Locked
- 429** Too Many Requests
- 451** Unavailable For Legal Reasons

ESEMPIO DI RISPOSTA 200

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
X-Rate-Limit: 1000

{
  "status": "success",
  "data": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "roles": ["user", "admin"]
  }
}
```

INTRODUZIONE A LARAVEL

v. 12.x

COS'È LARAVEL

Laravel è un framework PHP open source creato nel 2011 da Taylor Otwell.

Si basa sul pattern architetturale **MVC** (**Model – View – Controller**) e fornisce strumenti per la gestione delle richieste HTTP, l'elaborazione dati, l'accesso al database, la sicurezza e tanto altro.

LARAVEL COME FULL STACK FRAMEWORK

Laravel può essere usato come framework **full stack**: indirizza le richieste all'applicazione e rende il frontend via **Blade templates** o **single-page application** di tecnologia ibrida come **Inertia**.

LARAVEL COME API BACKEND

Laravel può anche servire come **API** ad una **single page application** JavaScript o ad un'app mobile. Ad esempio, si può usare Laravel come **API backend** per un'applicazione **Angular** o **Next.js**.

Il pattern MVC

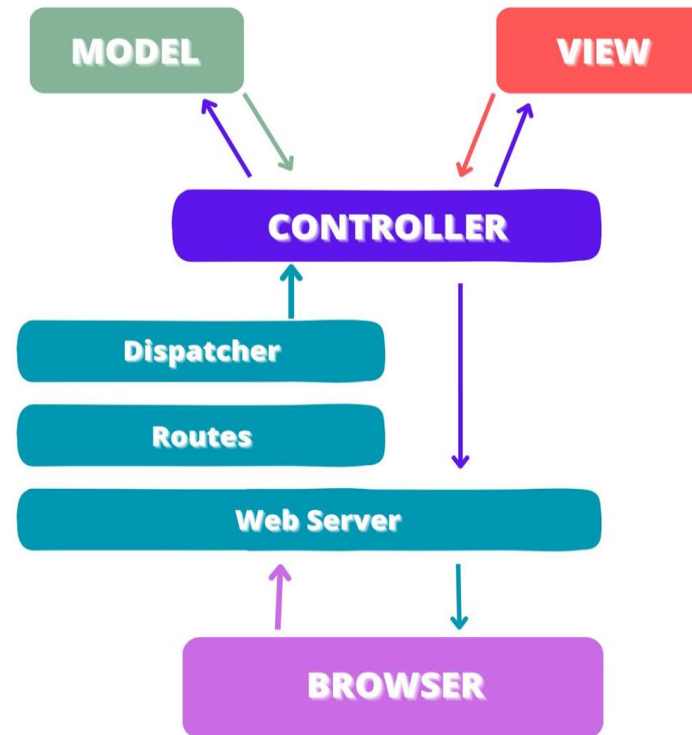
Pattern architetturale molto diffuso per lo sviluppo di applicazioni web.

In estrema sintesi: il **Model** (dati e logica dell'app) viene mostrato tramite la **View** (visualizzazione dei dati ed interfaccia utente) all'utente, il quale genera gli input con cui il **Controller** aggiorna il **Model**.

Questa separazione di funzionalità rende il codice **modulare** e più facilmente **manutenibile**.

Consente inoltre a persone differenti di lavorare contemporaneamente su parti diverse del codice.

ES: un designer può lavorare sulle view mentre il programmatore lavora sul controller.



L'**utente**, mediante il browser, interagisce con l'applicazione.

Il **browser** invia al **Web server** l'azione svolta dall'utente.

L'applicativo deve quindi **intercettare la richiesta dell'utente (Routes)** e **valutarla (Dispatcher)** in modo da richiamare il **controller** giusto con i giusti parametri.

1- CONTROLLER (gestione degli input)

Sono responsabili della **gestione degli input** dell'utente e della **generazione della risposta appropriata**. Sono gli INTERMEDIARI tra View e Model

Solitamente il controller riceve la richiesta dell'utente, la **convalida** e la passa ad un modello per l'elaborazione.

Una volta che i dati sono stati elaborati, il controller genera l'output e lo passa alla view.

Cartella Laravel: *app/Http/Controllers*

2- MODELS (dati)

Sono i responsabili del **recupero, creazione, aggiornamento e cancellazione dei dati**. Incapsula tutta la logica necessaria per la **manipolazione dei dati**. Ricevono i dati dal controller con il quale dialogano per il passaggio di dati in ingresso e uscita.

Oltre alle operazioni CRUD di base (comunicherà dunque con il database), un model potrebbe contenere altre logiche come operazioni di casting e relazioni con altri modelli.

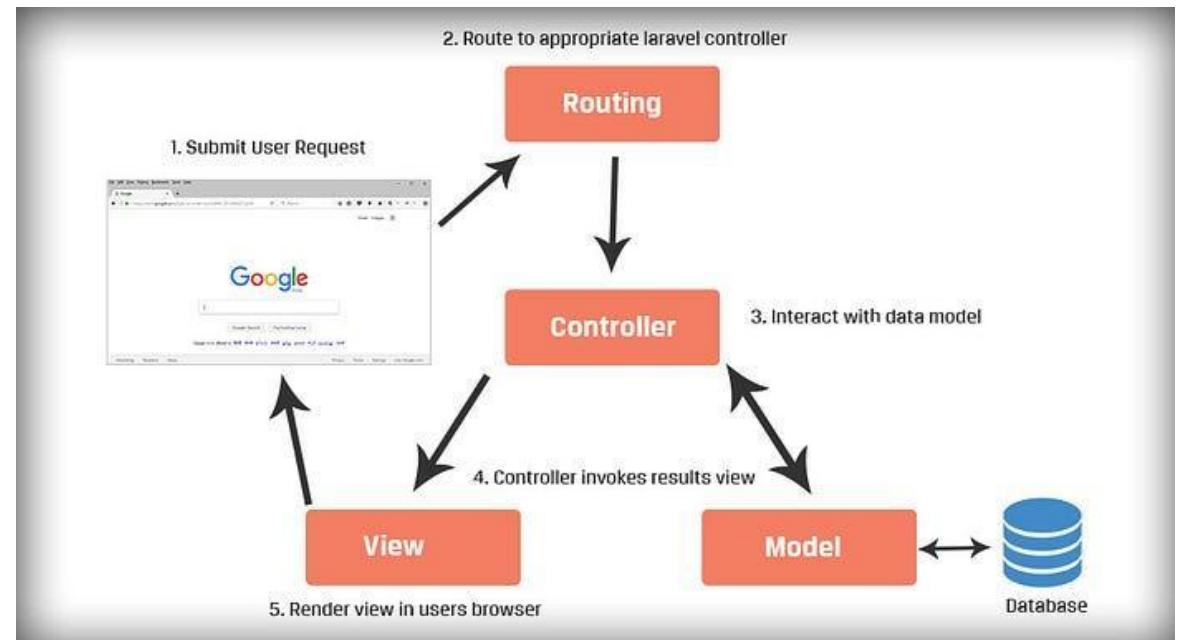
Per convenzione, in Laravel i Model sono dichiarati con il nome al **singolare** mentre la relativa **tabella** viene dichiarata usando il **plurale**.

Cartella Laravel : *app/Models*

3- VIEW (vista)

Componente per servire **l'interfaccia all'utente**.

Contengono solitamente HTML, gli assets JavaScript e CSS e dati dinamici passati dal controller.



INSTALLAZIONE DI LARAVEL

PRIMA DI INIZIARE

Bisogna prima di tutto assicurarsi che sulla propria macchina siano installati **PHP**, **Composer** e **Laravel installer**. Se Composer e PHP sono già installati, si può installare l'installer di Laravel via Composer:

```
composer global require laravel/installer
```

Se si utilizza macOS, PHP e [Composer](#) possono essere installati via [Homebrew](#).

Si raccomanda inoltre l'installazione di [Node](#) e [NPM](#).

COMPOSER

Composer è un tool per la **gestione delle dipendenze** (**dependency manager**) per PHP. Supponiamo di avere un progetto che dipende da **un certo numero di librerie ed alcune di queste librerie dipendono da altre**.

Composer consente di dichiarare le librerie da cui l'app dipende, trova le versioni dei pacchetti che possono e hanno bisogno di essere installate e le installa/aggiorna (le scarica all'interno del progetto nella cartella vendor). Tutte le dipendenze possono inoltre essere aggiornate con un solo comando.

Composer non si occupa solo di definire e recuperare le dipendenze di un progetto, ma **anche dell'autoloading delle classi PHP**.

PRINCIPALI COMANDI DI COMPOSER

a. Il file `composer.json`

Un progetto che utilizza Composer è caratterizzato dalla presenza del file **`composer.json`** nella directory principale. Questo file contiene **tutte le dipendenze per il progetto**. In un progetto Laravel viene generato contestualmente alla creazione dell'app.

```
{
    "require": {
        "monolog/monolog": "^3.0.0"
    }
}
```

ESEMPIO
MINIMALE DI
COMPOSER.JSON
*Dice che l'unica
dipendenza del
progetto è
Monolog.*

Ogni dipendenza è composta dal **nome del pacchetto** (**package name**). Composto a sua volta dal nome del **vendor** e nome del pacchetto separati da /) e da un **selettore di versione** (**version constraint***).

Con il comando

```
composer install
```

oppure

```
php composer.phar update
```

i pacchetti specificati all'interno del file verranno **scaricati e installati**, la loro esatta versione verrà **trascritta nel file composer.lock** (file che serve per «bloccare» **l'esatta versione delle dipendenze**) e le dipendenze saranno inserite nella cartella vendor del progetto.

b. Installare un pacchetto (require)

Il repository principale dal quale composer scarica i pacchetti è **Packagist**. Attraverso il comando **require** la libreria viene cercata automaticamente tra quelle presenti in questo repository.

```
composer require mypackage
```

c. Cancellare un pacchetto (remove)

```
Composer remove mypackage
```

Il pacchetto verrà eliminato dalle dipendenze del progetto e dalla cartella vendor

d. Aggiornare uno o più pacchetti (update)

Per aggiornare un pacchetto alla **versione più recente compatibile** con il selettore di versione indicato si usa il comando update seguito dal nome del pacchetto.

```
Composer update mypackage
```

Per **aggiornare tutti i pacchetti** del progetto basta eseguire il comando update senza specificare un pacchetto.

```
Composer update
```

NOTE:

*^3.0.0 vuol dire in pratica "ogni versione maggiore di 3.0.0, ma non la 4.0.0 e successive"

- e. **Mostrare informazioni su un pacchetto (show)**
Per ricevere informazioni su un pacchetto si usa il comando

composer **show** vendor/package

Per informazioni su **ulteriori comandi** di Composer usare

Composer **list** oppure composer **help**

Esempio installare laravel debugbar

```
composer require barryvdh/laravel-debugbar --dev
```


versioni

- ^ (caret) → “aggiorna in sicurezza”
- ^3.0.0
- Significa:
- 👉 accetta tutte le versioni **compatibili con la 3.x**

- ~ (tilde) → “aggiorna poco”
- ~3.0.0
- Significa:
- 👉 accetta solo aggiornamenti **della patch (3.0.x)**
- **Senza simboli**
- 3.0.0
- 👉 prende **esattamente quella versione**

versioni

- **>= (maggiore o uguale)**
- **>=3.0.0**
- **✓ 3.0.0, 3.1.0, 4.0.0...**
-
- **<= (minore o uguale)**
- **<=3.0.0**
-
- **Range**
- **>=3.0.0 <4.0.0**
- **👉 equivalente a:**
- **^3.0.0**

Simbolo	Significato	Aggiorna
<code>^3.0.0</code>	compatibile	minor + patch
<code>~3.0.0</code>	conservativo	solo patch
<code>3.0.0</code>	fisso	niente
<code>>=</code>	aperto	tutto sopra

NPM (Node Package Manager)

Gestore di pacchetti per progetti Node.js che permette l'installazione di librerie, framework e altri strumenti di sviluppo. L'interfaccia a riga di comando (CLI) consente di installare e disinstallare pacchetti e di accedere ad altre funzionalità.

PRINCIPALI COMANDI :

- 1) `Npm install`
Necessario all'installazione di un pacchetto
- 2) `Npm uninstall`
disinstalla Pacchetto
- 3) `Npm init`
usato per inizializzare un progetto. Quando si esegue questo comando, il file `package.json` viene creato
- 4) `Npm update`
Aggiorna un pacchetto all'ultima versione

PRIMO PROGETTO IN LARAVEL

E' ora possibile creare un nuovo progetto Laravel via Composer grazie al comando **create-project**

```
composer create-project  
laravel/laravel:"12.*" nome-progetto
```

Oppure è possibile creare un nuovo progetto installando globalmente Laravel via Composer

```
composer global require  
laravel/installer
```

```
laravel new example-app
```

Dopo la creazione del progetto, avviare il server di sviluppo locale utilizzando il comando **serve** della CLI (Command Line Interface) Artisan di Laravel

```
cd example-app  
php artisan serve
```

Se fossero necessari dei pacchetti node (in caso di SPA e non API Only):

```
npm install && npm run build  
composer run dev
```

Una volta lanciato il server di sviluppo, l'applicazione sarà accessibile nel browser all'indirizzo

```
http://localhost:8000
```

Configurazione iniziale

Tutti i file di configurazione di Laravel si trovano nella cartella **config**.

Questi file di configurazione permettono di impostare, ad esempio, le informazioni per la connessione al database, le info relative al server mail oppure l'encryption key.

Tuttavia, spesso Laravel non necessita di configurazioni aggiuntive. Può essere utile rivedere il file **config/app.php** e la sua documentazione. Esso contiene diverse opzioni come la **url** and **locale**.

CONFIGURAZIONE BASATA SULL'AMBIENTE

Molte opzioni di configurazione di Laravel possono cambiare a seconda che l'app stia girando su una macchina locale o su un web server di produzione. Per questo motivo molti importanti valori di configurazione sono definiti usando file **.env** che esistono alla radice dell'app.

Il file **.env** non dovrebbe essere committato.

DATABASE E MIGRATIONS

Di default l'applicazione interagirà con un **database SQLite**. Durante la creazione dell'app, Laravel genera un file **database/database.sqlite** e lancia le migrazioni necessarie per creare le tabelle del database.

Qualora si preferisca utilizzare un database MySQL, bisognerà aggiornare il file **.env**.

```
1 DB_CONNECTION=mysql
2 DB_HOST=127.0.0.1
3 DB_PORT=3306
4 DB_DATABASE=laravel
5 DB_USERNAME=root
6 DB_PASSWORD=
```

Una volta configurato il database, si lancia la migrazione del database dell'applicazione. Questa **creerà le tabelle del database**

php **artisan migrate**

STRUTTURA DELLE CARTELLE – Root

directory

ROOT DIRECTORY

- The **app** Directory
 - The **bootstrap** Directory
 - The **config** Directory
 - The **database** Directory
 - The **public** Directory
 - The **resources** Directory
 - The **routes** Directory
 - The **storage** Directory
 - The **tests** Directory
 - The **vendor** Directory
- **LA CARTELLA APP**
Contiene il cuore dell'applicazione. Quasi tutte le classi di un'applicazione si trovano in questa cartella
 - **CARTELLA BOOTSTRAP**
Contiene il file **app.php** che avvia il framework. Al suo interno si ha la **cartella cache** che contiene i files generati dal framework per **l'ottimizzazione delle performances** come i file di cache delle routes e dei services.
 - **CARTELLA CONFIG**
Contiene tutti i files di configurazione dell'applicazione.
 - **CARTELLA DATABASE**
Contiene le **migrazioni del database**, i **model factories** ed i **seeds**. Può essere usata per ospitare un database SQLite.
 - **CARTELLA PUBLIC**
Contiene il file **index.php** che è **l'entry point** di tutte le richieste effettuate all'applicazione. Contiene anche gli assets come immagini, JS e CSS

- **CARTELLA RESOURCES**

contiene le **views** e gli **assets** grezzi e non compilati come CSS e JS

- **CARTELLA ROUTES**

contiene la definizione di tutte le rotte dell'applicazione. Di default sono inclusi i file **web.php**, **api.php**, **console.php** e **channels.php**.

- File **web.php**: contiene le rotte che il RouteServiceProvider posiziona nel gruppo **middleware web** il quale fornisce lo stato di sessione, la CSRF protection e la cookie encryption. Se l'applicazione non prevede una Restful API stateless*, allora tutte le rotte saranno definite in questo file.
- File **console.php**: file nel quale definire i comandi della console basati su closure. Definisce gli entry point da console. In questo file è anche possibile pianificare attività

**(Stateless): In un'applicazione stateless, ogni richiesta del client al server contiene tutte le informazioni necessarie per comprendere e soddisfare quella richiesta*

Opzionalmente si possono prevedere file per rotte API (api.php) e per canali broadcastin (channels.php) grazie ai comandi `install:api` e `install:broadcasting`.

- File **api.php**: contiene le **rotte del middleware api**. Queste rotte devono essere **stateless** quindi le richieste che accedono all'app attraverso queste rotte devono essere **autenticate via tokens** e non avranno accesso alla stato della sessione
- Cartella **channels.php**: cartella nella quale registrare tutti i canali di broadcasting degli eventi supportati dall'applicazione

- **CARTELLA STORAGE**

contiene i logs, i templates Blade compilati, sessioni basate su file, le cache dei file e altri file generati dal framework. Contiene tre sottodirectory: app (per memorizzare file generati da app), framework (file e cache generati da framework) e logs (contiene file di log).

- **CARTELLA TEST**: contiene i test automatizzati. Ogni classe di test dovrebbe avere il suffisso Test.

- **CARTELLA VENDOR**: contiene le dipendenze di Composer

STRUTTURA DELLE CARTELLE – App

directory

APP DIRECTORY

- **Console** Directory
- **Events** Directory
- **Exceptions** Directory
- **Http** Directory
- **Jobs** Directory
- **Listeners** Directory
- **Mail** Directory
- **Models** Directory
- **Notifications** Directory
- **Policies** Directory
- **Providers** Directory
- **Rules** Directory

La maggior parte del codice dell'app si trova all'interno della directory app.

Contiene numerose sottodirectory e molte altre vengono create quando si utilizza il comando Artisan **make** per la generazione di classi.

Per esempio, app/Jobs non esiste finché non viene eseguito il comando **artisan make:job** con il quale Artisan crea una classe job.

Per una **lista dei comandi Artisan** con i quali creare directories all'interno della cartella App, digitare nel terminale il comando

```
php artisan list make
```

LEGEND:

Default directory

Not default directory

- **CARTELLA CONSOLE**

contiene tutti i comandi Artisan creati dall'utente con il comando

make:command

- **CARTELLA EVENTI**

Non esiste di default. Creata con il primo evento.

Comandi:

event:generate oppure **make:event**

Ospita le classi relative agli eventi. Questi possono essere utilizzati per avvisare un'altra parte dell'applicazione che una data azione si è verificata.

- **CARTELLA ECCEZIONI**

contiene tutte le eccezioni personalizzate dell'applicazione.

Le eccezioni possono essere generate usando il comando **make:exception**

- **CARTELLA HTTP** ★

Cartella fondamentale che **contiene i controllers, i middleware e le richieste dei form**. Quasi tutta la logica di gestione delle requests che arrivano all'app si trova all'interno di questa cartella.

- **CARTELLA JOBS**

Non esiste di default.

Comando per creazione di Jobs è

make:job

- **CARTELLA LISTENERS**

No di default ma generata con

event:generate oppure **make:listener**

Contiene le classi che gestiscono gli eventi: gli **Event listeners** ricevono l'istanza di un evento e in risposta all'evento elaborano delle azioni o logiche. Ad esempio: un evento **UserRegistered** può essere gestito da un listener **SendWelcomeEmail**

- **CARTELLA MAIL**

Non esiste di default.

Comando Artisan **make:email**.

Contiene le classi che rappresentano **le mail spedite dall'applicazione**.

Gli oggetti mail permettono di incapsulare tutta la logica di generazione di una mail in una singola classe. La mail può essere spedita usando il metodo **Mail::send**

- **CARTELLA MODELS**★

Contiene tutte le classi del **model Eloquent**. L'ORM

Eloquent fornisce una semplice implementazione

ActiveRecord per lavorare con il database: **ogni tabella del database ha un Model che è usato per interagire con quella stessa tabella. I Models permettono di eseguire query per i dati nelle tabelle**

- **CARTELLA NOTIFICATIONS**

No di default.

make: notification

Contiene tutte le notifiche «transazionali» che sono inviate dall'applicazione come notifiche circa eventi.

- **CARTELLA POLICIES**

No di default

make: policy

Contiene le **classi di autorizzazione alla policy**. Le policies sono utilizzate per determinare se un utente può eseguire una determinata azione.

- **CARTELLA PROVIDERS**

contiene tutti i service providers per l'applicazione. Di default contiene AppServiceProvider. I service providers inizializzano l'applicazione effettuando il binding dei servizi nel service container o eseguendo altre operazioni necessarie per preparare l'applicazione a gestire le richieste in arrivo.

- **CARTELLA RULES**

No di default.

make:rule

Contiene gli oggetti delle regole di validazione personalizzate dall'utente. Le regole sono utilizzate per incapsulare logiche di validazione complesse in un oggetto semplice.

FRONTEND

Esistono principalmente due modi per affrontare lo sviluppo del frontend quando si realizza un'applicazione in Laravel:

- Costruire il frontend sfruttando **PHP**
- Usare **framework Javascript** come Vue e React / Angular

Costruire il frontend con PHP e Blade

In passato, la maggior parte delle applicazioni PHP generavano HTML da mostrare nel browser usando semplici templates HTML intervallati da istruzioni echo che riportavano i dati recuperati dal database durante la request:

In Laravel questo approccio può essere ancora usato usando le **views** e **Blade**.

BLADE

Blade è un linguaggio di templating estremamente leggero che fornisce sintassi corte e comode per mostrare dati, per iterare sui dati e tanto altro

Quando si costruiscono applicazioni in questo modo le interazioni con la pagina ricevono in genere un documento HTML completamente nuovo dal server e l'intera pagina viene riprodotta dal browser.

```
<div>
    <?php foreach ($users as $user): ?>
        Hello, <?php echo $user-
>name; ?> <br />
    <?php endforeach; ?>
</div>
```

```
<div>
    @foreach ($users as $user)
        Hello, {{ $user->name }} <br
/>
    @endforeach
</div>
```

Ancora oggi, molte applicazioni possono essere perfettamente adatte a costruire il loro frontend in questo modo, utilizzando semplici modelli Blade.

Tuttavia, con le crescenti aspettative nei confronti delle applicazioni web, molti sviluppatori hanno sentito l'esigenza di costruire frontend più dinamici.

Alla luce di questo alcuni sviluppatori hanno deciso di iniziare a costruire il frontend delle loro applicazioni usando framework JS come Vue e React.

Altri hanno preferito rimanere conformi al linguaggio di backend e hanno sviluppato soluzioni che permettono la realizzazione di interfacce utente moderne usando lo stesso linguaggio di backend.

All'interno dell'ecosistema Laravel, la necessità di creare frontend moderni e dinamici usando prevalentemente PHP ha portato alla creazione di Laravel Livewire e Alpine.js

OTTIMIZZAZIONE

CONFIGURAZIONE DEL CACHING

In fase di deployment, accertarsi di lanciare il comando Artisan

```
php artisan config:cache
```

In modo da combinare tutti i file di configurazione di Laravel in un unico file che riduce il numero di movimenti all'interno del filesystem quando si caricano i valori di configurazione.

CACHING DEGLI EVENTS

Se l'applicazione sta utilizzando event discovery, è necessario memorizzare nella cache le mappature degli eventi e dei listeners.

```
php artisan event:cache
```

OTTIMIZZAZIONE

Quando si distribuisce l'applicazione in produzione, ci sono diversi file che dovrebbero essere memorizzati nella cache, inclusi quelli di configurazione, eventi, route e viste.

Laravel fornisce il comando optimize di Artisan che memorizza nella cache tutti questi file.

Questo comando dovrebbe fare parte del processo di distribuzione dell'applicazione.

```
php artisan optimize
```

Il metodo optimize:clear può essere utilizzato per rimuovere tutti i file di cache generati dal comando optimize così come tutte le chiavi memorizzate nel driver di cache predefinito

```
php artisan optimize:clear
```

CONFIGURAZIONE CACHING

In fase di deployment, accertarsi di lanciare il comando Artisan

```
php artisan config:cache
```

In modo da combinare tutti i file di configurazione di Laravel in un unico file memorizzato nella cache, riducendo il numero di accessi al filesystem per il caricamento dei valori di configurazione.

*Se si esegue il comando **config:cache** durante il processo di distribuzione, bisogna assicurarsi di chiamare la funzione **env** solo all'interno dei file di configurazione. Una volta che la configurazione è stata memorizzata nella cache, il file **.env** non verrà caricato e tutte le chiamate alla funzione **env** per le variabili definite in **.env** restituiranno **null**.*

CACHING DEGLI EVENTS

Bisognerebbe memorizzare in cache le mappature tra eventi e listener durante il processo di distribuzione.

```
php artisan event:cache
```

CACHING DELLE ROTTE

Se si sta realizzando un'applicazione con molte rotte, bisognerebbe accertarsi di lanciare il comando

```
php artisan route:cache
```

Questo comando riduce tutte le registrazioni delle rotte in un unico file memorizzato in cache, migliorando le performance della registrazione delle rotte quando se ne devono gestire centinaia.

CACHING DELLE VIEWS

```
php artisan view:cache
```

Questo comando precompila tutte le viste Blade così che non siano compilate on demand, migliorando le prestazioni di ogni request che ritorna una view.

DEBUG MODE

Le opzioni di debug all'interno del file di configurazione config/app.php determinano quante informazioni su un errore vengono mostrate all'utente.

Di default è impostato per rispettare il valore della variabile d'ambiente `APP_DEBUG` che si trova all'interno del file .env dell'applicazione.

In ambiente di produzione questo valore dovrebbe sempre essere impostato su false.

Se fosse settato su true, il rischio è di esporre importanti valori di configurazione agli utenti finali dell'app.

LA ROUTE HEALTH

Laravel include una **rotta built-in di health check che permette di monitorare lo stato dell'app**. Di default, la route è disponibile all'URL /up e restituirà una risposta HTTP 200 se l'app si è avviata senza errori. In caso contrario, restituirà una risposta HTTP 500.

L'URI della rotta può essere configurata nel file bootstrap/app.php

```
->withRouting(  
    web: __DIR__.'/../routes/web.php',  
    commands: __DIR__.'/../routes/console.php',  
    health: '/up',  
    health: '/status',  
)
```

Quando vengono effettuate chiamate a questa rotta, Laravel invia anche l'evento `Illuminate\Foundation\Events\DiagnosisHealth` permettendo di eseguire altri controlli circa la salute dell'applicazione.

Nel listener di questo evento, si può verificare lo stato del database, della cache o di altri componenti.

Se si rileva un problema, si può lanciare un'eccezione nel listener per segnalare l'errore.

curl <http://localhost:8000/up>

Oppure da browser

<http://localhost:8000/up>



ARCHITECTURE CONCEPTS

COSA SONO LE RICHIESTE HTTP

Le richieste HTTP sono messaggi inviati al server dal client per iniziare una determinata azione.

Le richieste hanno una struttura ben precisa e sono caratterizzate da un metodo (o verbo) che permette di distinguere il tipo di azione richiesta.

STRUTTURA DELLE RICHIESTE HTTP

La richiesta HTTP è composta da:

- Un **METODO** che indica l'azione richiesta
- Una **RISORSA** verso cui si sta eseguendo l'azione
- Potrebbero avere uno o più **HEADER** (facoltativo) che caratterizzano la richiesta
- Un **BODY** (opzionale) che contiene i dati da inviare alla risorsa

ESEMPIO DI RICHIESTA HTTP:

GET /guide/laravel/index.php HTTP/2 ← **Indicazione del metodo e della risorsa (guide/laravel/index.php)**

Host: https://example.com

Accept-Encoding: gzip, deflate, br

Accept-Language: it-IT,it;q=0.9,en-US;q=0.8,en;q=0.7

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like

Gecko) Chrome/108.0.0.0 Safari/537.36

HEADERS nella forma **Nome-Header:valore**
L'header Host permette di sapere verso quale host indirizzare la richiesta.
Non è presente il body.

COSA SONO LE RICHIESTE HTTP – i metodi

I **metodi** delle richieste HTTP identificano **l'azione desiderata da compiere sulla risorsa**. Possono essere nomi ma sono di solito indicati come verbi HTTP.

Il set completo dei metodo HTTP è pensato per permettere ai client di caricare o modificare risorse sul server.

I **principali metodi HTTP** sono:

- **GET**: richiede la risorsa indicata che viene **servita dal server come body** della risposta. Essendo usata per richiedere dati al server, non dovrebbe contenere un body
- **HEAD**: stesso effetto della GET ma **la risposta del server è senza il body**. Può essere **utile da eseguire prima della GET per ottenere informazioni sulla risorsa** dal server come, ad esempio, la dimensione della risorsa fornite nella risposta con l'header **Content-Length**
- **POST**: **invia dati al server** usando la risorsa indicata. Viene generalmente inviata tramite form HTML e comporta una **creazione** sul server. In questo caso il tipo di contenuto (valori possibili sono: application/x-www-form-urlencoded e multipart/form-data) viene gestito dall'elemento <form>.
- **PUT**: **crea una nuova risorsa o sostituisce la risorsa indicata** con il **body della richiesta**. È un metodo idempotente: chiamarlo una o più volte successivamente ha lo stesso effetto, mentre successive richieste POST identiche possono aggiungere risorse (ad esempio ad ogni POST su un server e-commerce creo un nuovo ordine).
- **DELETE**: **elimina** la risorsa indicata
- **PATCH**: **applica una modifica parziale** alla risorsa. Come il metodo POST può avere effetti aggiuntivi ogni volta che è chiamata su una risorsa. A differenza del metodo PUT, **nel body possono essere indicate «istruzioni» per modificare la risorsa**. Non tutti i server web implementano questo metodo per le varie risorse
- **OPTIONS**: chiede le **opzioni di comunicazione per la risorsa** indicata (ad esempio il server può rispondere quali altri metodi sono applicabili alla risorsa).

CICLO DI VITA DELLA REQUEST – Request Lifecycle

1- FIRST STEP

Il punto di ingresso di tutte le requests ad un'applicazione Laravel è il file **public/index.php**.

Tutte le richieste vengono dirette a questo file dal Web Server.

Il file index.php non contiene molto codice, è piuttosto un punto di inizio per il caricamento di tutto il framework.

Carica la definizione dell'autoloader di Composer e poi recupera un'istanza dell'applicazione da **bootstrap/app.php**

La prima azione svolta da Laravel stesso è la creazione di un'istanza dell'applicazione /service container

```
#!/usr/bin/env php
<?php

use Illuminate\Foundation\Application;
use Symfony\Component\Console\Input\ArgvInput;

define('LARAVEL_START', microtime(true));

// Register the Composer autoloader...
require __DIR__.'/vendor/autoload.php';

// Bootstrap Laravel and handle the command...
/** @var Application $app */
$app = require_once __DIR__.'/bootstrap/app.php';

$status = $app->handleCommand(new ArgvInput);

exit($status);
```

2 – HTTP / CONSOLE KERNEL

Dopo un primo caricamento del framework, la request è inviata o al **kernel HTTP** o al **kernel console**, a seconda del tipo di richiesta.

Questi due kernel servono come **luogo centrale in cui tutte le richieste passano**.

L'HTTP kernel è un'istanza di **Illuminate\Foundation\Http\Kernel** e definisce un array di «bootstrappers» che verranno eseguiti prima della request.

Questi «bootstrappers» configurano la **gestione degli errori**, il **logging**, rilevano **l'ambiente dell'applicazione** ed eseguono altre azioni che devono essere svolte prima che la richiesta sia effettivamente gestita.

L'HTTP Kernel è anche responsabile di far passare la richiesta attraverso lo stack di **Middleware** dell'applicazione. Questi Middleware gestiscono la lettura e scrittura della sessione http, determinano se l'applicazione è in modalità manutenzione, verificano il token CSRF (Cross-site Request Forgery) e altro.

Il metodo handle del kernel http è molto semplice:

Riceve una Request e ritorna una Response.

in **Laravel 12**:

non esiste più config/app.php con l'array dei provider.

- **Middleware & CSRF**: si configurano in `bootstrap/app.php` con `->withMiddleware(...)` (append/prepend ai gruppi `web/api`, alias, esclusioni CSRF, ecc.). [Laravel+1Stack OverflowLaravel News](#)
- **Scheduling / Console**: niente `App\Console\Kernel`; usi `routes/console.php` (o `->withSchedule(...)` dove previsto) per definire i task con il `Schedule` facade / comandi Artisan.
- Per installare app completa di kernel in laravel 11 (non più nel 12)
- `php artisan install:full`

```
bootstrap/app.php
aggiungi il tuo provider manualmente:
$app->register(App\Providers\MioServiceProvider::class);
```

3- SERVICE PROVIDERS

Una delle più importanti azioni del kernel di avvio è il **caricamento dei providers dei servizi** per l'applicazione.

I service providers sono responsabili **dell'avvio di tutti i vari componenti del framework** come il **database**, le **code**, la **validazione** e i **componenti di routing**.

Laravel ciclerà all'interno di una lista di providers e istanzierà ognuno di essi.

Dopo aver istanziato i providers, verrà chiamato il metodo register su tutti i provider.

Una volta registrati tutti i provider, verrà chiamato il metodo boot su ognuno di essi.

Essenzialmente, **ogni maggiore funzionalità di Laravel è avviata e configurata da un service provider**. Questo rende i service providers uno degli aspetti fondamentali dell'intero processo di avvio di Laravel.

Laravel usa internamente dozzine di service providers. La lista di service providers personalizzati o di terze parti si trova nel file **bootstrap/providers.php** mentre i service providers definiti dall'utente sono memorizzati nella directory **app/Providers**

Laravel 11 e 12 la struttura è stata semplificata:

- L'**elenco dei service provider** non è più in un file dedicato (**bootstrap/providers.php** o **config/app.php** per quelli core), ma viene definito nel **bootstrap/app.php** tramite il metodo `->withProviders([...])`.
- I **service provider personalizzati** li crei comunque in **app/Providers/** (ad es. **App\Providers\RouteServiceProvider**), ma li registri manualmente in **bootstrap/app.php** se non sono caricati automaticamente.

4 – ROUTING

Una volta che l'applicazione è stata inizializzata e tutti i service providers sono stati registrati, la **Request sarà affidata al router**. Questo, a sua volta, **la indirizzerà verso una rotta o un controller, oltre ad eseguire qualsiasi specifico middleware della rotta**.

Il middleware fornisce un comodo meccanismo per il filtraggio o l'analisi delle richieste HTTP ricevute dall'app.

Ad esempio Laravel include **un middleware che consente di verificare se l'utente è autenticato**. In caso negativo, **il middleware reindirizzerà l'utente alla schermata di login**.

Se, al contrario, l'utente è già autenticato, il middleware consentirà alla request di procedere oltre nell'app.

Alcuni middleware sono assegnati a tutte le rotte dell'applicazione mentre altri sono assegnati solo a specifiche rotte o gruppi di esse.

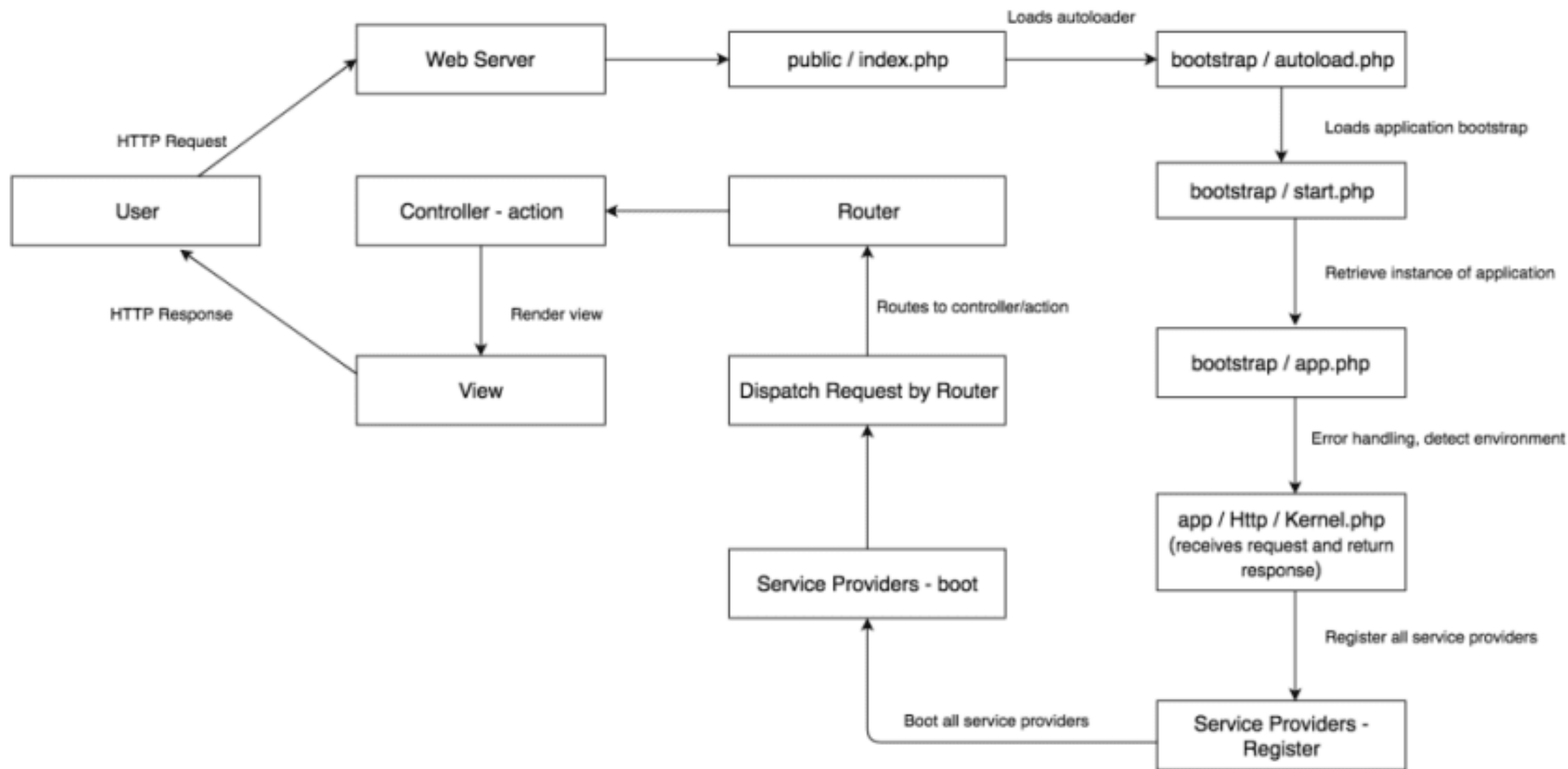
Se la richiesta passa tutti i middleware assegnati alla rotta corrispondente, il metodo associato alla rotta o al controller sarà eseguito e la risposta HTTP sarà ritornata attraverso la catena di middleware.

5-TERMINE: RESTITUZIONE RESPONSE

Una volta che la response è stata restituita a seconda del metodo della rotta o del controller, questa viaggia indietro attraverso il middleware della rotta dando all'applicazione la possibilità di modificare o esaminare la risposta in uscita.

Una volta che la risposta è ripassata attraverso il middleware, il metodo **handle** del kernel HTTP restituisce l'oggetto risposta all'**handleRequest** dell'istanza. A sua volta, questo metodo chiama il metodo **send** sulla risposta restituita. Il metodo **send** invia il contenuto della risposta al browser web dell'utente.

Request Life Cycle of Laravel



SERVICE CONTAINER

Il service container di Laravel è uno strumento potente per la gestione delle dipendenze* delle classi e per eseguire la dependency injection.

DEPENDANCY INJECTION significa che le dipendenze di una classe vengono "iniettate" nella classe stessa tramite il costruttore o, in alcuni casi, con i metodi "setter". Questo sistema ci permette di evitare di istanziare oggetti all'interno della classe.

Nell'esempio a fianco, UserController ha bisogno di recuperare gli utenti da una fonte di dati. Inietteremo quindi un service che è in grado di recuperarli senza dovere istanziare la classe.

***DIPENDENZE:** sono le altre classi, oggetti o servizi di cui una classe ha bisogno per funzionare correttamente.

Esempio: supponiamo di avere una classe OrderProcessor che gestisce degli ordini. Questa classe potrebbe aver bisogno di un Payment Gateway per elaborare i pagamenti. In questo caso, OrderProcessor dipende da PaymentGateway

```
class UserController extends Controller
{
    protected $userService;

    public function
    __construct(UserService $userService)
    {
        $this->userService =
        $userService;
    }

    public function index()
    {
        // Utilizzo del servizio utente
        $users = $this->userService-
        >getAllUsers();
        // ...
    }
}
```

FACADES

Le facades forniscono un'interfaccia «statica» delle classi che sono disponibili nel service container dell'applicazione.

Laravel viene fornito con molte facades che consentono l'accesso a quasi tutte le funzionalità di Laravel.

Tutte le facades di Laravel sono definite nel namespace `Illuminate\Support\Facades` quindi possiamo facilmente accedere ad esse nel seguente modo:

```
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Route;

Route::get('/cache', function () {
    return Cache::get('key');
});
```

Esempio da tinker:

```
Log::info('ciao')
Config::get('app.name')
Config::get('app.timezone')
URL::to('miosito')
```

```
<p>{{URL::current()}}</p>
```

HELPER FUNCTIONS

Per completare le facades, Laravel offre una varietà di «helper functions» globali che rendono ancora più semplice interagire con le più comuni funzionalità di Laravel.

Alcune delle helper functions più comuni sono view, response, url, config,...

Ad esempio, invece di usare la facade `Illuminate\Support\Facade\Response` per generare una risposta JSON, possiamo semplicemente usare la funzione `response()`.

Siccome le helper functions sono disponibili globalmente non c'è bisogno di importare alcuna classe per utilizzarle.

Esempio da tinker:

```
config('app.name')  
config('app.timezone')  
url('miosito')
```

Gli helper di Laravel sono funzioni globali che semplificano alcune operazioni comuni. La maggior parte degli helper in Laravel è definita come funzioni globali, ma alcuni sono accessibili tramite la sintassi della Facade come `Arr::sort($array)`.

Alcuni hanno delle sintassi simili alle Facade

```
{{ url()->current() }}
```

FACADES – quando usarle

Facades hanno molti benefici tra i quali quello di fornire una sintassi facile da ricordare che permette di utilizzare funzionalità di Laravel senza dover ricordare nomi di classi lunghissimi che dovrebbero essere configurate o iniettate manualmente.

Vanno però utilizzate con cautela.

Il rischio principale è che, non richiedendo injections, è facile che la classe continui a crescere e che usi molte facades in una singola classe.

Bisogna dunque prestare particolare attenzione, quando si usano facades, all'ampiezza della classe e fare in modo che il suo ambito di responsabilità rimanga limitato.

I NAMESPACES



- I namespaces permettono di **raggruppare classi, interfacce, funzioni e costanti** correlate tra loro al fine di evitare problemi di denominazione.

- In questo modo potranno **coesistere classi con lo stesso nome** inserite («incapsulate») in namespaces **differenti** (come accade per i file con lo stesso nome ma posizionati in directory differenti)

ESEMPIO:

In una directory «var/www/html/test» abbiamo il file «prova1.php» che contiene la classe Prova

```
class Prova{
    public function m1(){
        echo "ciao dal metodo ".__METHOD__." della classe <b>".__CLASS__;
    }
}
```

Avremo poi un file prova2.php contenente anch'esso una classe «Prova» analoga alla precedente.
Abbiamo poi un index.php in cui vengono inclusi entrambi i file

```
<?php
include "prova1.php";
include "prova2.php";
?>
```

Il risultato sarà un **fatal error** perché i due file hanno lo stesso nome.

```
Fatal error: Cannot declare class Prova, because the name is already in use in /var/www/html/test/prova2.php on line 2
```

- Pensiamo alle due classi come a due file di un filesystem: due file con lo stesso nome non possono coesistere nella stessa directory. Devono essere in directory differenti per mantenere lo stesso nome
- I namespace consentono di superare questo problema di omonimia introducendo un sistema simile a quello che può essere definito quasi come un «filesystem digitale». All'interno dei file che contengono le classi dobbiamo indicare dove le classi contenute al loro interno sono localizzate in questo filesystem virtuale.
- Questo è possibile inserendo la keyword **namespace** seguita dal nome che viene assegnato a questa directory (spazio).

File prova1.php

```
namespace Spazio1;
class Prova{
    public function m1(){
        echo "ciao dal metodo ".__METHOD__." della classe <b>".__CLASS__;
    }
}
```

La classe Prova è definita
all'interno del namespace
Spazio1

File prova2.php

La classe Prova è definita
all'interno del namespace
Spazio2

```
namespace Spazio2;
class Prova(){
    public function m1(){
        echo "ciao dal metodo ".__METHOD__." della classe <b>".__CLASS__;
    }
}
```

- Se rilanciamo index.php noteremo che l'errore è sparito

- Se, all'interno di index.php, creiamo un'istanza della classe «Prova» e lanciamo la index otterremo un errore fatale

```
Fatal error: Uncaught Error: Class 'Prova' not found in /var/www/html/test/index.php:5 Stack trace: #0 {main} thrown in /var/www/html/test/index.php on line 5
```

- Questo perché nel file delle classi abbiamo dichiarato che risiedono in uno specifico namespace. Se vogliamo istanziare un oggetto della classe Prova dovremo **specificare in quale namespace si trova la classe Prova**.

```
$obj=new Spazio1\Prova;
```

Se vogliamo istanziare la classe Prova che si trova nel namespace spazio1

```
$obj=new Spazio2\Prova;
```

Se vogliamo istanziare la classe Prova che si trova nel namespace spazio2

- Nel file index.php andiamo ad istanziare due oggetti delle due classi, specificando il relativo namespace

```
include "prova1.php";
include "prova2.php";

$obj=new Spazio1\Prova;
$obj2=new Spazio2\Prova;

$obj->m1();
echo "<br>";
$obj2->m1();
```

Se lanciamo la index il risultato sarà:

```
ciao dal metodo Spazio1\Prova::m1 della classe Spazio1\Prova
ciao dal metodo Spazio2\Prova::m1 della classe Spazio2\Prova
```


- Se pensiamo a tutti quei casi in cui il nostro sito o applicazione hanno già delle librerie con classi ed aggiungiamo una nuova libreria che contiene classi con nomi già presenti nelle librerie: i namespaces consentono di superare i conflitti di denominazione.
- Permettono inoltre a più persone di lavorare contemporaneamente allo stesso progetto senza il problema che vengano dichiarate classi con lo stesso nome.
- Abbiamo equiparato i namespaces a directories e, come queste, anche i namespaces possono avere un'alberatura come, ad esempio, App\Backend\User\Sicurezza\Controlli

VINCOLI NELL'UTILIZZO DEI NAMESPACE

- I namespaces vanno inseriti subito dopo l'apertura del codice php, **nessun codice deve precederli, tranne il caso di commenti ed eventuali declare come nell'esempio seguente:**

```
<?php
declare(encoding='UTF-8');
namespace Spazio1;
class Prova(){
    public function m1(){
        echo "ciao dal metodo ".__METHOD__." della classe <b>".__CLASS__;
    }
}
?>
```

- I nomi dei namespace **non possono iniziare con numeri** e non possono contenere keywords del linguaggio php come «Class» o «Function»

GLOBAL SPACE E UTILIZZO DEL NAMESPACE

Quando in un file non viene definito un namespace, si dice che la classe è inserita in uno spazio globale, o **GLOBAL SPACE**.

ESEMPIO: abbiamo un file statistiche.php in cui è stata definita la classe «statistiche» e ne è stata lanciata un'istanza

```
class Statistiche{
    public function myStat() {
        echo "ciao";
    }
}

$obj = new Statistiche;
```

Creiamo un secondo file index.php dove indichiamo il namespace «App\Backend» e che include il file statistiche.php
Creiamo un'istanza dell'oggetto di classe «Statistiche»

```
<?php
namespace App\Backend;
require_once('statistiche.php');
$obj = new Statistiche;
$obj->myStat();
?>
```

Se proviamo a lanciare questo file otterremo un errore fatale: la classe Statistiche non è stata trovata perché in questo file ci troviamo nello spazio «App/Backend» mentre la classe Statistiche si trova nella root «\».

```
Fatal error: Class 'App\Backend\Statistiche' not found in /var/www/html/test/index.php on line 5
```

Per accedere ad una classe definita in uno spazio globale dobbiamo indicare un **namespace separator** prima della classe, cioè una backslash (dobbiamo in pratica entrare nella root).

Aggiungiamo ora il namespace separator nel file index.php e così possiamo accedere alla classe «Statistiche»:

```
<?php
namespace App\Backend;
require_once('statistiche.php');
$obj = new \Statistiche;
$obj->myStat();
?>
```

Rilanciando il file otterremo la risposta «ciao» del metodo myStat.

In questo caso abbiamo fatto riferimento alla classe Statistiche con un **FULLY QUALIFIED NAME**, cioè utilizzando un namespace separator all'inizio. È come utilizzare un percorso assoluto. Altri esempi di fully qualified names sono:

\Esempio

\Esempio\Prova

\Esempio\Prova\Livello

È possibile fare riferimento al nome di una classe in altri due modi:

- Con un **QUALIFIED NAME** (nome qualificato): paragonabile al **percorso relativo rispetto al namespace** in cui mi trovo e **contiene almeno un namespace**, ma non all'inizio
- Con un **UNQUALIFIED NAME** (non contiene namespace)

IL TYPE HINTING (definire il tipo del dato)

- Il Type Hinting («suggerimento del tipo») è una funzionalità introdotta dalla versione 5 di Php che consente di specificare il tipo di oggetto/dato passato come argomento, ad una funzione o ad un metodo. Stiamo quindi dicendo al metodo che tipo di valore stiamo passando, come argomento, ad una funzione o al metodo stesso.
- A partire da php 5 è possibile dichiarare che tipo di dato viene assegnato ad una variabile e passato al metodo o alla funzione.
- Forzando, con il type hinting, il tipo di dato in ingresso, PHP considererà valido solo il dato della tipologia indicata, in caso contrario avremo un «fatal error».

IL TYPE HINTING

1. **int**: per numeri interi.
2. **float**: per numeri in virgola mobile (float).
3. **string**: per stringhe di testo.
4. **bool**: per valori booleani (true/false).
5. **array**: per array.
6. **callable**: per oggetti chiamabili (funzioni, metodi di oggetti, ecc.).
7. **object**: per istanze di una classe specifica.
8. **iterable**: per oggetti iterabili (array o oggetti che implementano l'interfaccia `\Traversable`).
9. **?Tipo**: per indicare che il parametro può essere di tipo Tipo oppure null (introdotto in PHP 7.1).
10. **self**: per fare riferimento alla stessa classe della classe che dichiara il type hinting.
11. **parent**: per fare riferimento alla classe genitore di quella che dichiara il type hinting.

```
function stampaIntero(int $numero) {  
    echo $numero;  
}
```

```
stampaIntero(5); // Questo funzionerà  
correttamente
```

```
stampaIntero("hello"); // Questo genererà un  
errore poiché il tipo di dato passato non è un intero
```

Artisan

- Artisan è l'interfaccia a riga di comando inclusa con Laravel.
- Artisan esiste alla radice dell'applicazione come script artisan e fornisce una serie di comandi utili che possono aiutare durante la creazione dell'applicazione. Per visualizzare un elenco di tutti i comandi Artisan disponibili, puoi utilizzare il comando list:

```
php artisan list
```

- Ogni comando include anche un "help" screen che mostra e descrive gli argomenti e le opzioni disponibili per il comando

```
php artisan help migrate
```

- Laravel Tinker è un potente REPL (Read-Eval-Print-Loop) per Laravel framework, sviluppato da PsySH package. È un ambiente di programmazione interattivo che prende gli input dell'utente (Read), li valuta (Eval), e poi ritorna il risultato (Print), ripetendo questo procedimento in un ciclo (Loop). Tinker permette di interagire con l'applicazione Laravel via riga di comando, rendendo più semplici il test del codice, le query al database e il debug.

```
php artisan tinker
```

<https://laravel.com/docs/12.x/artisan>

POSTMAN headers

- Gli headers più comuni di una richiesta

POST

▼

 http://localhost:8000/api/get-token

Params Authorization Headers (12) Body ● Pre-request Script Tests Settings

Headers

8 hidden

	Key	Value
☒	Content-Type	application/json
☒	Accept	application/json
☒	User-Agent	Postman/7.36.0
☒	Cache-Control	no-cache, max-age=0
	Key	Value

POSTMAN

csrf

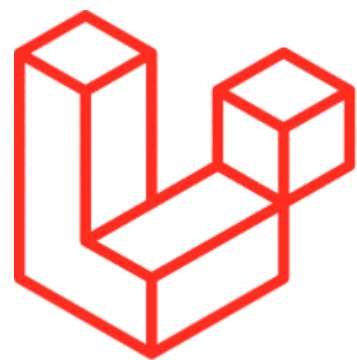
- Per evitare l'errore 419 passare alle chiamate POST / PUT / PATCH

POST ▼ http://127.0.0.1:8000/product

Params Authorization Headers (10) Body ● Pre-request Script Tests Settings

Headers 👁 8 hidden

	Key	Value
☒	X-CSRF-TOKEN	h3lcBq0AKu8LJv41EC8jCNYIWPfKSRYwPV85YVjb
☐	cont	
	Key	Value



Laravel

LE BASI

Il routing

ROUTING

Basic routing

Una rotta (Route) consente di configurare l'applicazione per **rispondere ad una chiamata per una determinata URI** in modo che possa rispondere a richieste di risorse che non sono necessariamente file specifici presenti sul server.

La route più semplice che si può definire in Laravel è:

```
use Illuminate\Support\Facades\Route;

Route::get('/greeting', function () {
    return 'Hello World';
});
```

Essa è composta da:

- **Metodo statico** della classe Route, generalmente legato al metodo HTTP della richiesta. In questo caso GET
- La stringa che rappresenta il **path o parte del path** della URI della richiesta (/greeting)
- Una **callback**, ovvero closure (function () {...}) che definisce la risposta che verrà restituita

Queste definizioni di route vanno aggiunte al file **routes/web.php** per le richieste legate alle pagine web. È importante ricordare che **l'ordine in cui le rotte vengono definite è fondamentale per il meccanismo di match**: la **prima route definita** che può gestire la richiesta arrivata sarà quella che, effettivamente, **restituirà la risposta**, anche se dovesse essercene definita una più specifica più avanti nel file. Per ogni richiesta ricevuta, infatti, Laravel scansiona le rotte registrate fino a trovarne una in grado di poter gestire il metodo e il path della richiesta.

ROUTING

Basic routing

- I FILE PREDEFINITI DELLE ROTTE

Tutte le rotte Laravel sono definite nei files che si trovano all'interno della **cartella routes**.

Questi files sono **caricati automaticamente** usando la configurazione specificata nel file `bootstrap/app.php`.

WEB ROUTES

Il file **routes/web.php** definisce le rotte per l'interfaccia web. A queste rotte viene assegnato il **gruppo middleware web** che fornisce funzionalità quali lo stato della sessione (*session state*) e la protezione CSRF (*Cross-Site Request Forgery*).

Si può accedere alle rotte definite in `web.php` **inserendo l'URL della rotta nel browser**.

```
use App\Http\Controllers\UserController;

Route::get('/user', [UserController::class, 'index']);
```

Si può accedere a questa rotta digitando <http://example.com/user> nella barra degli indirizzi del browser

API ROUTES

Nel caso un'applicazione offra anche API stateless, si può abilitare il routing delle API con il comando Artisan:

```
php artisan install:api
```

Questo comando installa Laravel Sanctum che fornisce un robusto, ma semplice, sistema di autenticazione API a token.

Il comando, inoltre, crea il file `routes/api.php`

ROUTING

Basic routing

Le rotte in **routes/api.php** sono **stateless** e hanno assegnato il **middleware api**.

Inoltre, il **prefisso /api** dell'URI è applicato automaticamente in modo che non debba essere aggiunto manualmente ad ogni rotta del file.

Si possono **modificare il prefisso e altre opzioni** del gruppo di rotte **nel file bootstrap/app.php**

```
1  →withRouting(  
2      api: __DIR__.'/../routes/api.php',  
3      apiPrefix: 'api/admin',  
4      // ...  
5  )
```

ROUTING

Basic routing

- I METODI DEL ROUTER

Il router permette di registrare rotte che rispondono ad ogni verbo HTTP. Le rotte più immediate che si possono registrare sono:

```
1 Route::get($uri, $callback);
2 Route::post($uri, $callback);
3 Route::put($uri, $callback);
4 Route::patch($uri, $callback);
5 Route::delete($uri, $callback);
6 Route::options($uri, $callback);
```

In caso si abbia la necessità di registrare una **rotta che risponde a più metodi HTTP**, per ottenere la medesima risposta, si può usare il metodo **match** oppure si può registrare una rotta che risponde a tutti i verbi HTTP usando il metodo **any**

```
Route::match(['get', 'post'], '/', function () {
    // ... Risposta comune per metodi GET e POST
});

Route::any('/', function () {
    // ... Risposta comune per OGNI RICHIESTA
});
```

!!! L'ORDINE È FONDAMENTALE !!!

Quando si definiscono **rotte multiple** che condividono lo stesso URI, le rotte che usano i metodi **get, post, put, patch, delete e options** dovrebbero essere definite **prima delle rotte che usano i metodi any, match e redirect.**

Questo assicura che la richiesta in entrata sia gestita dalla rotta corretta.

ROUTING

Basic routing

- **DEPENDENCY INJECTION**

Nel caso fosse necessario utilizzare una **dipendenza esterna nella closure** di una route, è possibile sfruttare la **dependency injection** di Laravel.

Esempio: vogliamo che la risposta alla rotta /hello contenga il nome passato opzionalmente come query string alla URI.

Quindi:

- Se l'utente richiede GET/hello la risposta sarà «Hello Laravel»
- Se viene richiesto GET/hello?name=Foo la risposta sarà «Hello Foo»

```
use Illuminate\Http\Request;
Route::get('hello', function (Request $request) {
    $name = $request->query('name');
    if ($name == '') {
        $name = 'Laravel';
    }
    return 'Hello $name';
});
```

La funzione associata alla route dichiara di avere una **dipendenza aggiuntiva**, specificando un parametro con il **type-hinting (dichiarazione del tipo di dati)**. Nel momento in cui la funzione verrà eseguita, Laravel inietterà la richiesta arrivata come tipo `Illuminate\Http\Request`. Sarà quindi possibile utilizzare questo oggetto all'interno della callback, per esempio per estrarre dati della richiesta necessari ad elaborare la risposta.

NOTA: STRUTTURA DELL URI

`<scheme>://<host><path>[?<query>]`

La URI

`https://www.example.com/path/to/foo?param=value&option=none` è dunque composta da:

Schema = https

Host = www.example.com

Path = path/to/foo

Query = param=value&option=none

Una URI ha sempre almeno un **path(/)** mentre la **query** è opzionale

ROUTING

Basic routing

- REDIRECT ROUTES

Per definire una rotta che redireziona ad un altro URI, si può utilizzare il metodo `Route::redirect`

```
Route::redirect('/here', '/there');
```

Di default il metodo redirect ritorna uno **status code 302**. Si può **personalizzare** lo status code indicandolo come **terzo parametro** opzionale del metodo redirect

```
Route::redirect('/here', '/there', 301);
```

Si può anche utilizzare il metodo **Route::permanentRedirect** per redirezioni permanenti. Esso ritorna uno status code 301:

```
Route::permanentRedirect('/here', '/there');
```

I parametri **destination** e **status** di una rotta sono riservati a Laravel e non possono essere usati nelle rotte di redirect

Esempio redirect

```
• Route::redirect('/news/2', '/news-new/2', 302);  
• Con parametri:  
• Route::get('/news-new/{id}', function(Request $request){  
•     return view('message')->with(['message'=>'news n.' .  
$request->segment(2)]);  
• });  
• Route::get('/news2/{id}', function(Request $request, int  
$id){  
•     return view('message')->with(['message'=>'news n.' .  
$id]);  
• });
```


ROUTING

Basic routing

- ROTTE DELLA VIEW

Se una rotta deve solamente ritornare una view, si può utilizzare il metodo **Route::view**.

Anche questo metodo costituisce una scorciatoia che permette di non dover definire tutta una rotta o un controller.

Come primo parametro accetta un URI e il nome di una view come secondo. Inoltre, come terzo argomento, può essere opzionalmente passato alla view un array di dati

```
Route::view('/welcome', 'welcome');  
  
Route::view('/welcome', 'welcome', ['name' => 'Taylor']);
```

Nelle rotte delle views, i seguenti parametri sono riservati a Laravel e non possono essere utilizzati: view, data, status, headers.

- CSRF PROTECTION

Ogni form HTML che punta a rotte POST, PUT, PATCH o DELETE definite nel file routes/web.php dovrebbero includere un campo per il token CSRF. In caso contrario la richiesta sarà rifiutata.

```
<form method="POST" action="/profile">  
    @csrf  
    ...  
</form>
```

ROUTES PARAMETRICHE

Laravel permette di gestire con una singola rotta URI, rotte in cui sono presenti elementi variabili. Questi parametri vengono passati automaticamente al controller o alla funzione di callback associata alla rotta.

La possibilità di definire rotte con parti dinamiche del path della URI permette di:

- Creare URI più semplici rispetto a quelle con query string ed in linea con le buone pratiche SEO (per esempio <https://www.example.com/blog/45887> invece di <https://www.example.com/blog/post?id=45887>)
- Essere coerenti con le buone pratiche delle API RESTful
- Gestire porzioni intermedie del path, per esempio <https://www.example.com/blog/topic/lifestyle/posts/> e <https://www.example.com/blog/topic/comedy/posts/>

ROUTING

Route Parameters

• PARAMETRI RICHIESTI (Required Parameters)

Nel definire una rotta è possibile istruire Laravel affinché una porzione della URI sia considerata **variabile** e venga messa a **disposizione dei controller o delle funzioni di callback**.

```
Route::get('/user/{id}', function (string $id) {  
    return 'User '.$id;  
});
```

Rispetto ad una rotta statica (rotta che accetta solo un possibile path), si può notare che:

- La **parte variabile è compresa tra parentesi graffe {}**
- Nella funzione di callback è presente un parametro (**\$id**)

IMPORTANTE:

- la parte variabile del path dovrebbe contenere **solo caratteri alfabetici e underscore** (_)
- È possibile indicare **più parti variabili** che verranno poi **associate ai parametri della callback in base all'ordine** (non è quindi necessario usare gli stessi nomi)

I parametri della rotta saranno iniettati nella callback o nel controller della rotta a seconda del loro ordine.

```
Route::get('/posts/{post_id}/comments/{comment_id}',  
    function ($post, $commentId) {  
        return 'This is the comment '.$commentId.' for post '.$post;  
    });
```

ROUTING

Route Parameters

- PARAMETRI OPZIONALI

Qualora fosse necessario specificare un parametro che non è sempre presente nella URI (parametro opzionale) basterà **aggiungere un punto di domanda (?) dopo il nome del parametro** ed indicare un valore di default alla variabile corrispondente

```
Route::get('/user/{name?}', function (string $name = null) {  
    return $name;  
});  
  
Route::get('/user/{name?}', function (string $name = 'John') {  
    return $name;  
});
```

ROUTING

Route Parameters

• PARAMETRI E DEPENDENCY INJECTION

- La **Dependency Injection (DI)** è un meccanismo che Laravel usa per **fornirti automaticamente** oggetti o classi di cui hai bisogno in una funzione o controller.
- Laravel capisce cosa ti serve **guardando il tipo della variabile** nei parametri della funzione. Se Laravel può costruire l'oggetto, te lo "inietta" automaticamente.

Per utilizzare una dipendenza esterna nella funzione di callback di una route con parametri, è necessario **indicare i parametri con dependency injection prima dei parametri di rotta**

```
use Illuminate\Http\Request;

Route::get('/user/{id}', function (Request $request, string $id) {
    return 'User '.$id;
});
```

Parametro con dependency injection

Parametro di
rotta

ROUTING

Route Parameters

- VINCOLI REGULAR EXPRESSION e VALIDAZIONE

Nel caso sia necessario **validare il valore ricevuto come parametro**, si può concatenare il metodo **where** alla definizione della rotta indicando, come nome della where, il **nome del parametro** e un'espressione regolare di validazione. Nel caso la richiesta non rispetti i vincoli di pattern, viene ritornata una risposta HTTP 404

Ogni parte variabile della URI può avere la propria validazione

```
Route::get('/user/{name}', function (string $name) {  
    // ...  
})->where('name', '[A-Za-z]+');  
  
Route::get('/user/{id}', function (string $id) {  
    // ...  
})->where('id', '[0-9]+');  
  
Route::get('/user/{id}/{name}', function (string $id, string $name) {  
    // ...  
})->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

ROUTING

Route Parameters

Per comodità, per gli schemi di regular expressions più comuni esistono dei **metodi helper**.

```
Route::get('/user/{id}/{name}', function (string $id, string $name) {  
    // ...  
})->whereNumber('id')->whereAlpha('name');  
  
Route::get('/user/{name}', function (string $name) {  
    // ...  
})->whereAlphaNumeric('name');  
  
Route::get('/user/{id}', function (string $id) {  
    // ...  
})->whereUuid('id');  
  
Route::get('/user/{id}', function (string $id) {  
    //  
})->whereUlid('id');  
  
Route::get('/category/{category}', function (string $category) {  
    // ...  
})->whereIn('category', ['movie', 'song', 'painting']);
```

ROUTING

Route Parameters

ROUTING

Named Routes

Le **rotte nominate** (a cui viene assegnato un nome) permettono la **generazione di URLs o redirects per rotte specifiche**. Si può specificare il nome di una rotta concatenando il metodo **name** alla definizione della rotta.

```
Route::get('/user/profile', function () {  
    // ...  
})->name('profile');
```

Si può anche specificare nomi di rotta per le azioni del controller:

```
Route::get(  
    '/user/profile',  
    [UserProfileController::class, 'show']  
)->name('profile');
```

!!! I NOMI DELLE ROTTE DOVREBBERO SEMPRE ESSERE UNICI!!!

Dopo aver nominato una rotta

```
Route::get('posts/{id}', function($id){  
    return 'id='.$id;  
})->name ('route.name');  
?>
```

è possibile fare riferimento ad essa nel **front-end** in questo modo:

```
<a href="<? route('route.name', ['id'=> 5])?>"></a>
```

La funzione **route** è un helper di Laravel che consente di trasformare il nome nell'url della rotta.

I **VANTAGGI** di dare un nome alle rotte sono principalmente due:

- Possibilità di fare **riferimento a rotte anche complesse** in modo rapido
- **Se una rotta cambia, non si deve modificare anche il front-end**

ROUTING

Named Routes

GENERAZIONE DI ROTTE PER ROTTE NOMINATE

Una volta assegnato un nome ad una data rotta, si può **utilizzare il nome della rotta quando si generano URLs o redirects** attraverso le funzioni helper **route** e **redirect**

```
Route::get('/user/{id}/profile', function (string $id) {  
    // ...  
})->name('profile');  
  
$url = route('profile', ['id' => 1]);
```

Se si passano parametri aggiuntivi nell'array, queste coppie chiave/valore saranno automaticamente aggiunte nella query string dell'URL generato

```
// Generating URLs...  
$url = route('profile');  
  
// Generating Redirects...  
return redirect()->route('profile');  
  
return to_route('profile');
```

Se la rotta ha dei **parametri**, questi vengono passati come **secondo argomento della funzione route**. Questi parametri saranno automaticamente inseriti nell'URL generato e nella loro esatta posizione.

```
Route::get('/user/{id}/profile', function (string $id) {  
    // ...  
})->name('profile');  
  
$url = route('profile', ['id' => 1, 'photos' => 'yes']);  
  
// /user/1/profile?photos=yes
```

ROUTING

Named Routes

CONTROLLARE LA ROTTA CORRENTE

Per determinare se la richiesta corrente è stata indirizzata ad una determinata rotta nominata, si può usare il metodo **named** sull'istanza di una Route. Ad esempio, si può controllare il nome della rotta corrente da un route middleware

```
use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

/**
 * Handle an incoming request.
 *
 * @param \Closure(\Illuminate\Http\Request):
 * (\Symfony\Component\HttpFoundation\Response) $next
 */
public function handle(Request $request, Closure $next): Response
{
    if ($request->route()->named('profile')) {
        // ...
    }

    return $next($request);
}
```

GRUPPI DI ROTTE

I gruppi di rotte permettono di **condividere attributi**, come i middleware, tra un numero variabile di rotte senza il bisogno di definirli per ogni singola rotta.

I gruppi annidati cercano di «unire» gli attributi con quelli del gruppo genitore: I middleware e le condizioni **where** sono unificati (merged) mentre i nomi ed i prefissi sono aggiunti (appended).

I delimitatori del namespace e gli slashes nei prefissi dell'URI sono automaticamente aggiunti dove necessario.

ROUTING

Gruppi di rotte

- **MIDDLEWARE**

Per assegnare middleware ad un gruppo di rotte si usa il metodo **middleware** prima di definire il gruppo.
I middleware **vengono eseguiti nell'ordine in cui sono elencati nell'array**:

```
Route::middleware(['first', 'second'])->group(function () {  
    Route::get('/', function () {  
        // Uses first & second middleware...  
    });  
  
    Route::get('/user/profile', function () {  
        // Uses first & second middleware...  
    });  
});
```

ESEMPIO:

Nell'esempio viene creato un gruppo di rotte con due middleware: auth (built-in. Verifica se l'utente è autenticato) e is_admin (not built-in. Verifica il ruolo dell'utente).
L'utente potrà accedere alle rotte del gruppo solo se supera entrambi i middleware.

```
use Illuminate\Support\Facades\Route;  
  
Route::middleware(['auth', 'is_admin'])->group(function () {  
  
    Route::get('/admin/dashboard', function () {  
        return "Benvenuto nella dashboard dell'admin!";  
    });  
  
    Route::get('/admin/users', function () {  
        return "Lista degli utenti amministrabili.";  
    });  
  
    Route::get('/admin/settings', function () {  
        return "Pagina delle impostazioni dell'admin.";  
    });  
});
```

ROUTING

Gruppi di rotte

- **CONTROLLERS**

Se un gruppo di rotte utilizza lo stesso controller, si può utilizzare il metodo `controller` per definire il controller comune a tutte le rotte all'interno del gruppo. Poi, nel definire le singole rotte, sarà necessario specificare solo il metodo del controller che viene richiamato.

```
use App\Http\Controllers\OrderController;

Route::controller(OrderController::class)→group(function () {
    Route::get('/orders/{id}', 'show');
    Route::post('/orders', 'store');
});
```

ROUTING
Gruppi di
rotte

- **PREFISSI DELLE ROTTE**

Il metodo **prefix** può essere utilizzato per aggiungere un prefisso ad ogni rotta nel gruppo con un dato URI. Si potrebbe, ad esempio, voler aggiungere il prefisso 'admin' a tutte le URI nel gruppo.

```
Route::prefix('admin')->group(function () {  
    Route::get('/users', function () {  
        // Matches The "/admin/users" URL  
    });  
});
```

Si pensi al caso in cui si devono gestire URI con path come /user/orders e /user/profile. Si possono raggruppare insieme usando il metodo prefix:

```
Route::prefix('/user')->group(function () {  
    Route::get('/profile', function () {  
        // risposta per l'URI "/user/profile"  
    });  
  
    Route::get('/orders', function () {  
        // risposta per l'URI "/user/orders" URL  
    });  
});
```

ROUTING

Gruppi di rotte

- **PREFISSI DEL NOME DELLE ROTTE**

Il metodo **name** può essere utilizzato per aggiungere una stringa ad ogni nome di rotta nel gruppo. Si potrebbe, ad esempio, voler aggiungere il prefisso 'admin' ai nomi di tutte le rotte nel gruppo. La stringa è aggiunta al nome della rotta esattamente così come è specificata, bisogna quindi **essere certi di inserire il carattere finale .** nel prefisso

```
Route::name('admin.')->group(function () {  
    Route::get('/users', function () {  
        // Route assigned name "admin.users"..  
    }->name('users');  
});
```

ROUTING
Gruppi di
rotte

ROUTING

Route Model Binding

IMPLICIT BINDING

Quando si passa l'ID di un modello ad una rotta o all'azione di un controller, bisognerebbe eseguire una query sul database per recuperare il modello corrispondente all'ID.

Ad esempio:

```
Route::get('/users/{id}', function ($id) {  
    $user = App\Models\User::findOrFail($id);  
    return $user;  
});
```

Il Route Model Binding di Laravel semplifica il procedimento iniettando automaticamente le istanze dei modelli direttamente nelle rotte.

BINDING IMPLICITO

Laravel risolve automaticamente i modelli Eloquent definiti nelle rotte o nelle azioni del controller quando il nome della variabile con type-hint (User \$user) corrisponde al nome di un segmento della rotta.

```
1 use App\Models\User;  
2  
3 Route::get('/users/{user}', function (User $user) {  
4     return $user->email;  
5 });
```

Poiché la variabile \$user è dichiarata come modello Eloquent App\Models\User ed il nome della variabile corrisponde al segmento {user} dell'URI, Laravel inietterà automaticamente l'istanza del modello che ha ID corrispondente al valore presente nell'URI della richiesta.

Se nel database non viene trovata alcuna istanza corrispondente, verrà generata una risposta 404.

ROUTING

Route Model Binding

IMPLICIT BINDING

Il binding implicito è possibile anche nei **metodi del controller**.

Anche in questo caso, il segmento dell'URI {user} deve corrispondere alla variabile \$user nel controller, dichiarata con il type-hint `App\Models\User`

```
1 use App\Http\Controllers\UserController;
2 use App\Models\User;
3
4 // Route definition...
5 Route::get('/users/{user}', [UserController::class, 'show']);
6
7 // Controller method definition...
8 public function show(User $user)
9 {
10     return view('user.profile', ['user' => $user]);
11 }
```

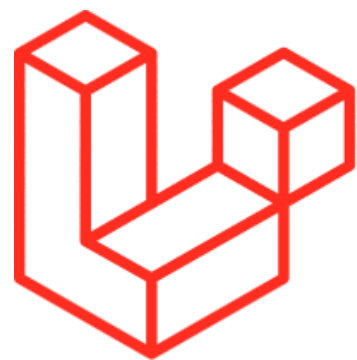
PERSONALIZZARE LA CHIAVE

Si possono risolvere i modelli Eloquent usando colonne differenti dall'id. Per farlo, basta specificare la colonna nella definizione dei parametri della rotta.

```
1 use App\Models\Post;
2
3 Route::get('/posts/{post:slug}', function (Post $post) {
4     return $post;
5 });
```

Se si vuole che il model binding usi sempre una colonna diversa dall'id si può sovrascrivere il metodo **getRouteKeyName** nel modello Eloquent

```
1 /**
2  * Get the route key for the model.
3  */
4 public function getRouteKeyName(): string
5 {
6     return 'slug';
7 }
```



Laravel

LE BASI

I middleware

MIDDLEWARE

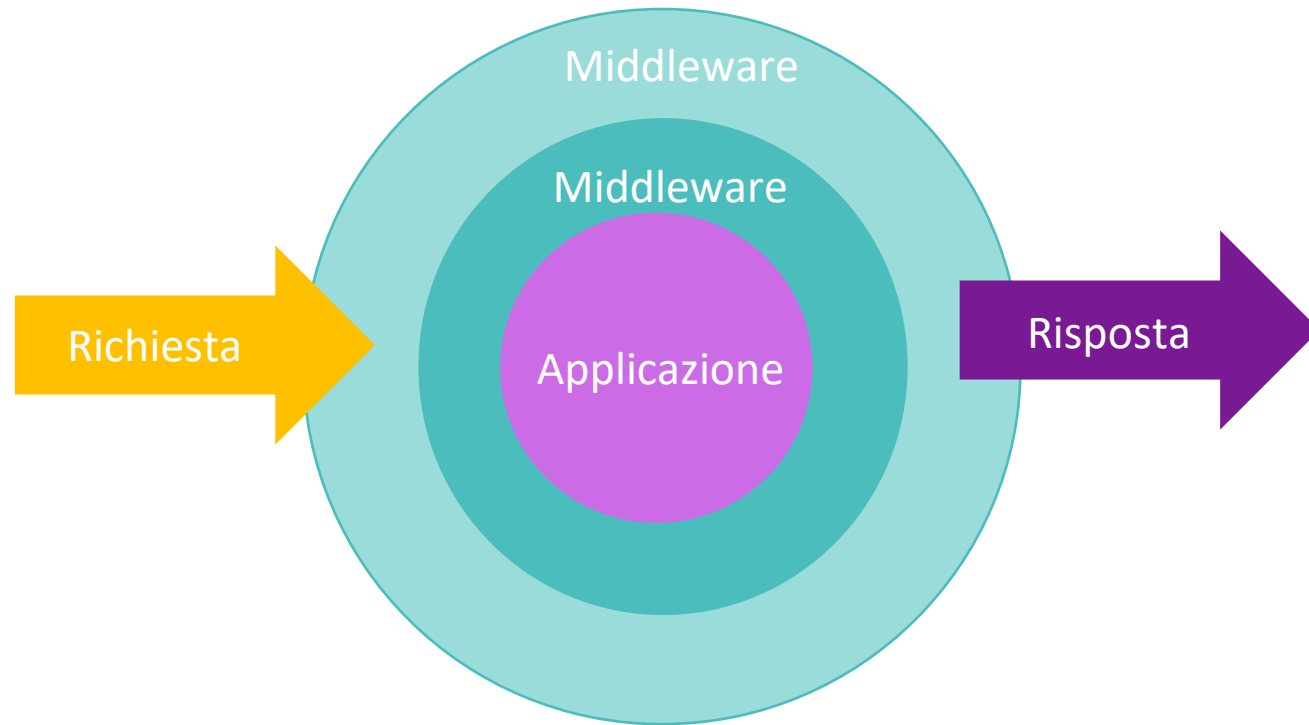
Il Middleware fornisce un comodo meccanismo per **esaminare e filtrare** le **richieste HTTP** che arrivano all'applicazione.

Ad esempio, Laravel include un middleware che verifica se l'utente è autenticato. In caso negativo, il middleware ridirezionerà l'utente alla schermata di login. Invece, se l'utente è autenticato, il middleware consentirà alla richiesta di procedere.

Middleware aggiuntivi possono essere creati per eseguire una varietà di compiti. Per esempio, un middleware di logging, uno per limitare le richieste per secondo all'applicazione, cambiare lingua del sito, ecc.

All'interno di Laravel sono inclusi diversi Middleware, inclusi quelli per l'autenticazione e per la protezione CSRF (Cross Site Request Forgery) e si trovano nella cartella **app/http/Middleware**.

COME SI COMPORTA UN MIDDLEWARE?



1. Il middleware intercetta una Request
2. Verifica delle condizioni
3. Se le condizioni NON sono soddisfatte, allora effettua una REDIRECT verso una route predeterminata
4. Se le condizioni sono verificate, la richiesta viene processata normalmente.

MIDDLEWARE

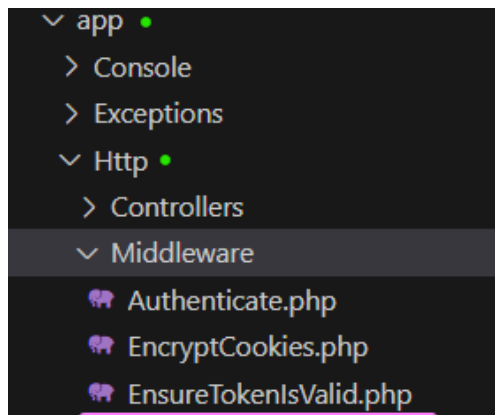
1- DEFINIRE MIDDLEWARE

Per creare un nuovo middleware, si usa il comando Artisan

```
make:middleware
```

```
php artisan make:middleware EnsureTokenIsValid  
  
Middleware created successfully.
```

Questo comando posizionerà una nuova classe **EnsureTokenIsValid** all'interno della directory **app/http/Middleware**.



In questo middleware ci limiteremo a consentire l'accesso alla rotta solo se il token fornito in input corrisponde ad un determinato valore. In caso negativo, gli utenti saranno ridirezionati alla home.

Perché la richiesta possa passare il middleware, si deve chiamare la callback **\$next** con la **\$request**.

È bene immaginare un middleware come una **serie di strati attraverso i quali la richiesta HTTP deve passare prima che arrivi all'applicazione vera e propria**.

Ogni strato o livello può esaminare la richiesta e rigettarla.

MIDDLEWARE

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureTokenIsValid
{
    /**
     * Handle an incoming request.
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Closure(\Illuminate\Http\Request): (\Illuminate\Http\Response|\Illuminate\Http\RedirectResponse)  $next
     * @return \Illuminate\Http\Response|\Illuminate\Http\RedirectResponse
     */
    public function handle(Request $request, Closure $next): Response
    {
        if ($request->input('token') !== 'my-secret-token'){
            return redirect('home');
        }
        return $next($request);
    }
}
```

Tutto avviene all'interno del metodo pubblico **handle** che prende come parametri la **\$request** (contenente l'oggetto **Request**) e la callback **\$next** che ha il compito di **processare la request se le condizioni sono soddisfatte**.

MIDDLEWARE

1.1 MIDDLEWARE E RISPOSTE

Un middleware può svolgere delle azioni prima o dopo aver passato la richiesta più internamente all'applicazione. Ad esempio, il middleware seguente esegue certe funzioni prima che la richiesta sia gestita dall'applicazione.

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class BeforeMiddleware
{
    public function handle(Request $request, Closure $next): Response
    {
        // Perform action

        return $next($request);
    }
}
```

Mentre quest'altro middleware eseguirà i suoi compiti dopo che la richiesta è passata all'applicazione

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class AfterMiddleware
{
    public function handle(Request $request, Closure $next): Response
    {
        $response = $next($request);

        // Perform action

        return $response;
    }
}
```

Middleware in base ad orario

- Bootstrap/app.php
- ->withMiddleware(function (Middleware \$middleware): void {
- \$middleware->alias([
- 'hour' => HourMiddleware::class]
- });
- })
- Api.php
- Route::group(['middleware' => ['auth:sanctum', 'hour']], function ()

- HourMiddleware
- public function handle(Request \$request, Closure \$next): Response
- {
- \$hour = now()->hour; // ora corrente (0-23)
- if (\$hour >= 9 && \$hour < 13) {
- // siamo nella fascia 9-12:59
- return \$next(\$request);
- }
- // fuori orario → blocco accesso
- return response()->json([
- 'message' => 'Accesso consentito solo dalle 9 alle 13'
-], 403);
- }
- }

MIDDLEWARE

2- REGISTRARE MIDDLEWARE

2.1.1 Gestione Manuale dei Middleware globali di default

Si può fornire lo stack di default dei global middleware al metodo `use` ed aggiustarlo come si desidera.

```
1  →withMiddleware(function (Middleware $middleware) {
2      $middleware→use([
3          \Illuminate\Foundation\Http\Middleware\InvokeDeferredCallbacks::class,
4          // \Illuminate\Http\Middleware\TrustHosts::class,
5          \Illuminate\Http\Middleware\TrustProxies::class,
6          \Illuminate\Http\Middleware\HandleCors::class,
7          \Illuminate\Foundation\Http\Middleware\PreventRequestsDuringMaintenance::class,
8          \Illuminate\Http\Middleware\ValidatePostSize::class,
9          \Illuminate\Foundation\Http\Middleware\TrimStrings::class,
10         \Illuminate\Foundation\Http\Middleware\ConvertEmptyStringsToNull::class,
11     ]);
12 })
```

2.1 MIDDLEWARE GLOBALI

Se si desidera lanciare un middleware durante ogni richiesta HTTP, si può aggiungerlo allo stack di middleware globale nel file `bootstrap/app.php`

L'oggetto `$middleware` fornito dalla closure `withMiddleware` è un'istanza di `Illuminate\Foundation\Configuration\Middleware` ed è responsabile della gestione dei middleware assegnati alle rotte.

Il metodo `append` aggiunge il middleware alla fine della lista dei middleware globali.

Per aggiungere un middleware all'inizio della lista si usa il metodo `prepend`.

```
1  use App\Http\Middleware\EnsureTokenIsValid;
2
3  →withMiddleware(function (Middleware $middleware) {
4      $middleware→append(EnsureTokenIsValid::class);
5  })
```

2- REGISTRARE MIDDLEWARE

2.2 ASSEGNARE MIDDLEWARE ALLE ROTTE

Per assegnare il middleware a specifiche rotte, si invoca il metodo `middleware` quando si definisce la rotta:

```
Route::get('/', function () {  
    // ...  
})->middleware([First::class, Second::class]);
```

```
1 use App\Http\Middleware\EnsureTokenIsValid;  
2  
3 Route::get('/profile', function () {  
4     // ...  
5 })->middleware(EnsureTokenIsValid::class);
```

Si possono assegnare middleware multipli alla rotta passando un array di nomi di middleware al metodo `middleware`:

MIDDLEWARE

2.2.1 ESCLUDERE MIDDLEWARE

Per evitare che un middleware venga applicato ad una specifica rotta all'interno di un gruppo di rotte, si può utilizzare il metodo **withoutMiddleware**. Il metodo si applica solo ai middleware delle rotte e non si applica ai middleware globali

Si può anche escludere un dato set di middleware dalla definizione di un intero gruppo di rotte

Da notare group()

```
1  use App\Http\Middleware\EnsureTokenIsValid;
2
3  Route::middleware([EnsureTokenIsValid::class])→group(function () {
4      Route::get('/', function () {
5          // ...
6      });
7
8      Route::get('/profile', function () {
9          // ...
10     })→withoutMiddleware([EnsureTokenIsValid::class]);
11 });
```

```
use App\Http\Middleware\EnsureTokenIsValid;

Route::withoutMiddleware([EnsureTokenIsValid::class])→group(function () {
    Route::get('/profile', function () {
        // ...
    });
});
```

MIDDLEWARE

2.3 GRUPPI DI MIDDLEWARE

Si possono raggruppare diversi middleware sotto un'unica chiave per rendere più semplice l'assegnazione alle rotte.

Questo è possibile grazie al metodo `appendToGroup` all'interno del file `bootstrap/app.php`

```
1 use App\Http\Middleware\First;
2 use App\Http\Middleware\Second;
3
4 →withMiddleware(function (Middleware $middleware) {
5     $middleware→appendToGroup('group-name', [
6         First::class,
7         Second::class,
8     ]);
9
10    $middleware→prependToGroup('group-name', [
11        First::class,
12        Second::class,
13    ]);
14 })
```

La sintassi per l'assegnazione di gruppi di middleware a rotte e controller è uguale a quella dei singoli middleware

```
Route::get('/', function () {
    // ...
})→middleware('web');

Route::middleware(['web'])→group(function () {
    // ...
});
```

Laravel include gruppi di middleware web ed api predefiniti applicabili alle rotte web ed api. Laravel applica automaticamente questi gruppi di middleware ai file `routes/web.php` e `routes/api.php`

WEB MIDDLEWARE GROUP:

Illuminate\Cookie\Middleware\EncryptCookies
Illuminate\Cookie\Middleware\AddQueuedCookiesToResponse
Illuminate\Session\Middleware\StartSession
Illuminate\View\Middleware\ShareErrorsFromSession
Illuminate\Foundation\Http\Middleware\ValidateCsrfToken
Illuminate\Routing\Middleware\SubstituteBindings

WEB MIDDLEWARE GROUP:

Illuminate\Routing\Middleware\SubstituteBindings

MIDDLEWARE

Si possono aggiungere middleware ai gruppi predefiniti usando i metodi **web** e **api** all'interno del file **bootstrap/app.php**. Sono un'alternativa al metodo `appendToGroup`

```
1 use App\Http\Middleware\EnsureTokenIsValid;
2 use App\Http\Middleware\EnsureUserIsSubscribed;
3
4 →withMiddleware(function (Middleware $middleware) {
5     $middleware→web(append: [
6         EnsureUserIsSubscribed::class,
7     ]);
8
9     $middleware→api(prepend: [
10         EnsureTokenIsValid::class,
11     ]);
12 })
```

È anche possibile sostituire uno dei middleware all'interno dei gruppi predefiniti - **replace**

```
1 use App\Http\Middleware\StartCustomSession;
2 use Illuminate\Session\Middleware\StartSession;
3
4 $middleware→web(replace: [
5     StartSession::class ⇒ StartCustomSession::class,
6 ]);
```

O rimuovere interamente un middleware - **remove**

```
1 $middleware→web(remove: [
2     StartSession::class,
3 ]);
```

MIDDLEWARE

2.4 MIDDLEWARE ALIASES

Per comodità, si possono assegnare alias ai middleware nel file bootstrap/app.php.

Gli aliases consentono di definire un alias breve per una data classe di middleware.

```
1 use App\Http\Middleware\EnsureUserIsSubscribed;
2
3 →withMiddleware(function (Middleware $middleware) {
4     $middleware→alias([
5         'subscribed' ⇒ EnsureUserIsSubscribed::class
6     ]);
7 })
```

Una volta assegnato l'alias al middleware nel kernel HTTP, lo si può utilizzare per assegnare il middleware alla rotta:

```
1 Route::get('/profile', function () {
2     // ...
3 })→middleware('subscribed');
```

Alcuni dei middleware predefiniti di Laravel hanno già degli alias.

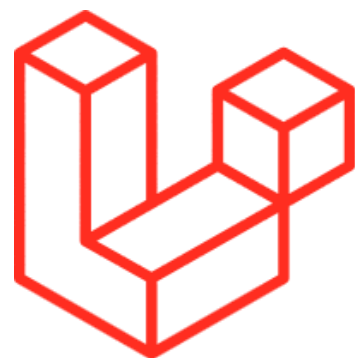
Alias	Middleware
auth	Illuminate\Auth\Middleware\Authenticate
auth.basic	Illuminate\Auth\Middleware\AuthenticateWithBasicAuth
auth.session	Illuminate\Session\Middleware\AuthenticateSession
cache.headers	Illuminate\Http\Middleware/SetCacheHeaders
can	Illuminate\Auth\Middleware\Authorize
guest	Illuminate\Auth\Middleware\RedirectIfAuthenticated
password.confirm	Illuminate\Auth\Middleware\RequirePassword
precognitive	Illuminate\Foundation\Http\Middleware\HandlePrecognitiveRequests
signed	Illuminate\Routing\Middleware\ValidateSignature
subscribed	\Spark\Http\Middleware\VerifyBillableIsSubscribed
throttle	Illuminate\Routing\Middleware\ThrottleRequests or Illuminate\Routing\Middleware\ThrottleRequestsRedis
verified	Illuminate\Auth\Middleware\EnsureEmailIsVerified

MIDDLEWARE

2.5 ORDINARE I MIDDLEWARE

Quando i middleware vengono assegnati alla rotta, non si ha controllo sul loro ordine. Nel caso si abbia la necessità di eseguire i middleware in un ordine specifico, si può specificare la priorità dei middleware usando il metodo **priority** nel file **bootstrap/app.php**

```
1  →withMiddleware(function (Middleware $middleware) {
2      $middleware→priority([
3          \Illuminate\Foundation\Http\Middleware\HandlePrecognitiveRequests::class,
4          \Illuminate\Cookie\Middleware\EncryptCookies::class,
5          \Illuminate\Cookie\Middleware\AddQueuedCookiesToResponse::class,
6          \Illuminate\Session\Middleware\StartSession::class,
7          \Illuminate\View\Middleware\ShareErrorsFromSession::class,
8          \Illuminate\Foundation\Http\Middleware\ValidateCsrfToken::class,
9          \Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful::class,
10         \Illuminate\Routing\Middleware\ThrottleRequests::class,
11         \Illuminate\Routing\Middleware\ThrottleRequestsWithRedis::class,
12         \Illuminate\Routing\Middleware\SubstituteBindings::class,
13         \Illuminate\Contracts\Auth\Middleware\AuthenticatesRequests::class,
14         \Illuminate\Auth\Middleware\Authorize::class,
15     ]);
16 })
```

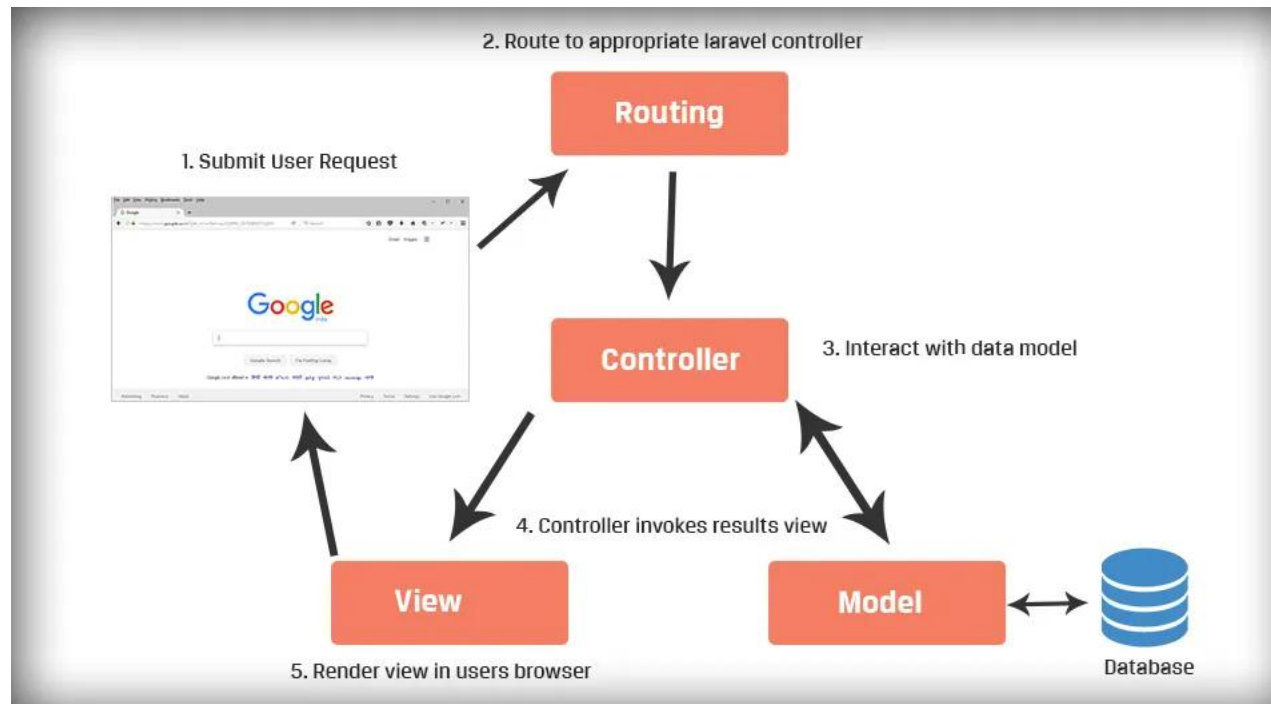


Laravel

LE BASI

Controllers

1 - Introduzione ai controller



Laravel è pensato per rendere semplice la **separazione dei compiti (separation of concern)** tra diverse classi.

Per questo motivo è possibile separare la definizione di una rotta (intesa come definizione della URI e del metodo HTTP) dalla gestione del comportamento per generare la risposta.

Le classi che si occupano della logica di gestione delle richieste sono i controller.

Una classe `UserController`, ad esempio, potrebbe gestire tutte le richieste in entrata relative agli utenti, incluse la lettura, creazione, modifica e cancellazione degli utenti.

Di default, tutti i controller si trovano all'interno della cartella **`app/http/Controllers`**.

2 - CREARE CONTROLLER

2.1 CONTROLLER BASE



```
php artisan make:controller UserController
```

Comando per la creazione di un nuovo controller all'interno della cartella `app/http/Controllers`.

È bene usare il **suffisso Controller** per il nome del controller e della rispettiva classe.

Un controller può avere **qualsiasi numero di metodi pubblici** che **rispondono alle richieste HTTP in entrata**.

Una volta creato un controller e i suoi metodi, dovremo **definire una rotta (o gruppo di rotte) alla quale collegare il metodo del controller**:

```
use App\Http\Controllers\UserController;

Route::get('/user/{id}', [UserController::class, 'show']);
```

Quando una richiesta in entrata corrisponde alla rotta URI specificata, il metodo `show` del controller `UserController` sarà invocato e i **parametri della rotta saranno passati al metodo**.

```
namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\View\View;

class UserController extends Controller
{
    /**
     * Show the profile for a given user.
     */
    public function show(string $id): View
    {
        return view('user.profile', [
            'user' => User::findOrFail($id)
        ]);
    }
}
```

```
use App\Http\Controllers\PostController;
Route::get('/posts', [PostController::class, 'index']);
//il metodo index della classe PostController gestirà le richieste GET/post
Route::get('/posts/{id}', [PostController::class, 'show']);
//il metodo show della classe PostController (che riceverà come parametro $id)
// gestirà le richieste GET/post/{id}
```

Quando si hanno **più rotte collegate ad un controller**, ad esempio quando si possono compiere più azioni sulla medesima entità, è possibile **raggruppare tali rotte usando il metodo controller per definire il controller comune** ed **indicare solo il metodo del controller da invocare sulla singola rotta**:

```
use App\Http\Controllers\PostController;
Route::controller(PostController::class)->group(function () {
    Route::get('/posts', 'index');
    Route::get('/posts/{id}', 'show');
    Route::post('/posts', 'store');
});
```



2.1.1 METODI DI UN CONTROLLER E PARAMETRI

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class PostController extends Controller
{
    public function index(): string
    {
        return 'List of blog posts';
    }
    public function show($id): string
    {
        return "Content for blog post $id";
    }
    public function store(Request $request): string
    {
        return 'Created new blog post';
    }
}
```

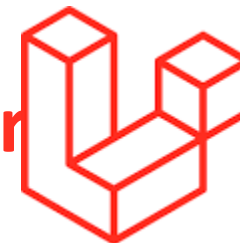
I controller **POSSONO** estendere la classe Controller, non è necessario.

I nomi dei metodi corrispondono a quanto dichiarato nella definizione della rotta. Se non corrispondessero, il metodo del controller non verrebbe invocato.

I metodi del controller **possono avere parametri** a cui verrà passato il valore del o dei parametri della rotta.

È possibile sfruttare la **dependency injection** per passare istanze specifiche al metodo, come ad esempio, l'istanza della Request.

2.2 SINGLE ACTION CONTROLLERS (invokable controller)



Se l'azione di un controller è particolarmente complessa, può essere utile dedicare un'intera classe controller a quella azione. Per ottenere questo si può definire un unico metodo `__invoke` all'interno del controller.

Quando si registrano le rotte per controller invocabile, non serve specificare il metodo del controller. Si può semplicemente passare il nome del controller al router:

```
use App\Http\Controllers\ProvisionServer;

Route::post('/server', ProvisionServer::class);
```

Si può creare un **invokable controller** usando l'opzione `--invokable` del comando Artisan `make:controller`

```
php artisan make:controller ProvisionServer --invokable
```

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  class ProvisionServer extends Controller
6  {
7      /**
8       * Provision a new web server.
9       */
10     public function __invoke()
11     {
12         // ...
13     }
14 }
```

Si può invocare senza definire il metodo

3 - CONTROLLER MIDDLEWARE



I middleware possono essere assegnati alle rotte dei controller durante la definizione delle rotte:

```
Route::get('profile', [UserController::class, 'show'])->middleware('auth');
```

Oppure il middleware può essere **specificato all'interno della classe del controller**. In questo caso il controller dovrebbe implementare l'interfaccia **HasMiddleware** che afferma che il controller dovrebbe avere un metodo **middleware** statico. Da questo metodo si ritorna un array di middleware che dovrebbero essere applicati alle azioni del controller.

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Routing\Controllers\HasMiddleware;
6  use Illuminate\Routing\Controllers\Middleware;
7
8  class UserController extends Controller implements HasMiddleware
9  {
10     /**
11      * Get the middleware that should be assigned to the controller.
12      */
13     public static function middleware(): array
14     {
15         return [
16             'auth',
17             new Middleware('log', only: ['index']),
18             new Middleware('subscribed', except: ['store']),
19         ];
20     }
21
22     // ...
23 }
```

```
1  use Closure;
2  use Illuminate\Http\Request;
3
4  /**
5   * Get the middleware that should be assigned to the controller.
6   */
7  public static function middleware(): array
8  {
9      return [
10         function (Request $request, Closure $next) {
11             return $next($request);
12         },
13     ];
14 }
```

- I controllers consentono inoltre di **registrare i middleware usando una closure**. Questo è un metodo utile per **definire un middleware inline per un singolo controller senza definire un'intera classe middleware**.

4 - RESOURCE CONTROLLERS



Se si pensa ad **ogni modello Eloquent come ad una «risorsa»**, è tipico eseguire lo **stesso set di azioni su ogni risorsa**. Ad esempio, immaginiamo che la nostra applicazione contenga un model Photo ed un model Movie. E' probabile che gli utenti possano creare, leggere, modificare o cancellare entrambe queste risorse.

Per questo il **routing delle risorse** di Laravel **asigna le tipiche rotte create, read, update e delete (CRUD)** ad un **controller con una singola riga di codice**.

Per iniziare si può usare l'opzione **--resource** del comando **make:controller** per creare rapidamente un **controller che gestisce queste azioni**:

```
php artisan make:controller PhotoController --resource
```

Verrà così creato un controller **app/http/Controllers/PhotoController.php** che **conterrà un metodo per ogni operazione possibile sulle risorse (index, create, store, show, edit, update, destroy)**. Si può in seguito registrare una rotta che punti a quel controller:

```
use App\Http\Controllers\PhotoController;  
  
Route::resource('photos', PhotoController::class);
```


Questa singola dichiarazione **crea molteplici rotte per gestire una varietà di azioni sulle risorse.**

Il controller creato avrà già i metodi per ognuna di queste azioni.

Si possono anche **registrare tanti resource controllers uno alla volta passando un array al metodo resource:**

```
Route::resources([
    'photos' => PhotoController::class,
    'posts' => PostController::class,
]);
```

Si ricorda che è possibile dare una veloce occhiata a tutte le rotte disponibili nell'applicazione grazie al comando **route:list**. Esso evidenzierà quale controller e quale metodo sono collegati alla rotta.

```
$ php artisan route:list --except-vendor
GET|HEAD posts ..... PostController@index
POST posts ..... PostController@store
GET|HEAD posts/{id} ..... PostController@show
GET|HEAD user .....
GET|HEAD user/orders .....
GET|HEAD user/profile .....
```

A - AZIONI GESTITE DAL CONTROLLER DELLE RISORSE

Pur non essendo fondamentale per la connessione tra rotte e controller, è buona pratica usare **alcuni nomi convenzionali per i metodi** del controller in base al tipo di richiesta che stanno gestendo.

VERB (METODO HTTP)	URI	METODO DEL CONTROLLER	ROUTE NAME	AZIONE
GET	/photos	index	photos.index	Elenca tutte le foto
GET	/photos/create	create	photos.create	Mostra il form per la creazione di una nuova risorsa
POST	/photos	store	photos.store	Crea una nuova foto
GET	/photos/{photo}	show	photos.show	Mostra la risorsa con specifico id
GET	/photos/{photo}/edit	edit	photos.edit	Mostra il form per modificare la risorsa specificata
PUT/PATCH	/photos/{photo}	update	photos.update	Modifica ed aggiorna la risorsa con id specificato
DELETE	/photos/{photo}	destroy	photos.destroy	Rimuove la risorsa specificata dal database



4.1 – PARTIAL RESOURCE ROUTES

Quando si dichiara la rotta di una risorsa, si può specificare un **sottoinsieme di azioni** che il controller dovrebbe gestire al posto dell'intero set di azioni di default:

```
use App\Http\Controllers\PhotoController;

Route::resource('photos', PhotoController::class)->only([
    'index', 'show'
]);

Route::resource('photos', PhotoController::class)->except([
    'create', 'store', 'update', 'destroy'
]);
```



4.1.1 – ROTTE DELLE RISORSE API

Quando si dichiarano rotte per le risorse che saranno consumate da APIs, si tende a voler escludere le rotte che presentano template HTML come **create** e **edit**.

Il metodo **apiResource** automaticamente **esclude** queste due rotte

```
use App\Http\Controllers\PhotoController;

Route::apiResource('photos', PhotoController::class);
```

Si possono **registrare più API resource controllers in una volta sola** passando un **array** al metodo **apiResources**

```
use App\Http\Controllers\PhotoController;
use App\Http\Controllers\PostController;

Route::apiResources([
    'photos' => PhotoController::class,
    'posts' => PostController::class,
]);
```

Per generare velocemente un **controller per le risorse API che non inci metodi create e edit**, si usa **--api** quando si esegue il comando **luda make:controller**

```
php artisan make:controller PhotoController --api
```



4.2 – RISORSE ANNIDATE

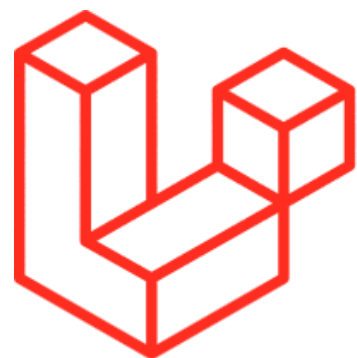
Si può avere la necessita di definire rotte a risorse annidate. Ad esempio una risorsa photo potrebbe avere diversi commenti allegati alla photo.

Per annidare i controller delle risorse, di può utilizzare la **dot notation nella dichiarazione della rotta**:

```
use App\Http\Controllers\PhotoCommentController;  
  
Route::resource('photos.comments', PhotoCommentController::class);
```

Questa rotta registrerà una risorsa annidata che potrebbe essere acceduta con gli URI come il seguente:

```
/photos/{photo}/comments/{comment}
```



Laravel

LE BASI

—

HTTP Requests

La classe `Illuminate\Http\Request` fornisce un modo object-oriented per interagire con l'attuale richiesta HTTP gestita dall'applicazione così come recuperare l'input, i cookies ed i file che sono stati inviati con la request.

1 - INTERAGIRE CON LA REQUEST

1.1 – Accedere la Request

Per ottenere un'istanza della request HTTP corrente attraverso la dependency injection, si può usare il type-hint della classe `Illuminate\Http\Request` nella closure della route o nel metodo del controller.

L'istanza della richiesta in entrata sarà automaticamente iniettata dal service container di Laravel:

```
use Illuminate\Http\Request;

Route::get('/', function (Request $request) {
    // ...
});
```

Closure della route

```
namespace App\Http\Controllers;

use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Store a new user.
     */
    public function store(Request $request): RedirectResponse
    {
        $name = $request->input('name');

        // Store the user...

        return redirect('/users');
    }
}
```

Metodo del controller

1.1.a – Dependency Injection e parametri della Route

Se il metodo del controller si aspetta input dal parametro della rotta, bisogna elencare i parametri della rotta dopo le altre dipendenze.

Ad esempio, se la nostra rotta è definita così:

```
use App\Http\Controllers\UserController;

Route::put('/user/{id}', [UserController::class, 'update']);
```

Possiamo ancora fare il type-hint della classe Request ed accedere il parametro id della rotta definendo il metodo del controller in questo modo:

```
namespace App\Http\Controllers;

use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Update the specified user.
     */
    public function update(Request $request, string $id): RedirectResponse
    {
        // Update the user...

        return redirect('/users');
    }
}
```


1.2 – PATH, HOST E METODI DELLA REQUEST

L'istanza della Request fornisce una varietà di metodi per esaminare la richiesta HTTP in entrata ed estende la classe `Symfony\Component\HttpFoundation\Request`.

1.2.a – Recuperare il path della Request (metodo `path`)

Il metodo `path` ritorna informazioni circa il path della request.

Se la richiesta è diretta a

`http://example.com/foo/bar`, il metodo `path` ritornerà `foo/bar`

```
$uri = $request->path();
```

1.2.b – Ispezionare il path della Request/ Route (metodo `is`)

Il metodo `is` permette di verificare che il path della richiesta corrisponda ad un dato pattern. Si può usare `*` come jolly:

```
if ($request->is('admin/*')) {  
    // ...  
}
```

Usando il metodo `routeIs` si può determinare se la richiesta corrisponde ad una named route.

```
if ($request->routeIs('admin.*')) {  
    // ...  
}
```

1.2.c – Recuperare l'URL della Request

Si usano i metodi `url` o `fullUrl`:

- Metodo `url`: ritorna l'URL senza la query string
- Metodo `fullUrl`: ritorna l'URL con la query string

```
$url = $request->url();  
  
$urlWithQueryString = $request->fullUrl();
```

Per aggiungere i dati della query string all'URL corrente, si usa il metodo `fullUrlWithQuery` che farà il merge dell'array delle variabili della query string con la query string corrente:

```
$request->fullUrlWithQuery(['type' => 'phone']);
```

Per ottenere l'Url corrente senza un dato parametro della query string, si può usare il metodo `fullUrlWithoutQuery`

```
$request->fullUrlWithoutQuery(['type']);
```

1.2.d – Recuperare l'host della Request

Di può recuperare l'"host" della request con i metodi `host`, `httpHost` e `schemeAndHttpHost`

```
$request->host();  
$request->httpHost();  
$request->schemeAndHttpHost();
```

1.2.e – Recuperare il metodo della Request

Il metodo `method` ritorna il metodo HTTP della request. Il metodo `isMethod` verifica se il metodo http corrisponde ad una data stringa.

```
$method = $request->method();  
  
if ($request->isMethod('post')) {  
    // ...  
}
```

1.3 HEADERS DELLA REQUEST

- Si può ricavare l'header di una request dall'istanza della classe `Illuminate\Http\Request` usando il metodo **header**. Se l'header non fosse presente sarà ritornato un **null**.
- Il metodo `header` accetta un secondo parametro opzionale che può essere ritornato in caso l'header non fosse presente nella request.
- Il metodo **hasHeader** può essere usato per determinare **se la request contiene un dato header**
- Per comodità il metodo **bearerToken** può essere usato per **recuperare un bearer token dall'header Authorization**. Se non è presente un token di questo tipo, verrà ritornata una stringa vuota

```
$value = $request->header('X-Header-Name');  
  
$value = $request->header('X-Header-Name', 'default');
```

```
if ($request->hasHeader('X-Header-Name')) {  
    // ...  
}
```

```
$token = $request->bearerToken();
```

1.4 INDIRIZZO IP DELLA REQUEST

- Il metodo **ip** serve a ricavare l'indirizzo ip del client che ha effettuato la richiesta all'applicazione.
- Per recuperare un array di indirizzi IP, inclusi tutti quelli del client inoltrati dai proxy, si usa il metodo **ips**. L'indirizzo 'originale' del client sarà alla fine dell'array.

```
$ipAddress = $request->ip();
```

```
$ipAddresses = $request->ips();
```

1.5 NEGOZIAZIONE DEL CONTENUTO

- Laravel fornisce diversi metodi per ispezionare il tipo di contenuto richiesto della request attraverso l'header **Accept**.
 - Prima, il metodo **getAcceptableContentTypes** ritorna un **array contenente tutti i tipi di contenuto accettati** dalla request
 - Metodo **accepts**: accetta un array di content types e ritorna **true** se uno qualsiasi dei tipi è accettato dalla request.

```
$contentTypes = $request->getAcceptableContentTypes();
```

```
if ($request->accepts(['text/html', 'application/json'])) {  
    // ...  
}
```

- Metodo **prefers**: determina quale tipo di contenuto, tra quelli contenuti in un dato array di content types, è preferito dalla request. Se nessuno dei tipi forniti è accettato dalla richiesta, viene restituito **null**.
- Siccome molte applicazioni servono solo HTML o JSON, il metodo **expectsJson** determina se la richiesta si aspetta una risposta JSON

```
$preferred = $request->prefers(['text/html', 'application/json']);
```

```
if ($request->expectsJson()) {  
    // ...  
}
```

2 - INPUT

2.1 RECUPERARE L'INPUT

2.1.1 RECUPERARE TUTTI I DATI DELL'INPUT

- Il metodo **all** permette di recuperare tutti i dati ricevuti in input con la request.
Si può utilizzare senza prestare troppa attenzione alla provenienza della richiesta (da un form HTML o da una richiesta XHR)
- Il metodo **collect** recupera tutti i dati di input della request **come una collection**.
Consente anche di recuperare un **subset di input** della request come collection.

```
$input = $request->all();
```

```
$input = $request->collect();
```

```
$request->collect('users')->each(function (string $user) {  
    // ...  
});
```

2.1.2 RECUPERARE IL VALORE DI UN INPUT

Grazie a pochi semplice metodi, si possono accedere tutti gli input dell'utente dall'istanza della Request senza preoccuparsi del verbo HTTP usato per la request stessa.

- Il metodo **input** recupera tutti i valori di input dell'utente come array associativo.
- Se viene specificato il primo argomento, recupera il valore del campo di input con quell nome
- Si può passare un valore di default come secondo argomento del metodo input. Questo valore sarà ritornato se il valore dell'input richiesto non è presente sulla request.
- Quando si lavora con form che contengono array di input, si usa la **dot notation** per accedere gli array.

Il metodo input recupera i valori dall'intero payload della request (inclusa la query string).

```
$input = $request->input();
```

```
$name = $request->input('name');
```

```
$name = $request->input('name', 'Sally');
```

```
$name = $request->input('products.0.name');
```

```
$names = $request->input('products.*.name');
```

2.1.3 RECUPERARE INPUT DALLA QUERY STRING

- Il metodo **query** recupera solo i valori dalla query string
- Se il valore del dato richiesto non è presente, viene ritornato il secondo argomento del metodo, se presente.
- Si usa il metodo query senza argomenti per recuperare tutti i valori della query string in un array associativo

```
$name = $request->query('name');
```

```
$name = $request->query('name', 'Helen');
```

```
$query = $request->query();
```

2.1.4 RECUPERARE INPUT STRINGABLE

- Invece di recuperare i dati di input come stringhe primitive, si può usare il metodo **string** per recuperarle come istanza di `Illuminate\Support\Stringable`
- Questa classe offre diversi metodi per la manipolazione delle stringhe in maniera fluida e "chainable"

```
$name = $request->string('name')->trim();
```

```
$request->string('name')->trim()->upper();
```

INVECE DI:

```
strtoupper(trim($request->input('name')));
```


2.2 DETERMINARE SE L'INPUT É PRESENTE

- Il metodo **has** consente di determinare se un valore è presente nella richiesta. Il metodo **has** ritorna **true** se il valore è presente

```
if ($request->has('name')) {  
    // ...  
}
```

- Dato un **array**, il metodo **has** determina se **TUTTI** i valori specificati sono presenti

```
if ($request->has(['name', 'email'])) {  
    // ...  
}
```

- Il metodo **hasAny** ritorna true se **almeno uno** dei valori specificati sono presenti.

```
if ($request->hasAny(['name', 'email'])) {  
    // ...  
}
```

- Il metodo **whenHas** esegue la closure data se un valore è presente nella richiesta.

Al metodo **whenHas** può essere passata una **seconda closure** che sarà eseguita se il valore specificato non è presente nella request

```
$request->whenHas('name', function (string $input) {  
    // ...  
});
```

```
$request->whenHas('name', function (string $input) {  
    // The "name" value is present...  
}, function () {  
    // The "name" value is not present...  
});
```

- Per determinare se un valore è presente nella richiesta e non è una stringa vuota, si usa il metodo **filled**

```
1  if ($request→filled('name')) {  
2      // ...  
3  }
```

- Al contrario, per determinare se un valore manca nella richiesta oppure è una stringa vuota, si usa il metodo **isNotFilled**.

Se il metodo `isNotFilled` riceve un **array** come argomento, verificherà se **tutti i valori** specificati sono vuoti o mancanti.

```
1  if ($request→isNotFilled('name')) {  
2      // ...  
3  }
```

```
1  if ($request→isNotFilled(['name', 'email'])) {  
2      // ...  
3  }
```

- Il metodo **anyFilled** ritorna true se **alcuni** dei valori specificati non sono stringhe vuote.

```
1  if ($request→anyFilled(['name', 'email'])) {  
2      // ...  
3  }
```

- Il metodo **whenFilled** esegue la closure data se un valore è presente nella richiesta e non è una stringa vuota.

Al metodo **whenFilled** può essere passata una **seconda closure** che sarà eseguita se il valore specificato non è "filled"

```
1  $request→whenFilled('name', function (string $input) {  
2      // ...  
3  });
```

```
1  $request→whenFilled('name', function (string $input) {  
2      // The "name" value is filled...  
3  }, function () {  
4      // The "name" value is not filled...  
5  });
```

- Per determinare se una data chiave è assente dalla request, si possono usare i metodi `missing` o `whenMissing`

2.3 MERGE DI INPUT AGGIUNTIVI

- Per unire manualmente ulteriori dati a quelli già presenti nella richiesta, si può usare il metodo `merge`. Se una determinata chiave è già presente, verrà sovrascritta dai dati forniti dal metodo `merge`.

```
1 $request->merge(['votes' => 0]);
```

- Il metodo `mergeIfMissing` può essere usato per unire input nella request se la chiave corrispondente non esiste nei dati di input

```
1 $request->mergeIfMissing(['votes' => 0]);
```

2.4 RECUPERARE VECCHI INPUT

- Per recuperare gli input memorizzati dalla richiesta precedente, si invoca il metodo `old` su un'istanza di `Illuminate\Http\Request`.
Il metodo `old` estrapolerà i precedenti dati di input dalla session:

```
$username = $request->old('username');
```

- Laravel fornisce anche un helper `old` globale.
Se si sta visualizzando un vecchio input all'interno di un template Blade, è più comodo usare l'helper `old` per ripopolare il form.
Se non esiste un vecchio input per il campo richiesto, sarà ritornato null:

```
<input type="text" name="username" value="{{ old('username') }}">
```

1- IL FILE INDEX.PHP

Il file public/index.php è quindi lo script PHP che viene invocato ogni volta che il server riceve una richiesta HTTP. Questo file contiene solamente il codice necessario a caricare il resto del framework.

```
require __DIR__.'/../vendor/autoload.php'; //require dell'autoload -> FONDAMENTALE
```

```
$app = require_once __DIR__.'/../bootstrap/app.php'; //recupera un'istanza dell'applicazione Laravel $app

$kernel = $app->make(Kernel::class); // carica il kernel opportuno (a seconda del tipo di richiesta sarà un kernel HTTP o un kernel console)

$response = $kernel->handle(
    $request = Request::capture()
)->send(); // fai gestire la $request arrivarta al kernel e invia una $response

$kernel->terminate($request, $response);
```

Questo ciclo viene ripetuto ad ogni richiesta che arriva al server, passando la richiesta da uno stato all'altro finno a giungere alla parte di codice che gestirà e restituirà il contenuto della risposta.

2- LA SELEZIONE DEL MIDDLEWARE

A seconda del tipo di richiesta, essa viene indirizzata verso un middleware (Gruppo di rotte) specifico.

Quando l'app viene inizializzata, viene caricato un servizio interno per la gestione delle rotte la cui versione di default è la classe App\Providers\RouteServiceProvider contenuta nel file app/Providers/RouteServiceProvider.php, prevede di default che:

```
$this->routes(function () {  
    Route::prefix('api')  
        ->middleware('api')  
        ->namespace($this->namespace)  
        ->group(base_path('routes/api.php'));  
  
    Route::middleware('web')  
        ->namespace($this->namespace)  
        ->group(base_path('routes/web.php'));  
});
```

Tutte le richieste di risorse il cui path inizia con **'api'** appartengono al **middleware api** e verranno smistate all'interno del file **routes/api.php** per un ulteriore successivo smistamento. Ogni altra richiesta va al **middleware web** e verrà successivamente smistata dal file **routes/web.php**

RICHIESTA	MIDDLEWARE
GET/ api /orders	Api
POST/page/form	Web
GET/docs/api	web

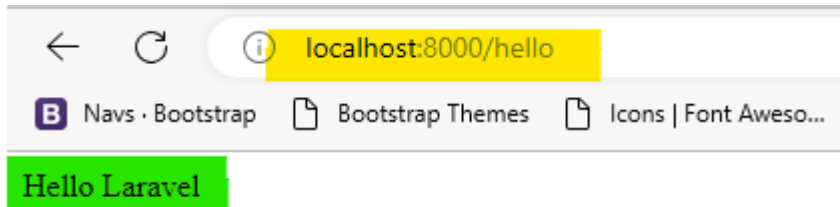
RICHIESTA	MIDDLEWARE
GET /blog/2022/12/23/post.html	Web
DELETE / api /orders/23455	api
DELETE /orders/23455	web

3 - LA SELEZIONE DELLA ROTTA REGISTRATA

I file routes/web.php e routes/api.php, anche se in modo leggermente differente, contengono le **effettive rotte registrate**

FILE routes/web.php

```
Route::get('/', function () {  
    return view('welcome');  
});  
Route::get('/hello', function () {  
    return "Hello Laravel";  
});
```



Se esiste una **rotta registrata per il path della richiesta**, allora sarà eseguita la **funzione corrispondente**.

ESEMPI:

Se nel middleware web arrivasse una richiesta con path **GET/hello**, verrà ritornato il testo «**Hello Laravel**» che **diventerà il body della risposta HTTP con status code 200 OK**.

Per il path **GET/** verrà invocata una funzione che effettuerà il **rendering della view 'welcome'**: verrà cercato il template associato, elaborato e il risultato restituito come testo (o come HTML) che diventerà il body della risposta HTTP.

Ogni altra richiesta, sia per metodo che per risorsa richiesta, **non avendo una rotta registrata, non potrà essere soddisfatta** e Laravel restituirà una risposta con **status 404**

FILE routes/api.php

```
Route::middleware('auth:sanctum')->get('/user', function (Request $request) {  
    return $request->user();  
});
```

NOTA: Laravel permette la definizione di una rotta anche **tramite controller**. Per ora lo si può pensare come un modo per raggruppare in una singola classe rotte che fanno parte dello stesso aggregato

LE ROTTE ED ARTISAN

È possibile ottenere un elenco completo delle rotte registrate tramite il comando Artisan:

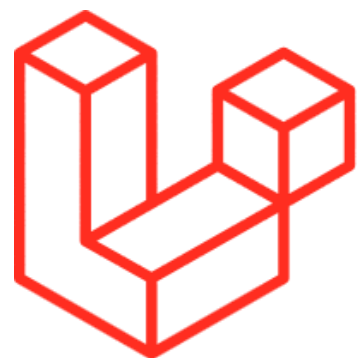
`php artisan route:list`

```
+-----+-----+-----+-----+-----+-----+
| Domain | Method | URI           | Name | Action                               | Middleware                               |
+-----+-----+-----+-----+-----+-----+
|         | GET|HEAD | /             |      | Closure                             | web                                     |
|         | GET|HEAD | api/user      |      | Closure                             | api                                   |
|         |         |               |      |                                     | App\Http\Middleware\Authenticate:sanctum |
|         | GET|HEAD | hello         |      | Closure                             | web                                   |
|         | GET|HEAD | sanctum/csrf-cookie |      | Laravel\Sanctum\Http\Controllers\CsrfCookieController@show | web                                   |
+-----+-----+-----+-----+-----+-----+
```

OPZIONI:

- **-v** mostra anche i middleware associati
- **-path** mostra solo le rotte che contengono solo una data stringa nell'URI. Esempio `-path=user`

```
>php artisan route:list --path=user
+-----+-----+-----+-----+-----+-----+
| Domain | Method | URI           | Name | Action                               | Middleware                               |
+-----+-----+-----+-----+-----+-----+
|         | GET|HEAD | api/user      |      | Closure                             | api                                   |
|         |         |               |      |                                     | App\Http\Middleware\Authenticate:sanctum |
+-----+-----+-----+-----+-----+-----+
```

Laravel

LE BASI

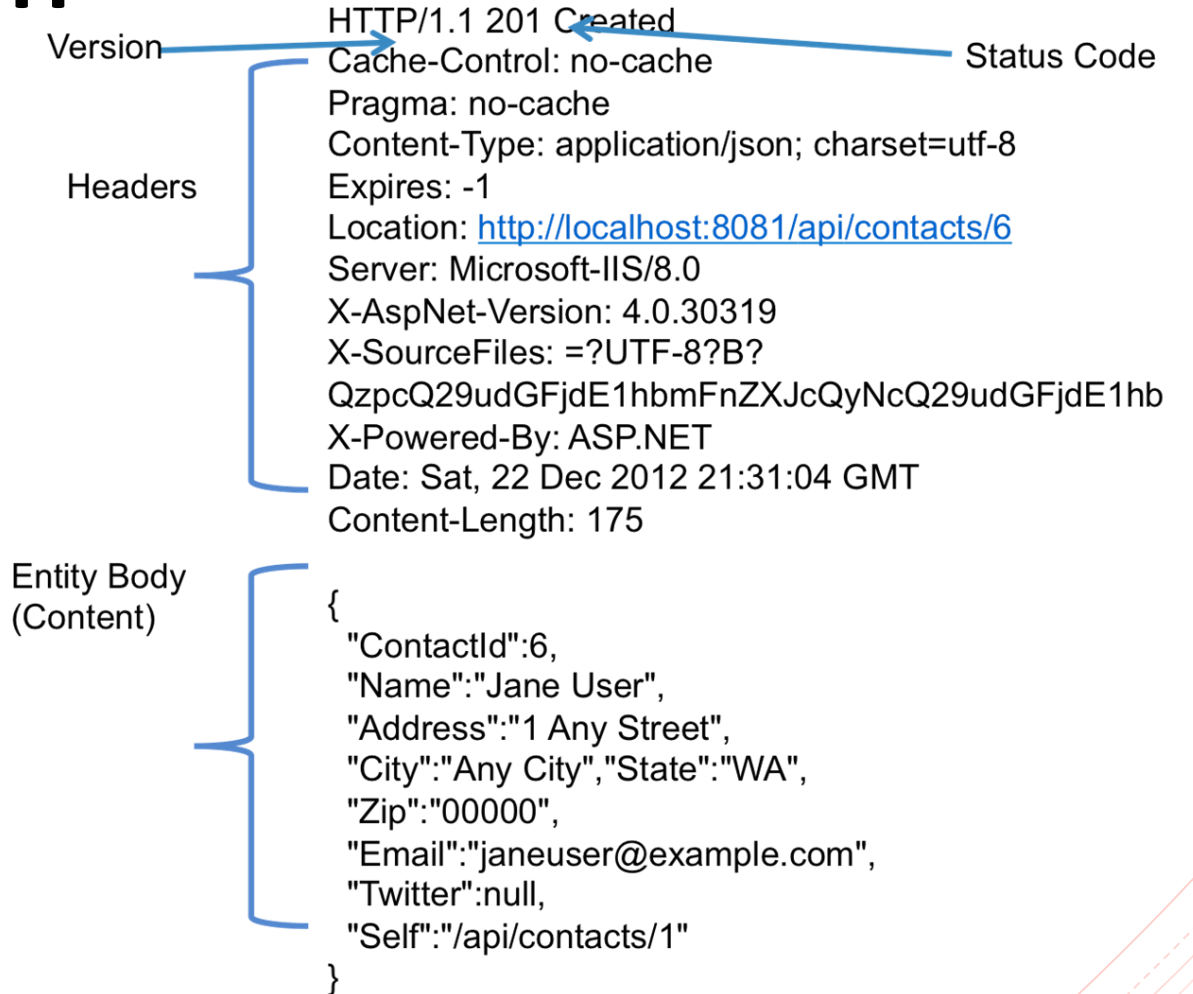
HTTP Responses

COSA SONO LE RISPOSTE HTTP

- Sono messaggi inviati dal server in risposta ad una richiesta inviata dal client.
- Hanno una struttura specifica e sono caratterizzate dal codice di risposta o status code che permette di comprendere l'esito della richiesta.

STRUTTURA DELLA RISPOSTA HTTP

- **Status code** della risposta
- **Status text:** versione human-readable dello status code
- Zero o più **headers**
- Un **body** (opzionale) che contiene i dati associati alla nostra richiesta.



STATUS CODE HTTP

- I codici delle risposte HTTP permettono al client di capire se una specifica richiesta HTTP è andata a buon fine.
- Sapere quali codici di risposta possono essere potenzialmente ricevuti permette di istruire l'applicazione per comportarsi in modi specifici a seconda del codice ricevuto.
- Gli status code sono codici a 3 cifre. La prima cifra permette di capire a che categoria appartiene lo status code:
 - 1xx (informativo): la richiesta è stata ricevuta, elaborazione in corso
 - 2xx (successo): la richiesta è stata ricevuta, compresa ed accettata con successo
 - 3xx (redirezione): per completare la richiesta servono altre azioni
 - 4xx (errore del client): la richiesta non può essere soddisfatta
 - 5xx (errore del server): il server non è riuscito a soddisfare una richiesta apparentemente valida.

Successful Requests

200 OK

La richiesta è avvenuta con successo. Indica un successo generico della richiesta. L'esatto significato di successo dipende dal metodo della richiesta, così come header e body della risposta. Il body di una 200 a seguito di una GET sarà la risorsa richiesta

201 Creato

La richiesta è andata a buon fine ed è stato creato una nuova risorsa il cui URI è di solito indicato nel body. Risposta tipica a seguito di una POST o PUT andata a buon fine

202 Accettato

La richiesta è stata ricevuta con successo, ma è ancora da elaborare.

203 Informazione non-autoritativa

La richiesta ha avuto successo ma le informazioni inviate al client circa la risposta vengono da un server di terze parti.

204 No content

La richiesta ha avuto successo ma non ci sono dati da restituire come risposta ma gli header potrebbero essere utili. Il client può ad esempio aggiornare gli header memorizzati nella cache per questa risorsa con quelle nuove.

205 Reset content

Il server richiede di resettare le informazioni inviate, come i dati del form

206 contenuto parziale

Risposta ad una richiesta solo per parte di un documento

Redirects

181

300 Scelte multiple

Il contenuto che è stato richiesto è stato spostato oppure ci sono più opzioni che corrispondono alla richiesta.

301 Spostato definitivamente

Il contenuto richiesto è stato spostato permanentemente e la risposta contiene la URI della nuova location. Importante quando si cambia il nome del dominio o gli URLs dei documenti esistenti.

302 Temporaneamente spostato

Il documento richiesto è stato temporaneamente spostato. L'URI della nuova location è inviato con la risposta. 303 e 307 sono versioni più specifiche di questo tipo.

303 Vedi altro (HTTP/1.1)

Il documento è stato trovato e la risposta contiene l'URI dove può essere trovato il documento

304 Non modificato

Il documento non è cambiato dall'ultima volta che è stato richiesto. Il client carica il documento dalla cache

305 Uso Proxy

IL documento richiesto può essere raggiunto solo tramite un proxy specifico

307 Redirect Temporanea (HTTP/1.1)

Il documento richiesto può essere temporaneamente trovato ad un URI differente che è passato nella risposta.

308 Redirect Permanente

La risorsa è stata spostata definitivamente in un altro URI che viene passato nella risposta. La differenza con 301 è che questa consente il cambio dal metodo POST a GET mentre il 308 no.

Errori del client

400 Bad Request

Il server non comprende la richiesta

401 Non autorizzato

UNAUTHORIZED: letteralmente non autorizzato ma semanticamente significa «non autenticato»: il client deve autenticarsi per ottenere la risposta richiesta.

403 Vietato

FORBIDDEN: il client non ha diritti di accesso al contenuto, ovvero non è autorizzato quindi il server si rifiuta di fornire la risorsa richiesta. A differenza del 401, l'identità del client è nota al server.

404 Not found

Il server non riesce a trovare la risorsa richiesta.

405 Metodo non consentito

Il metodo della richiesta non è consentito per la risorsa specificata

406 Non accettabile

La risorsa richiesta non può essere inviata in un modo che il client può comprendere

407 Richiesta autenticazione proxy

Il client deve essere autorizzato dal proxy prima che la risorsa richiesta possa essere spedita.

408 Request Timeout

Il tempo per la richiesta eccede il tempo per il quale il server è impostato per aspettare la richiesta

409 Conflitto

Il documento richiesto non può essere inviato a causa di un conflitto nella richiesta

Errori del client

152

410 Gone

Simile a 404 ma con motivazione diversa. In questo caso il contenuto richiesto è stato definitivamente eliminato dal server. Il client dovrebbe rimuovere le proprie cache ed i link alla risorsa. Importante per le strategie SEO perché indica ai crawler che quella risorsa può essere rimossa dalla SERP

411 Lunghezza richiesta

Richiesta rifiutata perché la lunghezza del contenuto deve essere specificata dal client.

412 Precondition Failed

Almeno una condizione della richiesta ha fallito

413 Request Entity Too Large

La richiesta è più grande di quanto il server possa gestire. .

414 Request URI too long

L'URI era più lunga di quanto il server potesse gestire.

415 Unsupported Media Type

IL formato di almeno parte della richiesta non è supportato

416 Requested Range Not Satisfiable

La richiesta non può essere soddisfatta. Può accadere che la richiesta del client sia parte di un documento che non esiste

417 Expectation Failed

Il server non può soddisfare i requisiti inviati nel campo «Expect» dell'header

Errori del server

500 Internal Server Error

Errore generico, indica che qualcosa non funziona ma non può essere inviato niente di più specifico.

501 Not Implemented

Il server non supporta ciò che serve per restituire la risposta

502 Bad Gateway

Il server che funge da gateway o proxy ha ricevuto una risposta da un server a monte ritenuta non valida.

503 Service Unavailable

Il server non è attualmente disponibile a causa di un carico eccessivo, manutenzione o altre situazioni temporanee

504 Gateway Timeout

IL server che funge da gateway o proxy non ha ricevuto risposte nel tempo prestabilito

505 HTTP Version Not Supported

Il server non supporta la versione HTTP usata dal client per la request.

1 - CREARE RESPONSES

1.a – STRINGHE ED ARRAYS

- Tutte le routes ed i controllers **dovrebbero ritornare una risposta da inviare al browser dell'utente.**

La risposta più basilare consiste nel ritornare una stringa da una route o da un controller. Laravel convertirà automaticamente la stringa in una risposta HTTP completa.

- Se si ritorna un **array**, Laravel lo convertirà **automaticamente in una risposta JSON**. Anche le collections Eloquent possono essere ritornate ed automaticamente convertite in JSON.

```
Route::get('/', function () {  
    return 'Hello World';  
});
```

```
Route::get('/', function () {  
    return [1, 2, 3];  
});
```

1.b RESPONSE OBJECTS

- Tipicamente, come risposte, ritorneremo istanze complete di `Illuminate\Http\Response` o `views`.
- Ritornare una `Response` completa consente di personalizzare lo status code e gli headers.
- Un'istanza di `Response` eredita dalla classe `Symfony\Component\HttpFoundation\Response` che fornisce diversi metodi per costruire le risposte HTTP

```
Route::get('/home', function () {  
    return response('Hello World', 200)  
        ->header('Content-Type', 'text/plain');  
});
```

1.c MODELLI E COLLECTIONS ELOQUENT

- Si può anche ritornare modelli e collections Eloquent direttamente dalle rotte e dai controller. Quando capita, Laravel converte automaticamente i modelli e le collections in risposte JSON rispettando gli hidden attributes del modello.

```
use App\Models\User;  
  
Route::get('/user/{user}', function (User $user) {  
    return $user;  
});
```


2 - REDIRECTS

- Le risposte redirect sono istanze della classe `Illuminate\Http\RedirectResponse` e contengono gli specifici header necessari al redirect dell'utente verso un altro url.
- Ci sono diversi modi per generare una `RedirectResponse`. Il modo più semplice è usare l'helper globale `redirect`:

```
Route::get('/dashboard', function () {  
    return redirect('home/dashboard');  
});
```

- Per reindirizzare l'utente alla **location precedente**, come ad un form invalido, si può usare l'helper globale `back`.
- Siccome questa caratteristica di Laravel utilizza le sessioni, assicurarsi che la rotta che chiama la funzione `back` stia usando il **web middleware group**.

```
Route::post('/user/profile', function () {  
    // Validate the request...  
  
    return back()->withInput();  
});
```

2.1 – REDIRECTS a NAMED ROUTES

- Quando si chiama l'helper `redirect` senza parametri, viene ritornata un'istanza della classe `Illuminate\Routing\Redirector` permettendo di chiamare qualsiasi metodo sull'istanza `Redirector`.
- Ad esempio, per creare un `RedirectResponse` ad una rotta con il nome, si può usare il metodo `route`:

```
return redirect()->route('login');
```

- Se la rotta ha dei **parametri**, questi possono essere passati **come secondo argomento** al metodo `route`.

```
// For a route with the following URI: /profile/{id}  
  
return redirect()->route('profile', ['id' => 1]);
```

2.1.1– POPOLARE PARAMETRI VIA ELOQUENT MODELS

- Se si sta redirezionando verso una rotta con un parametro ID che è stato popolato da un model Eloquent si può passare il modello stesso. L'ID sarà estratto automaticamente.

```
// For a route with the following URI: /profile/{id}

return redirect()->route('profile', [$user]);
```

- Per personalizzare il valore che si trova nel parametro della rotta, si può specificare la colonna nella definizione del parametro (`/profile/({id:slug})`) oppure si può sovrascrivere il metodo `getRouteKey` nel modello Eloquent.

```
/**
 * Get the value of the model's route key.
 */
public function getRouteKey(): mixed
{
    return $this->slug;
}
```

3- ALTRI TIPI DI RESPONSE

159

- L'helper **response** può essere usato per generare altri tipi di risposte. Quando l'helper è chiamato senza argomenti, viene ritornata un'implementazione del contract `Illuminate\Contracts\Routing\ResponseFactory`. Questo contract fornisce diversi metodi utili per generare risposte.

3.1 VIEW RESPONSE

- Se si vuole il controllo sullo stato della risposta e gli headers ma si vuole anche ritornare una view come contenuto della risposta, si usa il metodo **view**:

```
return response()  
    ->view('hello', $data, 200)  
    ->header('Content-Type', $type);
```

Se non si ha bisogno di passare uno status code o un header personalizzato, si può usare la funzione helper globale **view**

3.2 RISPOSTE JSON

- Il metodo **json** imposta automaticamente il Content-Type a application/json e converte l'array dato in un JSON usando la funzione php **json_encode**:

```
return response()->json([  
    'name' => 'Abigail',  
    'state' => 'CA',  
]);
```

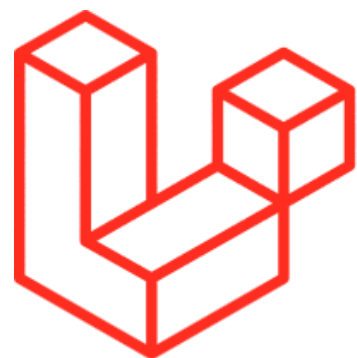
- Per creare una risposta JSONP (JSON with Padding) si usa il metodo **json** in combinazione con il metodo **withCallback**

```
return response()  
    ->json(['name' => 'Abigail', 'state' => 'CA'])  
    ->withCallback($request->input('callback'));
```

3.3 FILE DOWNLOADS

- Il metodo **download** può essere utilizzato per generare una risposta che forza il browser dell'utente a scaricare il file ad un **dato path**.
- Il metodo **accetta un filename** come secondo argomento che sarà il nome visualizzato dall'utente che scarica il file.
- Come **terzo argomento opzionale** si può passare un **array di headers HTTP**

```
return response()->download($pathToFile);  
  
return response()->download($pathToFile, $name, $headers);
```



Laravel

LE BASI

VALIDATION

Laravel fornisce diversi approcci per **validare i dati in ingresso**.

Comunemente si usa il metodo **validate**, disponibile per tutte le richieste HTTP in entrata.

Laravel include anche una gran varietà di regole di validazione che si possono applicare ai dati, consente addirittura di validare se i valori sono unici all'interno di una determinata tabella del database.

Per conoscere le funzionalità di validazione, faremo un esempio completo di validazione di un form e di visualizzazione di messaggi di errore.

I. DEFINIRE LE ROTTE

Ipotizziamo di avere le seguenti rotte definite nel file **routes/web.php**

```
use App\Http\Controllers\PostController;

Route::get('/post/create', [PostController::class, 'create']);
Route::post('/post', [PostController::class, 'store']);
```

La **rotta GET** mostrerà un **form per la creazione di un nuovo post** da parte dell'utente.

La **rotta POST** **memorizzerà il nuovo post nel database**

II. CREARE IL CONTROLLER

Abbiamo al momento un semplice controller che gestisce le richieste a queste rotte. Per ora lasciamo il metodo store vuoto.

```

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;

class PostController extends Controller
{
    /**
     * Show the form to create a new blog post.
     */
    public function create(): View
    {
        return view('post.create');
    }

    /**
     * Store a new blog post.
     */
    public function store(Request $request):
    RedirectResponse
    {
        // Validate and store the blog post...

        $post = /** ... */

        return to_route('post.show', ['post' =>
        $post->id]);
    }
}

```

III. SCRIVERE LE REGOLE DI VALIDAZIONE

Possiamo ora inserire nel metodo **store** la logica per validare il nuovo post.

Per farlo useremo il metodo **validate** fornito dall'oggetto `Illuminate\Http\Request`.

Se le regole di validazione passano, il codice continuerà ad essere eseguito normalmente; altrimenti, **se la validazione fallisce**, sarà lanciata **un'eccezione di validazione** (`Illuminate\Validation\ValidationException`) e l'opportuno **messaggio di errore** sarà mandato in risposta all'utente.

Se la validazione fallisce durante una normale richiesta HTTP, sarà generata una **risposta di redirect all'URL precedente**.

Se la richiesta in entrata è **una richiesta XHR**, sarà ritornata **una risposta JSON contenente il messaggio di errore di validazione**.


```

1  /**
2   * Store a new blog post.
3   */
4  public function store(Request $request): RedirectResponse
5  {
6      $validated = $request->validate([
7          'title' => 'required|unique:posts|max:255',
8          'body' => 'required',
9      ]);
10
11     // The blog post is valid...
12
13     return redirect('/posts');
14 }

```

Le regole di validazione sono passate dentro il metodo **validate**.

Possono essere specificate anche come **array di regole** invece che come una **stringa singola** delimitata da |

Vedi elenco regole di validazione

```

1  $validatedData = $request->validate([
2      'title' => ['required', 'unique:posts', 'max:255'],
3      'body' => ['required'],
4  ]);

```

FERMARSÌ ALLA PRIMA VALIDAZIONE FALLITA

Per fermare le regole di validazione su un attributo dopo il primo fallimento di validazione, si assegna la regola **bail** all'attributo.

Nell'esempio, se la regola **unique** fallisce, **max** non sarà controllato

```

1  $request->validate([
2      'title' => 'bail|required|unique:posts|max:255',
3      'body' => 'required',
4  ]);

```

FORM REQUEST VALIDATION

1-Creare Form Request

Per validazioni più complesse si potrebbe voler creare un "form request".

COSA SONO LE FORM REQUEST?

Sono classi request custom che incapsulano le loro proprie logiche di validazione ed autorizzazione.

Per creare una form request si usa il comando **Artisan make:request**

```
php artisan make:request StorePostRequest
```

La classe form request creata sarà posizionata nella directory **app/http/Requests**. Se questa directory non esiste, viene creata quando si lancia il comando **make:request**

Ogni form request generata da Laravel ha **2 metodi: authorize e rules.**

- **Metodo authorize:** si occupa di determinare se l'utente attualmente autenticato può svolgere le azioni presenti nella request
- **Metodo rules:** ritorna le regole di validazione che dovrebbero essere applicate ai dati della request.

```
/**
 * Get the validation rules that apply to the request.
 *
 * @return array<string, \Illuminate\Contracts\Validation\Rule|array|string>
 */
public function rules(): array
{
    return [
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ];
}
```

PERSONALIZZARE LA LOCATION DEL REDIRECT

La proprietà `$redirect` della form request permette di personalizzare il redirect conseguente ad un fallimento nella validazione.

```
/**
 * The URI that users should be redirected to if
 * validation fails.
 *
 * @var string
 */
protected $redirect = '/dashboard';
```

Per personalizzare una named route si usa invece la proprietà `$redirectRoute`

```
/**
 * The route that users should be redirected to if
 * validation fails.
 *
 * @var string
 */
protected $redirectRoute = 'dashboard';
```

2 – Autorizzare le Form Requests

Attraverso il metodo **authorize** della classe form request si può determinare su un utente già autenticato ha effettivamente l'autorità di aggiornare una determinata risorsa. Ad esempio, si può determinare se un commento ad un post appartiene all'utente e se questo ha effettivamente il permesso di modificarlo.

All'interno di questo metodo si interagirà con le policies ed i gates di autorizzazione:

```
use App\Models\Comment;

/**
 * Determine if the user is authorized to make this request.
 */
public function authorize(): bool
{
    $comment = Comment::find($this->route('comment'));

    return $comment && $this->user()->can('update',
    $comment);
}
```

Tutte le form requests estendono la classe request di Laravel; per questo motivo è possibile usare il metodo **user** per accedere all'attuale utente autenticato. Il metodo **route** permette invece di accedere ai parametri dell'URI definiti sulla rotta che viene richiamata.

```
Route::post('/comment/{comment}');
```

Inoltre, l'utilizzo del route model binding permette di accedere al model come proprietà della request.

```
return $this->user()->can('update', $this->comment);
```

Se il metodo **authorize** ritorna false, si avrà una risposta HTTP con **status code 403** ed il metodo controller non sarà eseguito.

Se si desidera gestire la logica delle autorizzazioni in un'altra parte dell'applicazione, si può rimuovere completamente il metodo `authorize` o ritornare **true** dal metodo **authorize**

```
/**
 * Determine if the user is authorized to make this request.
 */
public function authorize(): bool
{
    return true;
}
```

3 – Personalizzare i messaggi di errore

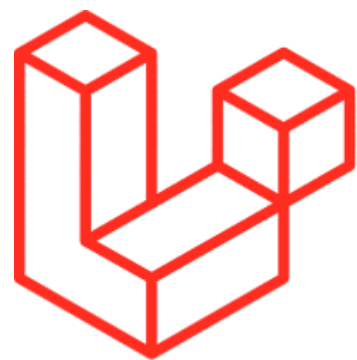
Per personalizzare i messaggi di errore di validazione si dovrà sovrascrivere il metodo **messages** che ritorna un array di coppie attributo/regola e i loro messaggi di errore corrispondenti.

```
/**
 * Get the error messages for the defined validation rules.
 */
@return array<string, string>
public function messages(): array
{
    return [
        'title.required' => 'A title is
required',
        'body.required' => 'A message is
required',
    ];
}
```

PERSONALIZZARE GLI ATTRIBUTI DI VALIDAZIONE

Molti dei messaggi di errore di validazione nativi di Laravel possiedono un placeholder `:attribute`. Se lo si vuole sostituire con un nome di attributo personalizzato, si specifica il nome desiderato sovrascrivendo il metodo `attributes`. Questo metodo ritorna un array di coppie attributo/nome.

```
/**
 * Get custom attributes for validator errors.
 *
 * @return array<string, string>
 */
public function attributes(): array
{
    return [
        'email' => 'email address',
    ];
}
```



Laravel

MIGRATIONS

- Una migrazione è un processo che permette di creare o modificare lo schema di una tabella.
- Le migrations possono essere «versionate» rendendo semplici operazioni di rollback e di deploy.
- Laravel offre pieno supporto alle migrations attraverso la **facade Schema** e vari comandi Artisan per la creazione o modifica delle migrazioni.
- Le migrazioni create si trovano nella cartella **database\migration**. Ogni filename delle migrazioni contiene un timestamp che consente di ordinare le migrazioni.

GENERARE MIGRATIONS

```
php artisan make:migration create_flights_table  
Schema::create('flights' ...
```

```
INFO Migration  
[database/migrations/2023_01_23_193847_create_posts_table.php] created successfully.
```

- Il nome indicato per la migrazione (**create_flights_table**) sarà usato da Laravel per cercare di capire il nome della tabella e se la migrazione sta creando una nuova tabella o no.
- Se Laravel è in grado di determinare il nome della tabella dal nome della migration, il file della migrazione sarà precompilato con la tabella specificata. Altrimenti si può specificare la tabella nel file della migrazione manualmente.
- Per specificare il path personalizzato di una migration, si può usare l'opzione **--path** eseguendo il comando **make:migration**. Il percorso dovrebbe essere relativo al base path dell'applicazione.

STRUTTURA DELLA MIGRATION

- Una classe migration contiene due metodi: **up** e **down**.
- Metodo **up**: indica le **modifiche da applicare quando la migrazione «va su»**. Le modifiche possono essere aggiunte di tabelle/ colonne/ indici, rimozioni o cambi.
- Metodo **down**: indica le **modifiche da applicare quando si vuole annullare le modifiche apportate dal metodo up**
- Nell'esempio a fianco viene creata la tabella flights
- Se la migration interagisce con altri database oltre a quello di default, si imposta la proprietà **\$connection** della migration

Migration file creato con il comando `php artisan make:migration create_flights_table` 181

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('flights', function (Blueprint $table) {
15             $table->id();
16             $table->string('name');
17             $table->string('airline');
18             $table->timestamps();
19         });
20     }
21
22     /**
23      * Reverse the migrations.
24      */
25     public function down(): void
26     {
27         Schema::drop('flights');
28     }
29 };
```


LANCIARE MIGRATIONS

- Per lanciare tutte le migrations in sospeso si usa il comando Artisan `migrate`

```
php artisan migrate
```

- Comando `migrate:status` per vedere le migrazioni lanciate fino a quel momento
- Per vedere le istruzioni SQL che saranno eseguite dalle migrations senza veramente eseguirle, si può aggiungere il flag `--pretend` al comando migrate

```
php artisan migrate --pretend
```

ROLLING BACK DELLE MIGRATIONS

- Per eseguire il roll back delle ultime migrazioni, si usa il comando Artisan `rollback`. Questo comando esegue il rollback dell'ultimo gruppo di migrazioni.

```
php artisan migrate:rollback
```

- L'opzione `--step` del comando `rollback` permette di tornare¹⁸² indietro di un dato numero di migrazioni.

```
php artisan migrate:rollback --step=5
```

- Specificando l'opzione `--batch` si può eseguire il rollback di un determinato gruppo di migrations. Il valore dell'opzione batch corrisponde al valore del batch all'interno della tabella migrations del database

```
php artisan migrate:rollback --batch=3
```

- Il comando `migrate:reset` eseguirà il roll back di tutte le migrazioni

```
php artisan migrate:reset
```

ROLL BACK E MIGRATE CON UN UNICO COMANDO

- Il comando `migrate:refresh` farà il roll back di tutte le migrazioni e poi eseguirà il comando `migrate`
- Questo comando **ricrea l'intero database**

```
php artisan migrate:refresh
```

```
# Refresh the database and run all database seeds...  
php artisan migrate:refresh --seed
```

- L'opzione **--step** eseguirà il roll back e migrerà nuovamente un limitato numero di migrations

```
php artisan migrate:refresh --step=5
```

TABELLE

CREARE TABELLE

- Per creare una nuova tabella del database si usa il metodo **create** sulla **facade Schema**. Il metodo accetta due argomenti:
 1. Il nome della tabella
 2. Una closure che riceve un oggetto **Blueprint** usato per definire la nuova tabella

CONTROLLARE L'ESISTENZA DI TABELLE/COLONNE

- Si usano i metodi **hasTable** e **hasColumn** e **hasIndex**

ELIMINARE TUTTE LE TABELLE E MIGRARE

- Il comando **migrate:fresh** eliminerà tutte le tabelle del database e poi eseguirà il comando **migrate**. Elimina solo le tabelle presenti sul database di default

```
php artisan migrate:fresh
```

```
php artisan migrate:fresh --seed
```

```
1 use Illuminate\Database\Schema\Blueprint;
2 use Illuminate\Support\Facades\Schema;
3
4 Schema::create('users', function (Blueprint $table) {
5     $table->id();
6     $table->string('name');
7     $table->string('email');
8     $table->timestamps();
9 });
```

```
1 if (Schema::hasTable('users')) {
2     // The "users" table exists...
3 }
4
5 if (Schema::hasColumn('users', 'email')) {
6     // The "users" table exists and has an "email" column...
7 }
8
9 if (Schema::hasIndex('users', ['email'], 'unique')) {
10    // The "users" table exists and has a unique index on the "email" column...
11 }
```

CONNESSIONE AL DATABASE E OPZIONI DELLA TABELLA

- Per effettuare un'operazione su un database che non è quello di default, si usa il metodo **connection**
- Le proprietà **charset** e **collation*** sono usate per specificare il set di caratteri e la collation per la tabella che si sta creando.
**la collation è un insieme di regole che determinano come vengono confrontati i caratteri, l'ordinamento dei dati testuali, la sensibilità a maiuscole e minuscole, la sensibilità agli accenti.*
- La proprietà **engine** può essere utilizzata per specificare il motore di archiviazione della tabella quando si usa MariaDB o MySQL.
- Il metodo **temporary** indica che la tabella deve essere temporanea: sarà visibile solo per la sessione attuale di connessione al database e sarà eliminata automaticamente quando la connessione viene chiusa
- Il metodo **comment** serve per aggiungere un commento alla tabella del database. Supportati solo da MariaDB, MySQL e PostgreSQL

```
1 Schema::connection('sqlite')->create('users', function (Blueprint $table) {
2     $table->id();
3 });
```

```
1 Schema::create('users', function (Blueprint $table) {
2     $table->charset('utf8mb4');
3     $table->collation('utf8mb4_unicode_ci');
4
5     // ...
6 });
```

```
1 Schema::create('users', function (Blueprint $table) {
2     $table->engine('InnoDB');
3
4     // ...
5 });
```

```
1 Schema::create('calculations', function (Blueprint $table) {
2     $table->temporary();
3
4     // ...
5 });
```

```
1 Schema::create('calculations', function (Blueprint $table) {
2     $table->comment('Business calculations');
3
4     // ...
5 });
```

AGGIORNARE TABELLE

- Il metodo **table** della facade **Schema** è usato per aggiornare tabelle esistenti.
Accetta **due argomenti**: il **nome della tabella** ed una closure che riceve un **oggetto Blueprint** usato per aggiungere colonne o indici

```
1 use Illuminate\Database\Schema\Blueprint;  
2 use Illuminate\Support\Facades\Schema;  
3  
4 Schema::table('users', function (Blueprint $table) {  
5     $table->integer('votes');  
6 });
```

RINOMINARE / ELIMINARE TABELLE

- Il metodo **rename** è usato per rinominare una tabella del database
- Per eliminare una tabella si usano i metodi **drop** o **dropIfExists**

```
1 use Illuminate\Support\Facades\Schema;  
2  
3 Schema::rename($from, $to);
```

```
1 Schema::drop('users');  
2  
3 Schema::dropIfExists('users');
```

- Prima di rinominare una tabella, bisognerebbe verificare che **tutti i vincoli di chiave esterna** sulla **tabella** abbiano un **nome esplicito** definito nella migration, invece di lasciare che Laravel assegni automaticamente un nome. In caso contrario, il nome del vincolo di chiave esterna farà riferimento al vecchio nome della tabella

COLONNE

- Il metodo **table** può essere usato per modificare tabelle esistenti. Così come il metodo **create**, accetta **due argomenti**: il **nome della tabella** ed una closure che riceve un **oggetto Blueprint** usato per aggiungere colonne alla tabella

```
1 use Illuminate\Database\Schema\Blueprint;
2 use Illuminate\Support\Facades\Schema;
3
4 Schema::table('users', function (Blueprint $table) {
5     $table->integer('votes');
6 });
```

Metodo	Tipo SQL Equivalente	Esempio	Note
<code>bigIncrements()</code>	UNSIGNED BIGINT (PK, Auto Increment)	<code>\$table->bigIncrements('id');</code>	Chiave primaria con incremento automatico
<code>bigInteger()</code>	BIGINT	<code>\$table->bigInteger('votes');</code>	
<code>boolean()</code>	BOOLEAN	<code>\$table->boolean('confirmed');</code>	
<code>char()</code>	CHAR (length)	<code>\$table->char('name', length: 100);</code>	Con definizione lunghezza
<code>dateTime()</code>	DATETIME	<code>\$table->dateTime('created_at', precision: 0);</code>	Supporta precision: definizione frazioni di secondo tramite numero di cifre con cui vengono rappresentati.
<code>date()</code>	DATE	<code>\$table->date('created_at');</code>	
<code>decimal()</code>	DECIMAL(total, places)	<code>\$table->decimal('amount', total: 8, places: 2);</code>	Indicazione delle cifre totali e numero di decimali
<code>double()</code>	DOUBLE	<code>\$table->double('amount');</code>	
<code>enum()</code>	ENUM([...])	<code>\$table->enum('difficulty', ['easy', 'hard']);</code>	
<code>float()</code>	FLOAT(precision)	<code>\$table->float('amount', precision: 53);</code>	Precision: numero di cifre
<code>foreignId()</code>	UNSIGNED BIGINT	<code>\$table->foreignId('user_id');</code>	
<code>foreignIdFor()</code>	Tipo dipende dal model key (BIGINT, CHAR(36), CHAR(26))	<code>\$table->foreignIdFor(User::class);</code>	Aggiunge una colonna equivalente a {colonna}_id per un determinato model. Il tipo di colonna creato dipende dal tipo di chiave primaria del modello.
<code>id()</code>	Alias di <code>bigIncrements()</code>	<code>\$table->id();</code>	Alias di <code>bigIncrements()</code> . Di default, crea una colonna id. Si può passare il nome della colonna per assegnarle un nome differente.
<code>increments()</code>	UNSIGNED INTEGER (PK, Auto Increment)	<code>\$table->increments('id');</code>	Crea una colonna auto-incrementing UNSIGNED INTEGER come primary key
<code>integer()</code>	INTEGER	<code>\$table->integer('votes');</code>	
<code>ipAddress()</code>	VARCHAR, INET (su PostgreSQL)	<code>\$table->ipAddress('visitor');</code>	
<code>json()</code>	JSON, TEXT (su SQLite)	<code>\$table->json('options');</code>	
<code>longText()</code>	LONGTEXT, LONGBLOB (con charset binario)	<code>\$table->longText('description');</code>	
<code>mediumIncrements()</code>	UNSIGNED MEDIUMINT (PK, Auto Increment)	<code>\$table->mediumIncrements('id');</code>	Crea una colonna auto-incrementing UNSIGNED MEDIUMINT come primary key
<code>mediumInteger()</code>	MEDIUMINT	<code>\$table->mediumInteger('votes');</code>	

mediumText()	MEDIUMTEXT, MEDIUMBLOB (con charset binario)	\$table->mediumText('description');	
morphs()	{col}_id, {col}_type	\$table->morphs('taggable');	188 Per relazioni polimorfiche Eloquent. Aggiunge due colonne: {column}_id e {column}_type di tipo VARCHAR che memorizza il nome del model a cui è associata la relazione.
rememberToken()	VARCHAR(100) nullable	\$table->rememberToken();	Per memorizzazione token "remember me"
set()	SET([...])	\$table->set('flavors', ['strawberry', 'vanilla']);	Crea una colonna con una lista di valori validi. Sono accettati più valori contemporaneamente. Enum accetta solo un valore
smallIncrements()	UNSIGNED SMALLINT (PK, Auto Increment)	\$table->smallIncrements('id');	
smallInteger()	SMALLINT	\$table->smallInteger('votes');	
softDeletes()	TIMESTAMP nullable	\$table->softDeletes('deleted_at', precision: 0);	Per soft delete senza timezone. Aggiunge una colonna deleted_at di tipo TIMESTAMP usato per i soft delete di Eloquent
string()	VARCHAR(length)	\$table->string('name', length: 100);	
text()	TEXT, BLOB (con charset binario)	\$table->text('description');	
time()	TIME	\$table->time('sunrise', precision: 0);	Supporta precision
timestamp()	TIMESTAMP	\$table->timestamp('added_at', precision: 0);	Supporta precision
timestamps()	created_at, updated_at	\$table->timestamps(precision: 0);	Crea le colonne created_at e updated_at
\$table->tinyInteger('votes');	\$table->tinyInteger('votes');	\$table->tinyInteger('votes');	\$table->tinyInteger('votes');
tinyText()	TINYTEXT, TINYBLOB (con charset binario)	\$table->tinyText('notes');	
unsignedBigInteger()	UNSIGNED BIGINT	\$table->unsignedBigInteger('votes');	Presenti anche i metodi unsignedMediumInteger, unsignedSmallInteger, unsignedTinyInteger
unsignedInteger()	UNSIGNED INTEGER	\$table->unsignedInteger('votes');	
ulid()	ULID	\$table->ulid('id');	Universally Unique Lexicographically Sortable Identifier
uuid()	UUID	\$table->uuid('id');	Universally Unique Identifier
vector()	VECTOR(dimensions)	\$table->vector('embedding', dimensions: 100);	
year()	YEAR	\$table->year('birth_year');	

MODIFICATORI DI COLONNA

- Oltre ai tipi di colonna, esistono diversi "modificatori" di colonna utilizzabili quando si aggiunge una colonna ad una tabella. Si può, ad esempio, utilizzare il metodo **nullable** per rendere una colonna nullable.

MODIFIER	DESCRIPTION
->after('column')	Posiziona una colonna dopo un'altra colonna. (MariaDB / MySQL).
->autoIncrement()	Imposta colonne INTEGER come auto-incrementing (primary key).
->charset('utf8mb4')	Specifica un character set per la colonna(MariaDB / MySQL).
->collation('utf8mb4_unicode_ci')	Specifica una collation per la Colonna
->comment('my comment')	Aggiunge un comment ad una Colonna (MariaDB / MySQL / PostgreSQL).
->default(\$value)	Specifica un valore di "default" per la Colonna
->first()	Mette la Colonna al primo posto della tabella (MariaDB / MySQL).
->from(\$integer)	Imposta il valore iniziale di un campo auto-increment (MariaDB / MySQL / PostgreSQL).
->invisible()	Rende la Colonna "invisible" alle queries di SELECT * (MariaDB / MySQL).
->nullable(\$value = true)	Permette ai valori NULL di essere inseriti nella tabella.
->storedAs(\$expression)	Crea una Colonna generate con valori memorizzati nel database(MariaDB / MySQL / PostgreSQL / SQLite).
->unsigned()	Imposta colonne INTEGER come UNSIGNED (MariaDB / MySQL).
->useCurrent()	Imposta le colonne TIMESTAMP per l'uso CURRENT_TIMESTAMP come valore di default.
->useCurrentOnUpdate()	Imposta colonne TIMESTAMP ad usare CURRENT_TIMESTAMP quando un record viene aggiornato (MariaDB / MySQL).
->virtualAs(\$expression)	Crea una Colonna virtuale(MariaDB / MySQL / SQLite).
->generatedAs(\$expression)	Crea una colonna con i valori derivati da una specifica sequenza(PostgreSQL).
->always()	Definisce la precedenza dei valori della sequenza (PostgreSQL).

INDICI (vedi tipi di indici)

- Laravel supporta **diversi tipi di indici** attribuibili sia in fase di creazione tabelle che in seguito.

```
1 use Illuminate\Database\Schema\Blueprint;
2 use Illuminate\Support\Facades\Schema;
3
4 Schema::table('users', function (Blueprint $table) {
5     $table->string('email')->unique();
6 });
```

- Nell'esempio viene creata la colonna email, all'interno della tabella users, e viene specificato che il suo valore deve essere unico utilizzando il metodo **unique** unito alla definizione della colonna.

- Alternativamente, si può creare l'indice dopo la definizione della colonna. In questo caso si invoca il metodo **unique** che accetta il **nome della colonna** come parametro.

```
1 $table->unique('email');
```

- Si può passare un **array di colonne** al metodo **index** per creare un **indice composto**. In questo caso si invoca il metodo **unique** che accetta il nome della colonna come parametro.

```
1 $table->index(['account_id', 'created_at']);
```

- Quando si crea un indice, Laravel genererà automaticamente il nome di un indice basato sulla tabella, sui nomi della colonna e sul tipo di indice. È possibile passare un secondo argomento per personalizzare il nome dell'indice

```
1 $table->unique('email', 'unique_email');
```

VINCOLI DI CHIAVE ESTERNA (Foreign Key Constraint)

- Laravel fornisce un metodo semplice per la definizione di Foreign Keys grazie all'utilizzo del metodo `foreignId`.
- Viene creata una colonna **UNSIGNED BIGINT**. Il metodo `constrained` usa le convenzioni per stabilire la tabella e la colonna che vengono referenziate

```
1 Schema::table('posts', function (Blueprint $table) {
2     $table->foreignId('user_id')->constrained();
3 });
```

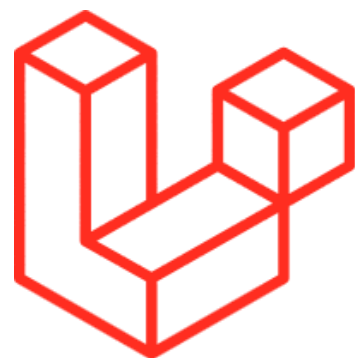
- Se il nome della tabella non rispetta le convenzioni di Laravel, la si può indicare come argomento del metodo `constrained`. Può anche essere specificato il nome da assegnare all'indice.

```
1 Schema::table('posts', function (Blueprint $table) {
2     $table->foreignId('user_id')->constrained(
3         table: 'users', indexName: 'posts_user_id'
4     );
5 });
```

- È inoltre possibile indicare le azioni da eseguire per le proprietà **"on delete"** e **"on update"** del vincolo. Esiste anche una sintassi semplificata

```
1 $table->foreignId('user_id')
2     ->constrained()
3     ->onUpdate('cascade')
4     ->onDelete('cascade');
```

METHOD	DESCRIPTION
<code>\$table->cascadeOnUpdate();</code>	Updates should cascade.
<code>\$table->restrictOnUpdate();</code>	Updates should be restricted.
<code>\$table->nullOnUpdate();</code>	Updates should set the foreign key value to null.
<code>\$table->noActionOnUpdate();</code>	No action on updates.
<code>\$table->cascadeOnDelete();</code>	Deletes should cascade.
<code>\$table->restrictOnDelete();</code>	Deletes should be restricted.
<code>\$table->nullOnDelete();</code>	Deletes should set the foreign key value to null.
<code>\$table->noActionOnDelete();</code>	Prevents deletes if child records exist.



Laravel

DATABASE

- Praticamente tutte le applicazioni web moderne interagiscono con un database.
- Laravel rende l'interazione con i database estremamente semplice grazie all'utilizzo di **SQL puro**, un **query builder** e l'**ORM Eloquent**.
- Attualmente, Laravel fornisce supporto diretto per 5 database:
 - MariaDB 10.3+ ([Version Policy](#))
 - MySQL 5.7+ ([Version Policy](#))
 - PostgreSQL 10.0+ ([Version Policy](#))
 - SQLite 3.26.0+
 - SQL Server 2017+ ([Version Policy](#))
- MongoDB è supportato grazie al pacchetto **mongodb/laravel-mongodb**, ufficialmente mantenuto da MongoDB.

CONFIGURAZIONE

- La configurazione database di Laravel si trova nel file di configurazione **config/database.php** nel quale vengono definite le connessioni con i database e viene indicato quale connessione deve essere usata di default. Molte delle opzioni di configurazione sono determinate dai valori delle variabili d'ambiente dell'applicazione (file **.env**).

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=
```

OPZIONE STICKY

- L'opzione **sticky** serve quando si utilizzano database separati per la lettura e per la scrittura. Se **sticky** è attiva, dopo che è stata effettuata una scrittura, le letture successive utilizzeranno lo stesso database di scrittura, anche se di solito dovrebbero essere fatte tramite il database di lettura. Questo permette di leggere subito i dati appena scritti, senza aspettare che vengano sincronizzati con il database di lettura.

```
1  'mysql' => [
2      'read' => [
3          'host' => [
4              '192.168.1.1',
5              '196.168.1.2',
6          ],
7      ],
8      'write' => [
9          'host' => [
10             '196.168.1.3',
11         ],
12     ],
13     'sticky' => true,
14
15     'database' => env('DB_DATABASE', 'laravel'),
16     'username' => env('DB_USERNAME', 'root'),
17     'password' => env('DB_PASSWORD', ''),
18     'unix_socket' => env('DB_SOCKET', ''),
19     'charset' => env('DB_CHARSET', 'utf8mb4'),
20     'collation' => env('DB_COLLATION', 'utf8mb4_unicode_ci'),
21     'prefix' => '',
22     'prefix_indexes' => true,
23     'strict' => true,
24     'engine' => null,
25     'options' => extension_loaded('pdo_mysql') ? array_filter([
26         PDO::MYSQL_ATTR_SSL_CA => env('MYSQL_ATTR_SSL_CA'),
27     ]) : [],
28 ],
```

1- ESEGUIRE QUERY SQL

Una volta configurato la connessione database, si possono lanciare le queries usando la facade **DB** che fornisce metodi per ogni tipo di query: **select**, **update**, **insert**, **delete** e **statement**.

1.1 - QUERY SELECT

Per lanciare una query SELECT base si usa il metodo **select** sulla facade DB.

Esso accetta la query SQL da eseguire come primo argomento, il secondo è il bindings dei parametri. Essi sono di solito i valori della clausola where.

Il binding dei parametri protegge dalla SQL injection.

Il metodo **DB::select** ritorna sempre **un array di risultati**, ciascuno dei quali è un oggetto **stdClass** che rappresenta un record del database.

Il metodo **DB::scalar** ritorna un **singolo valore scalare**, utile per le query «count».

```
namespace App\Http\Controllers;

use Illuminate\Support\Facades\DB;
use Illuminate\View\View;

class UserController extends Controller
{
    /**
     * Show a list of all of the application's users.
     */
    public function index(): View
    {
        $users = DB::select('select * from users where active = ?', [1]);
        $count = DB::scalar('select count(1) from posts where active = :active',
            ['active' => 1]);

        return view('user.index', ['users' => $users]);
    }
}
```

```
1 use Illuminate\Support\Facades\DB;
2
3 $users = DB::select('select * from users');
4
5 foreach ($users as $user) {
6     echo $user->name;
7 }
```

```
1 $burgers = DB::scalar(
2     "select count(case when food = 'burger' then 1 end) as burgers from menu"
3 );
```

UTILIZZO DEL NAMED BINDINGS

invece di usare **?** per il binding dei parametri, si può eseguire la query usando il **named binding**

```
$results = DB::select('select * from users where id = :id', ['id' => 1]);
```

LANCIARE UN INSERT

Si usa il metodo **DB::insert** della facade DB. Come il metodo select, accetta una query SQL come primo argomento e i bindings come secondo

```
use Illuminate\Support\Facades\DB;

DB::insert('insert into users (id, name) values (?, ?)', [1, 'Marc']);
```

LANCIARE UN UPDATE

Si usa il metodo **DB::update** per aggiornare records già esistenti nel database. Ritorna il numero delle righe interessate dall'aggiornamento.

```
use Illuminate\Support\Facades\DB;

$affected = DB::update(
    'update users set votes = 100 where name = ?',
    ['Anita']
);
```

LANCIARE UN DELETE

Si usa il metodo **DB::delete** per eliminare records dal database. Ritorna il numero delle righe eliminate.

```
use Illuminate\Support\Facades\DB;

$deleted = DB::delete('delete from users');
```

LANCIARE UNO STATEMENT GENERALE

Alcune istruzioni del database **non ritornano alcun valore**. Per questo tipo di operazioni si può usare il metodo **DB::statement**

```
DB::statement('drop table users');
```

LANCIARE UN'ISTRUZIONE UNPREPARED

Se si vuole eseguire un'istruzione SQL senza il binding dei valori, si può utilizzare il metodo **DB::unprepared**.

Siccome il binding dei parametri non viene eseguito, essi sono vulnerabili ad attacchi di SQL injection.

```
DB::unprepared('update users set votes = 100 where name = "Dries");
```

ORM ELOQUENT

Object Relational Mapping

- L'ORM (Object Relational Mapping) è una **tecnica di programmazione** che permette di **mettere in relazione una tabella di un database relazionale con un oggetto**.
- Laravel utilizza l'ORM **Eloquent** che si basa sulla corrispondenza tra una tabella del database e una **classe di tipo "Model"**.
- Nello specifico, **una tabella sul database corrisponde ad una classe PHP** e **una singola riga della tabella corrisponde ad una istanza della classe**.
- L'utilizzo di Eloquent e dei Model semplifica molto l'interazione con il database, eliminando, in molti casi, la necessità di scrivere query dedicate.

ELOQUENT – FEATURES PRINCIPALI

1. MODEL DRIVEN: ogni tabella del database è associata ad un modello corrispondente. I modelli definiscono la **struttura** ed il **comportamento dei dati** e permettono di lavorare con i record come oggetti

2. CREAZIONE DI QUERY ESPRESSIVE: fornisce un costruttore di query espressivo che consente la **costruzione di query complesse utilizzando una sintassi semplice** e leggibile evitando SQL puro nella maggior parte dei casi

3. RELAZIONI: Eloquent consente di definire e lavorare con le relazioni del database in modo semplice. Le relazioni sono definite come **metodi del Model** facilitando il recupero dei dati.

4. CONVALIDA: Eloquent include un **supporto integrato per la convalida dei dati**, assicurando che **solo** i dati validi vengano inseriti nel database contribuendo all'integrità dei dati.

5. EVENTI ELOQUENT: è possibile **collegare gli event listeners ai models** per eseguire azioni prima o dopo determinati eventi del model, come il salvataggio, eliminazione o creazione di un record.

CREARE UN MODEL

- Prima di utilizzare Eloquent bisogna **impostare la connessione al database** nei file di configurazione di Laravel. Laravel supporta **connessioni multiple** al database, configurabili nel file **config/database.php**.
- Una volta configurata la connessione, si creano modelli per rappresentare le tabelle del database.
- **Per collegare tabella e model in Eloquent, si utilizza una convenzione sui rispettivi nomi.**
- La **classe** del **modello** è indicata al **singolare**
- La **tabella** corrispondente è indicata al **plurale** con **snake case**
- Comando Artisan per creare un nuovo model: **artisan make:model ModelName**
- Il comando **make:model** supporta varie **opzioni** per creare classi collegate al model (controller, form....). Per creare un nuovo modello con annessa migration e, quindi, per creare automaticamente anche la nuova tabella si usa l'opzione **-m** (o **--migration**), se si vuole aggiungere il controller **-c** (o **--controller**), per indicare se deve essere una risorsa **-r** (o **--resource**)(CRUD methods)

```
$ php artisan make:model Post -m
```

```
INFO Model [app/Models/Post.php] created  
successfully.
```

Viene creato un nuovo file model nella cartella **app/Models**. In questo file si possono **definire le proprietà e le relazioni del modello**.

OPZIONI

Si possono generare diversi tipi di classi quando si genera un model, come seeders, factories, controllers. Queste opzioni possono anche essere combinate.

Il comando Artisan **model:show** rappresenta un metodo comodo per visualizzare tutte le relazioni

Puoi anche creare model con migration e controller insieme:

bash

```
php artisan make:model Product -mc
```

Significato:

- **-m** → crea anche la migration
- **-c** → crea anche il controller



```
1 # Generate a model and a FlightFactory class...
2 php artisan make:model Flight --factory
3 php artisan make:model Flight -f
4
5 # Generate a model and a FlightSeeder class...
6 php artisan make:model Flight --seed
7 php artisan make:model Flight -s
8
9 # Generate a model and a FlightController class...
10 php artisan make:model Flight --controller
11 php artisan make:model Flight -c
12
13 # Generate a model, FlightController resource class, and form request classes...
14 php artisan make:model Flight --controller --resource --requests
15 php artisan make:model Flight -crR
16
17 # Generate a model and a FlightPolicy class...
18 php artisan make:model Flight --policy
19
20 # Generate a model and a migration, factory, seeder, and controller...
21 php artisan make:model Flight -mfsc
22
23 # Shortcut to generate a model, migration, factory, seeder, policy, controller, and form requests...
24 php artisan make:model Flight --all
25 php artisan make:model Flight -a
26
27 # Generate a pivot model...
28 php artisan make:model Member --pivot
29 php artisan make:model Member -p
```

CONVENZIONI MODELLI ELOQUENT

- **NOMI TABELLE**: per convenzione, il **nome plurale e snake case** del model viene usato per la **tabella**, a meno che non sia dichiarato diversamente. Nell'esempio, Eloquent, supporrà che il modello **Flight** memorizzerà records nella tabella **flights**. Un modello **AirTrafficController** farà lo stesso nella tabella **air_traffic_controllers**.

Se il nome della tabella corrispondente al model non rispetta le convenzioni di Eloquent, si può specificare il nome della tabella cui si riferisce attraverso la proprietà **table** del model.

```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Flight extends Model
8  {
9      // ...
10 }
```



```
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Flight extends Model
8  {
9      /**
10       * The table associated with the model.
11       *
12       * @var string
13       */
14     protected $table = 'my_flights';
15 }
```

CONVENZIONI MODELLI ELOQUENT

- **PRIMARY KEYS**: Eloquent, supporrà inoltre la tabella corrispondente al model abbia una **colonna id** come chiave primaria.

Se necessario, si può definire una **proprietà protected \$primaryKey** sul modello per specificare una colonna differente che serve come chiave primaria.

Eloquent si aspetta anche che la **chiave primaria** sia un **auto-incrementing integer**. Per impostare la chiave primaria a valori non-incrementing o non-numeric, si imposta a **false** la proprietà pubblica **\$incrementing**.

Se la primary key non è un integer, si definisce il tipo di dato nella proprietà **\$keyType**.

```
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Model;
6
7 class Flight extends Model
8 {
9     /**
10      * The primary key associated with the table.
11      *
12      * @var string
13      */
14     protected $primaryKey = 'flight_id';
15 }
```

```
3 class Flight extends Model
4 {
5     /**
6      * The data type of the primary key ID.
7      *
8      * @var string
9      */
10     protected $keyType = 'string';
11 }
```



```
3 class Flight extends Model
4 {
5     /**
6      * Indicates if the model's ID is auto-incrementing.
7      *
8      * @var bool
9      */
10     public $incrementing = false;
11 }
```

CONVENZIONI MODELLI ELOQUENT

- **TIMESTAMPS**: Eloquent si aspetta che esistano le colonne **created_at** e **updated_at** nella tabella collegata al model. Eloquent imposterà queste colonne automaticamente in fase di creazione o aggiornamento dei models.

Per evitare che Eloquent le gestisca automaticamente, si imposta **false** la proprietà **\$timestamps** del model

Per personalizzare:

- il formato dei timestamps, impostare la proprietà **\$dateFormat**.
- Il nome delle colonne usate per registrare i timestamps, definire le costanti **CREATED_AT** e **UPDATED_AT**

```
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Model;
6
7 class Flight extends Model
8 {
9     /**
10      * Indicates if the model should be timestamped.
11      *
12      * @var bool
13      */
14     public $timestamps = false;
15 }
```

```
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Model;
6
7 class Flight extends Model
8 {
9     /**
10      * The storage format of the model's date columns.
11      *
12      * @var string
13      */
14     protected $dateFormat = 'U';
15 }
```

```
3 class Flight extends Model
4 {
5     const CREATED_AT = 'creation_date';
6     const UPDATED_AT = 'updated_date';
7 }
```

DEFINIRE PROPRIETA' E RELAZIONI

Esempio di model rappresentante una tabella «users»

```
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    // La tabella associata al model
    protected $table = 'users';

    // La primary key per il model.
    protected $primaryKey = 'id';

    // Gli attributi assegnabili in serie.
    protected $fillable = ['name', 'email', 'password'];

    // definire relazioni
    public function posts()
    {
        return $this->hasMany(Post::class);
    }
}
```

Nell'esempio specifichiamo il **nome della tabella**, la **primary key** e gli **attributi** assegnabili per il model User.

Inoltre definiamo una **relazione one-to-many** tra il model User e il model Post usando il metodo **hasMany**.

Il nostro modello User **estende la classe** astratta **Model di Eloquent** e da essa **eredita metodi** che permettono di interagire con i record del database senza dover scrivere righe di codice.

METODI DELLA CLASSE

Eloquent fornisce dunque metodi e funzionalità che semplificano le comuni operazioni sul database. I vari metodi statici del model restituiscono **istanze della classe** quando vengono recuperate **singole righe** (ad esempio con il metodo **find**) oppure istanze della classe **Illuminate\Database\Eloquent\Collection** nel caso di metodi che recuperano **array di righe**.

RECUPERARE RECORDS

Per recuperare records dal database, si possono usare i **metodi** del query builder di Eloquent.

Eloquent consente inoltre di **concatenare metodi diversi** per costruire query complesse in maniera semplice.

// Trovare uno user dall'ID

```
$user = User::find(1) ;
```

// Trovare users con uno specifico attributo. Get() Ritorna un array sul quale è possibile ciclare

```
$users = User::where('status','active')->get();
```

// Trovare user con uno specifico attributo. first()ritorna un solo record, il primo che soddisfa la condizione

```
$JohnDoe = User::where('name', '=', 'John Doe')->first();
```

// Recuperare tutti gli users

```
$allUsers = User::all();
```

// Recuperare users con i loro relativi posts

```
$userWithPosts = User::with('posts')->find(1)
```


Il metodo **all** recupera tutti i records dalla tabella associata al modello.

Siccome ogni modello Eloquent funziona come query builder, si possono **aggiungere vincoli aggiuntivi** alle query e poi invocare il metodo **get** per recuperare i risultati.

REFRESH MODELS

Se un'istanza del model Eloquent è già stata recuperata dal database, si può fare il "refresh" del model usando i metodi **fresh** o **refresh**.

Il metodo **fresh** recupererà nuovamente il model dal database, creando una nuova istanza. L'istanza già esistente non sarà influenzata.

Il metodo **refresh** ricaricherà il model esistente usando i dati aggiornati dal database. Anche tutte le relazioni saranno aggiornate.

```
1 use App\Models\Flight;
2
3 foreach (Flight::all() as $flight) {
4     echo $flight->name;
5 }
```

```
1 $flights = Flight::where('active', 1)
2     ->orderBy('name')
3     ->take(10)
4     ->get();
```

```
$user = User::find(1);           // name: "Mario"
$user->name = "Giuseppe";        // Modifica solo in memoria

$updatedUser = $user->fresh();    // Crea una nuova istanza con i dati del DB

echo $user->name;                 // → "Giuseppe" (modifica locale ancora presente)
echo $updatedUser->name;         // → "Mario" (dato reale dal DB)
```

```
$user = User::find(1);           // name: "Mario"
$user->name = "Giuseppe";        // Modifica solo in memoria

// Nel frattempo, qualcun altro aggiorna il DB direttamente, es: name = "Luigi"

$user->refresh();                // Ricarica i dati veri dal DB

echo $user->name;                // → "Luigi"
```

COLLECTIONS

Il metodi come **all** e **get** recuperano records multipli dal database. Non ritornano semplici array PHP ma istanze di `Illuminate\Database\Eloquent\Collection`.

La classe `Collection` di Eloquent fornisce diversi metodi per interagire con le collections.

Ad esempio, il metodo **reject** può essere usato per rimuovere modelli da una collection in base ai risultati di una closure.

```
1 $flights = Flight::where('destination', 'Paris')->get();
2
3 $flights = $flights->reject(function (Flight $flight) {
4     return $flight->cancelled;
5 });
```

SUDDIVIDERE I RISULTATI

Invece di caricare migliaia di risultati con i metodi **all** o **get**, può essere utile usare il metodo **chunk** per processare grandi numeri di dati in maniera più efficiente.

Un subset di dati viene recuperato e passato ad una closure per essere processato.

```
1 use App\Models\Flight;
2 use Illuminate\Database\Eloquent\Collection;
3
4 Flight::chunk(200, function (Collection $flights) {
5     foreach ($flights as $flight) {
6         // ...
7     }
8 });
```

SOTTOQUERY AVANZATE

- **Sottoquery select**

Eloquent offre un supporto avanzato per le sottoquery. Immaginiamo di avere una tabella con le **destination** e una tabella di **flights**. La tabella **flights** contiene un campo **arrived_at**. Usando la funzionalità di subquery per i metodi **select** e **addSelect**, possiamo selezionare tutte le destinazioni ed il nome del viaggio che è arrivato più recentemente in quella destinazione usando una sola query.

- **Sottoquery ordering**

Possiamo ordinare i risultati in base all'ultimo viaggio arrivato a destinazione.

```
1 use App\Models\Destination;
2 use App\Models\Flight;
3
4 return Destination::addSelect(['last_flight' => Flight::select('name')
5     ->whereColumn('destination_id', 'destinations.id')
6     ->orderByDesc('arrived_at')
7     ->limit(1)
8 ])->get();
```

```
1 return Destination::orderByDesc(
2     Flight::select('arrived_at')
3     ->whereColumn('destination_id', 'destinations.id')
4     ->orderByDesc('arrived_at')
5     ->limit(1)
6 )->get();
```

RECUPERARE SINGOLI MODELLI/AGGREGATI

Si possono recuperare singoli record usando i metodi **find**, **first** o **firstWhere** che **ritornano una singola istanza del modello**, non una collection.

```
1 use App\Models\Flight;
2
3 // Retrieve a model by its primary key...
4 $flight = Flight::find(1);
5
6 // Retrieve the first model matching the query constraints...
7 $flight = Flight::where('active', 1)→first();
8
9 // Alternative to retrieving the first model matching the query constraints...
10 $flight = Flight::firstWhere('active', 1);
```

Per intraprendere delle **azioni se non vengono trovati risultati**, si possono utilizzare i metodi **findOr** e **firstOr** che eseguiranno la **closure** nel caso non venisse trovato alcun risultato.

```
1 $flight = Flight::findOr(1, function () {
2     // ...
3 });
4
5 $flight = Flight::where('legs', '>', 3)→firstOr(function () {
6     // ...
7 });
```

Per **lanciare un exception** se un model non viene trovato, si usano i metodi **findOrFail** e **firstOrFail** che recuperano il primo risultato della query o, in caso questo non venga trovato, lanceranno un **ModelNotFoundException**.

```
1 $flight = Flight::findOrFail(1);
2
3 $flight = Flight::where('legs', '>', 3)→firstOrFail();
```

RECUPERARE O CREARE MODELLI – RECUPERARE AGGREGATI

- Metodo **firstOrCreate**: cerca di individuare un record usando una data colonna o coppie di valori. **Se il valore non viene trovato nel database, viene inserito un nuovo record** con gli attributi risultanti dal merge dell'array passato come primo argomento e l'array opzionalmente passato come secondo argomento
- Metodo **firstOrCreate**: cerca di individuare un record che corrisponda agli attributi dati. Se il model non viene trovato, viene **ritornata una nuova istanza del model**. Il modello ritornato **non è ancora stato memorizzato** in maniera permanente nel database. Per salvarlo va invocato il metodo **save**.
- I metodi di aggregazione come **count**, **sum**, **max** ritornano un **valore scalare** invece dell'istanza di un model Eloquent.

```
1 use App\Models\Flight;
2
3 // Retrieve flight by name or create it if it doesn't exist...
4 $flight = Flight::firstOrCreate([
5     'name' => 'London to Paris'
6 ]);
7
8 // Retrieve flight by name or create it with the name, delayed, and arrival_time attributes...
9 $flight = Flight::firstOrCreate(
10     ['name' => 'London to Paris'],
11     ['delayed' => 1, 'arrival_time' => '11:30']
12 );
13
14 // Retrieve flight by name or instantiate a new Flight instance...
15 $flight = Flight::firstOrCreate([
16     'name' => 'London to Paris'
17 ]);
18
19 // Retrieve flight by name or instantiate with the name, delayed, and arrival_time attributes...
20 $flight = Flight::firstOrCreate(
21     ['name' => 'Tokyo to Sydney'],
22     ['delayed' => 1, 'arrival_time' => '11:30']
23 );
```

```
1 $count = Flight::where('active', 1)→count();
2
3 $max = Flight::where('active', 1)→max('price');
```

INSERIRE MODELLI

- Per inserire un nuovo record nel database, bisogna creare una **nuova istanza del model** e impostarne gli attributi. Bisogna poi chiamare il metodo **save** sull'istanza.
- Nell'esempio, quando si chiama il metodo **save**, un record viene inserito nel database ed i timestamps **created_at** e **updated_at** sono automaticamente valorizzati.
- Alternativamente, si potrebbe usare il metodo **create** per salvare un nuovo model usando una singola dichiarazione PHP. Il metodo **create** ritornerà un'istanza del model inserito.
- Tuttavia, prima di usare il metodo **create**, bisogna specificare una proprietà **fillable** o **guarded** sulla classe del model. Queste proprietà sono obbligatorie perché Laravel blocca l'inserimento di massa se non si è specificato quali campi sono consentiti (**\$fillable**) o vietati (**\$guarded**).

```
1 use App\Models\Flight;
2
3 $flight = Flight::create([
4     'name' => 'London to Paris',
5 ]);
```

- Se si ha già un'istanza del model, si usare il metodo **fill** per popolarla con un array di attributi

```
1 $flight->fill(['name' => 'Amsterdam to Frankfurt']);
```

```
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use App\Models\Flight;
6 use Illuminate\Http\RedirectResponse;
7 use Illuminate\Http\Request;
8
9 class FlightController extends Controller
10 {
11     /**
12      * Store a new flight in the database.
13      */
14     public function store(Request $request): RedirectResponse
15     {
16         // Validate the request...
17
18         $flight = new Flight;
19
20         $flight->name = $request->name;
21
22         $flight->save();
23
24         return redirect('/flights');
25     }
26 }
```

ESEMPIO MODEL E CREATE

App\Models\Product.php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    // Solo questi campi possono essere assegnati in massa
    protected $fillable = ['name', 'price', 'description'];
}
```

Se non vengono indicate le proprietà \$fillable, Laravel lancerà un'eccezione di tipo `Illuminate\Database\Eloquent\MassAssignmentException`

Uso del metodo create in un controller o seeder. Un nuovo prodotto verrà salvato nel database e \$product conterrà l'istanza appena creata.

```
$product = Product::create([
    'name' => 'Tisana Detox',
    'price' => 12.50,
    'description' => 'Miscela di erbe per depurazione naturale.'
]);
```

Alternativa a \$fillable con \$guarded

```
protected $guarded = []; // Nessun campo è protetto, tutto è assegnabile
```

Nessun campo è protetto, tutto è assegnabile.

Meno sicuro del \$fillable perché lascia passare tutto.

UPDATES

Il metodo **save** può essere usato per aggiornare modelli già esistenti nel database.

Per farlo bisogna recuperare il record che si vuole modificare, impostare ogni attributo che si desidera modificare e chiamare il metodo **save**.

Per creare un nuovo record nel caso non venga trovato quello da aggiornare, si usa il metodo **updateOrCreate**.

Questo metodo salva automaticamente il record nel database quindi non serve invocare il metodo **save**.

Aggiornamento di massa

Gli aggiornamenti possono essere eseguiti su modelli che corrispondono ad una determinata query. Ad esempio, tutti i voli che sono attivi e hanno come destinazione San Diego saranno contrassegnati come in ritardo.

Il metodo **update** si aspetta un array di coppie colonna/valore che rappresentano le colonne da aggiornare. Il metodo **update** restituisce il numero di righe modificate.

```
1 use App\Models\Flight;
2
3 $flight = Flight::find(1);
4
5 $flight->name = 'Paris to London';
6
7 $flight->save();
```

```
1 $flight = Flight::updateOrCreate(
2     ['departure' => 'Oakland', 'destination' => 'San Diego'],
3     ['price' => 99, 'discounted' => 1]
4 );
```

```
Model::updateOrCreate(
    /* criteri di ricerca */,
    /* attributi da aggiornare o inserire */
);
```

```
1 Flight::where('active', 1)
2     ->where('destination', 'San Diego')
3     ->update(['delayed' => 1]);
```


UPSERTS

Combina due azioni: inserimento ed aggiornamento.

Se il record esiste già nel database, Laravel lo aggiorna, altrimenti lo inserisce come nuovo record.

Utile per garantire che un record con determinati dati esista, senza dover prima verificare se il record è presente o meno.

PARAMETRI:

- **\$values**: array di dati che rappresentano i **valori da inserire o aggiornare**. Ogni elemento deve essere un array associativo
- **\$uniqueBy**: colonne che determinano se un record esiste già (basato su una condizione di unicità, ad esempio id o altre colonne uniche)
- **\$update**: colonne da aggiornare se il record esiste già

ESEMPIO: se un volo con un determinato flight_number esiste già, aggiorniamo il prezzo, altrimenti lo inseriamo come nuovo.

Se i voli con i flight_number indicati esistono già, vedranno aggiornati i loro prezzi e status. Se non esistono, verranno inseriti come nuovi record con prezzi e status forniti.

Metodo efficiente perché esegue un'unica query per inserire o aggiornare i dati. Molto utile anche per bulk operations evitando cicli di controllo ed operazioni individuali.

```
Model::upsert(  
    $values,      // I dati da inserire o aggiornare  
    $uniqueBy,    // Le colonne su cui fare la verifica (uniche)  
    $update       // Le colonne da aggiornare se il record esiste  
);
```

```
$flights = [  
    ['flight_number' => 'AA123', 'price' => 150, 'status' => 'on time'],  
    ['flight_number' => 'AA456', 'price' => 200, 'status' => 'delayed'],  
];  
  
Flight::upsert(  
    $flights,      // I dati da inserire o aggiornare  
    ['flight_number'], // La colonna unica (che identifica univocamente un volo)  
    ['price', 'status'] // Le colonne da aggiornare se il record esiste  
);
```

CANCELLARE MODELLI (delete, destroy)

- Metodo **delete**: viene recuperato il model dal database prima di cancellarlo.
- Metodo **destroy**: permette di cancellare il record conoscendo la sua chiave primaria e senza recuperarlo esplicitamente. Oltre alla singola chiave primaria, può accettare un array o una collections di chiavi. Il metodo **destroy** carica ogni record individualmente e chiama il metodo delete in modo che gli eventi **deleting** e **deleted** vengano propriamente lanciati per ogni record.
- Se si utilizzano modelli soft deleting, si possono eliminare permanentemente i record con il metodo **forceDestroy**
- Per eliminare tutti i record che corrispondono a criteri di una query Eloquent, si usa sempre il metodo **delete**.
- Per eliminare tutti i record in una tabella, si esegue una query senza condizioni.

delete

```
1 use App\Models\Flight;
2
3 $flight = Flight::find(1);
4
5 $flight->delete();
```

destroy

```
1 Flight::destroy(1);
2
3 Flight::destroy(1, 2, 3);
4
5 Flight::destroy([1, 2, 3]);
6
7 Flight::destroy(collect([1, 2, 3]));
```

forceDestroy

```
1 Flight::forceDestroy(1);
```

```
1 $deleted = Flight::where('active', 0)->delete();
```

Delete + query

```
1 $deleted = Flight::query()->delete();
```

CANCELLARE MODELLI (soft Deleting)

- Eloquent consente di **non cancellare effettivamente un record dal database**, ma segnarlo come eliminato popolando la colonna **deleted_at** che riporta la data e l'ora in cui il record è stato "eliminato". Questo permette di recuperarlo in futuro, se necessario.
- Per abilitare il soft delete su un model, bisogna aggiungere il trait **Illuminate\Database\Eloquent\SoftDeletes** al model
- Si deve inoltre aggiungere la colonna **deleted_at** alla tabella. Esiste un helper per creare questa colonna: **softDeletes** e **dropSoftDeletes**
- Quando si chiama il metodo **delete** sul model, la colonna **deleted_at** verrà impostata con la data e l'orario correnti. Tuttavia il record sarà conservato in tabella ma escluso dai risultati delle query.
- Per determinare se un determinato record è stato soft deleted, si usa il metodo **trashed**

```
1  if ($flight->trashed()) {  
2      // ...  
3  }
```

```
3  namespace App\Models;  
4  
5  use Illuminate\Database\Eloquent\Model;  
6  use Illuminate\Database\Eloquent\SoftDeletes;  
7  
8  class Flight extends Model  
9  {  
10      use SoftDeletes;  
11  }
```

```
1  use Illuminate\Database\Schema\Blueprint;  
2  use Illuminate\Support\Facades\Schema;  
3  
4  Schema::table('flights', function (Blueprint $table) {  
5      $table->softDeletes();  
6  });  
7  
8  Schema::table('flights', function (Blueprint $table) {  
9      $table->dropSoftDeletes();  
10 });
```

```
$flight-
```

```
1 Flight::withTrashed()  
2   →where('airline_id', 1)  
3   →restore();
```

```
$flight->history()-  
>restore();
```

```
$flight->  
>forceDelete();
```

```
1 use App\Models\Flight;  
2  
3 $flights = Flight::withTrashed()  
4   →where('account_id', 1)  
5   →get();
```

```
1 $flights = Flight::onlyTrashed()  
2   →where('airline_id', 1)  
3   →get();
```

restore per ripristinare un record che è stato soft deleted. Il campo **deleted_at** viene impostato a null.

Si può anche eseguire in una query per ripristinare records multipli.

restore per ripristinare relazioni **history()**

forceDelete rimuove in maniera effettiva e permanente un record dal database

withTrashed per includere i record soft deleted nei risultati della query.

onlyTrashed per recuperare solo i record soft deleted

LOCAL SCOPES

- I local scopes di Laravel sono un modo per definire insiemi di vincoli di query che possono essere riutilizzati in tutta l'applicazione. Questo permette di astrarre query comuni in metodi che poi possono essere richiamati su modelli Eloquent, evitando la ripetizione dello stesso codice più volte.
- Per definire un local scope si aggiunge l'attribute **Scope** ad un metodo Eloquent. I metodi così creati possono essere utilizzati come se fossero metodi di query direttamente sull'istanza del modello.
- Si utilizzano nelle query invocando il metodo. Si possono anche concatenare più scopes.

```
use App\Models\User;

$users = User::popular()→active()→orderBy('created_at')→get();
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Attributes\Scope;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Scope a query to only include popular users.
     */
    #[Scope]
    protected function popular(Builder $query): void
    {
        $query→where('votes', '>', 100);
    }

    /**
     * Scope a query to only include active users.
     */
    #[Scope]
    protected function active(Builder $query): void
    {
        $query→where('active', 1);
    }
}
```

DYNAMIC LOCAL SCOPES

- Sono scopes che accettano dei parametri. Si aggiungono i parametri alla signature del metodo dello scope, dopo il parametro `$query`.
- Una volta che il parametro è stato aggiunto, si passa l'argomento quando si chiama lo scope.

```
$users = User::ofType('admin')->get();
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Attributes\Scope;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Scope a query to only include users of a given type.
     */
    #[Scope]
    protected function ofType(Builder $query, string $type): void
    {
        $query->where('type', $type);
    }
}
```

MUTATORS E ACCESSORS

Gli accessors e i mutators consentono di trasformare e/o formattare gli attributi di Eloquent quando questi vengono recuperati o impostati sulle istanze del modello. È possibile, ad esempio, criptare un valore mentre viene memorizzato nel database e poi decriptarlo automaticamente quando vi si accede in un modello Eloquent.

DEFINIRE UN ACCESSOR

Un accessor **trasforma il valore di un attributo Eloquent** nel momento in cui questo viene recuperato.

Per definire un accessor, creare un metodo protetto nel model per rappresentare l'attributo accessibile.

Il nome del metodo deve corrispondere alla rappresentazione "**camel case**" del **vero attributo** sottostante o della **colonna** del database.

Per esempio, definiamo un accessor per l'attributo **first_name**. L'accessor sarà richiamato automaticamente da Eloquent quando si tenterà di recuperare il valore dell'attributo **first_name**.

Tutti i metodi accessor/mutator devono dichiarare un return type-hinted di **Illuminate\Database\Eloquent\Casts\Attribute**.

Tutti i metodi accessor ritornano un'istanza di **Attribute** che definisce come l'attributo è acceduto e, opzionalmente, mutato. Per farlo si fornisce l'argomento **get** al costruttore della classe **Attribute**.

```
namespace App\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Get the user's first name.
     */
    protected function firstName(): Attribute
    {
        return Attribute::make(
            get: fn (string $value) => ucfirst($value),
        );
    }
}
```

Il valore originario della colonna, è passato all'accessor permettendo di manipolare e ritornare il valore. Per accedere al valore dell'accessor, basta accedere all'attributo `first_name` su un'istanza del model

```
use App\Models\User;

$user = User::find(1);

$firstName = $user-
>first_name;
```

A volte l'accessor potrebbe dover trasformare più attributi del modello in un singolo "valore". Per farlo, la closure `get` può accettare un secondo argomento `$attributes`, che verrà automaticamente fornito alla closure e che conterrà un array di tutti gli attributi correnti del modello.

```
use App\Support\Address;
use Illuminate\Database\Eloquent\Casts\Attribute;

/**
 * Interact with the user's address.
 */
protected function address(): Attribute
{
    return Attribute::make(
        get: fn (mixed $value, array $attributes) => new Address(
            $attributes['address_line_one'],
            $attributes['address_line_two'],
        ),
    );
}
```


DEFINIRE UN MUTATOR

I mutators permettono di modificare il valore di un attributo quando questo viene impostato.

Per definire un mutator, bisogna fornire l'argomento **set** quando si definisce l'attributo. Questo mutator sarà chiamato automaticamente ogni volta che si tenta di impostare il valore dell'attributo.

La closure del mutator riceve il valore assegnato all'attributo permettendo di manipolarlo e di restituire il valore modificato.

Per usare il mutator, basta solo impostare l'attributo **first_name** su un model.

Nell'esempio, la callback set sarà chiamata con il valore "Sally". Il mutator applicherà la funzione **strtolower** al nome ed imposterà il valore risultante nell'array degli attributi interni del model.

```
use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Interact with the user's first name.
     */
    public function setFirstNameAttribute($value) {
        $this->attributes['firstName'] = strtolower($value);
    }
    public function getFirstNameAttribute($value) {
        return ucfirst($value);
    }
}
```

```
use App\Models\User;

$user = User::find(1);

$user->first_name = 'Sally';
```

RELAZIONI

Le relazioni possono essere definite come connessioni tra tabelle e permettono di organizzare e strutturare i dati, migliorandone e semplificandone la gestione.

Nei database esistono principalmente 3 tipi di relazioni:

- **Uno a uno**: un record di una tabella è associato **ad uno, e uno solo**, di un'altra tabella. Ad esempio una persona ed il suo codice fiscale.
- **Uno a molti**: un record è associato **a più record** di un'altra tabella. Ad esempio uno scrittore ed i suoi libri.
- **Molti a molti**: **più record** di una tabella sono associati a più record di un'altra tabella. Ad esempio studenti e corsi ai quali sono iscritti.

Per definire relazioni con Eloquent e fare in modo che questo le riconosca automaticamente, è sufficiente seguire alcune **convenzioni nella creazione della foreign key** che associa le tabelle.

Supponiamo di avere due tabelle, books e authors, e di voler creare una relazione tra le due.

La colonna con la foreign key avrà come nome il **nome del modello al singolare** e minuscolo, **seguito dal suffisso `_id`**.

```
$table->unsignedBigInteger('category_id');  
$table->foreign('category_id')->references('id')->on('categories');  
  
// in Schema::create('books' ..  
  
$table->foreignId('author_id');  
$table->foreign('author_id')->references('id')->on('authors');
```

RELAZIONI

Una volta creata la colonna con la foreign key, per terminare la relazione tra i due model, serve indicare nel model `App\Model\Book` la **relazione** che lo lega al model `App\Model\Author`. Nel caso dell'esempio, un libro appartiene ad un singolo autore.

Sarà così possibile scrivere una rotta nella quale sarà possibile, a partire dal libro recuperato dal database, estrarre le informazioni sul suo autore, grazie alla proprietà dinamica aggiunta da Eloquent.

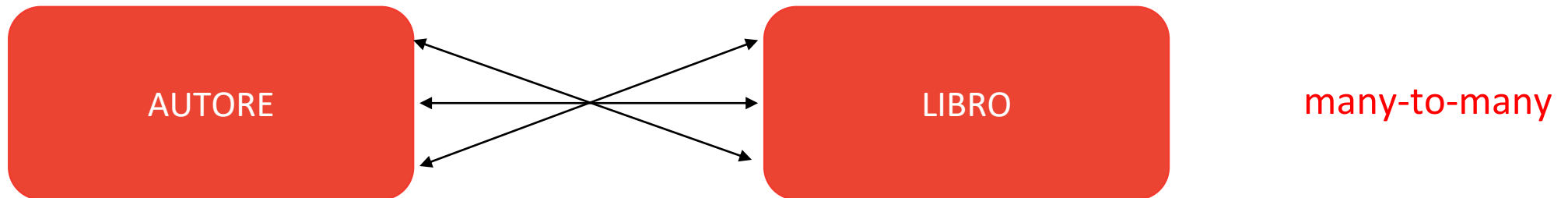
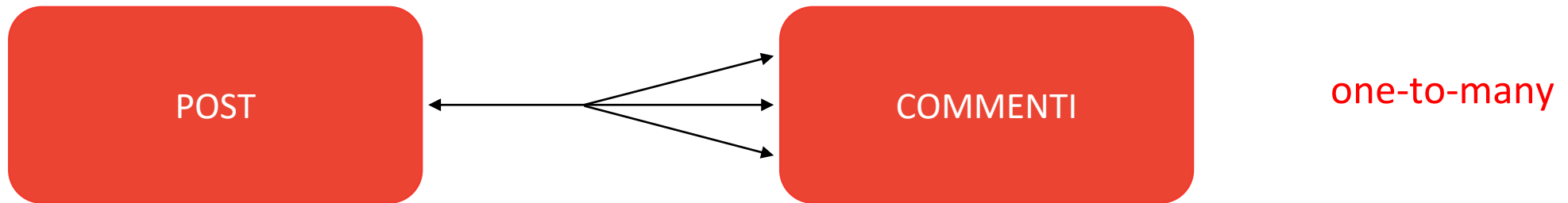
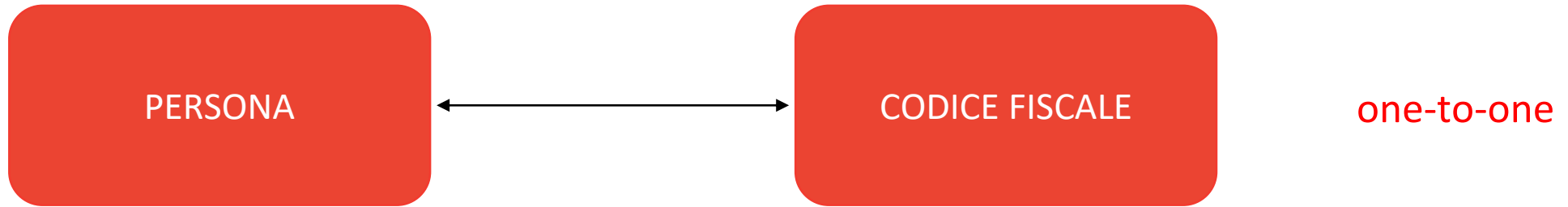
Laravel ed Eloquent si occuperanno di costruire ed eseguire le query necessarie per recuperare l'autore del libro che stiamo chiedendo di visualizzare e restituiranno un'istanza del model collegato.

```
class Book extends Model
{
    public function author() :BelongsTo
    {
        return $this->belongsTo(Author::class);
    }
}
class Author extends Model
{
    public function books(): HasMany
    {
        return $this->hasMany(Book::class);
    }
}
```

```
Route::get('/books/{book}', function (Book $book) {
    // property dinamica author aggiunta da Eloquent
    $author = $book->author;
    // posso accedere a tutte le property di Author
    return "$book->title - $author->first_name $author->last_name";
});
```

Quindi, alla base di ogni relazione utilizzabile con Eloquent vi sono due condizioni:

1. **Rispetto delle convenzioni** su nomi di tabelle e colonne, in particolare per la **foreign key**
2. Aver **dichiarato i metodi di relazione** nelle classi del model



RELAZIONE UNO A UNO – One to One HasOne

Associa due tabelle in modo che una riga della prima tabella sia correlata ad UNA SOLA RIGA dell'altra tabella.

Ad esempio, un modello **User** può essere associato ad un modello **Phone**. Per definire questa relazione, assegniamo un metodo **phone** al modello **User**. Il metodo **phone** chiamerà il metodo **hasOne** e ritornerà il suo risultato.

Il primo argomento del metodo **hasOne** è il nome del modello collegato.

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasOne;
class User extends Model
{
    /** * Recupera il record phone associato allo user. */
    public function phone() :HasOne {
        return $this->hasOne(Phone::class);
    }
}
```

Una volta che la relazione è definita, possiamo recuperare il record collegato usando le proprietà dinamiche di Eloquent che permettono di accedere ai metodi della relazione come se fossero proprietà del modello stesso.

```
$phone = User::find(1)->phone;
```

RELAZIONE UNO A UNO – One to One

HasOne

Eloquent determina la **foreign key** della relazione basandosi sul nome del modello padre. Nel caso dell'esempio, si presuppone che il modello **Phone** abbia una foreign **key user_id**. Se si vuole sovrascrivere questa convenzione, si può passare un secondo argomento al metodo **hasOne**:

```
return $this->hasOne(Phone::class, 'foreign_key');
```

Inoltre, Eloquent suppone che la foreign key abbia un valore corrispondente nella colonna **id** (o la primary key personalizzata) del parent. In altre parole, Eloquent cercherà il valore della colonna **id** dello user nei record della colonna **user_id** della tabella Phone.

Se si desidera che la relazione usi un altro valore invece **dell'id**, si può passare un terzo argomento al metodo **hasOne** specificando la custom key.

```
return $this->hasOne(Phone::class, 'foreign_key', 'local_key');
```

Possiamo adesso accedere al model Phone dal model User

RELAZIONE UNO A UNO – One to One

HasOne

DEFINIRE LA RELAZIONE INVERSA

È possibile accedere all'utente che possiede un determinato telefono definendo il metodo `belongsTo` (inverso della relazione `hasOne`) nel `Phone` model.

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Phone extends Model
{
    /**
     * Get the user that owns the phone.
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Quando si chiama il metodo `user`, Eloquent cercherà una corrispondenza tra il record `user_id` del modello `Phone` e l'`id` del modello `User`.

Eloquent determina il nome di default della foreign key esaminando il nome del metodo della relazione e aggiungendo il suffisso `_id`. Come il metodo `hasOne`, anche `belongsTo` accetta il secondo ed il terzo parametro per customizzare il nome della key o per indicare che il join avviene su colonne diverse delle tabelle.

```
/**
 * Get the user that owns the phone.
 */
public function user(): BelongsTo
{
    return $this->belongsTo(User::class, 'foreign_key', 'owner_key');
}
```

RELAZIONE UNO A MOLTI – One to Many HasMany

Definisce la relazione in cui un singolo model è genitore di uno o più modelli figli: una riga della tabella A può essere legata a più righe della tabella B, ma ogni riga della tabella B è legata a una sola riga della tabella A.

Ad esempio, un modello **Post** può avere un infinito numero di commenti.

La relazione uno a molti è definita inserendo il metodo **hasMany** nel modello Eloquent

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Post extends Model
{
    /**
     * Get the comments for the blog post.
     */
    public function comments(): HasMany
    {
        return $this->hasMany(Comment::class);
    }
}
```

Per convenzione, Eloquent prenderà il nome «snake case» del modello padre ed aggiungerà il suffisso **_id**.
In questo caso Eloquent supporrà che la foreign key sul modello **Comment**, sarà **post_id**.

Una volta che la relazione è stata definita, possiamo accedere alla collection dei commenti accedendo alla proprietà **comments**.

```
use App\Models\Post;

$comments = Post::find(1)->comments;

foreach ($comments as $comment) {
    // ...
}
```


RELAZIONE UNO A MOLTI – One to Many

Poiché tutte le relazioni fungono anche da **generatori di query**, si possono **aggiungere condizioni** per definire quali commenti verranno recuperati chiamando il metodo **comments**.

```
$comment = Post::find(1)→comments()  
    →where('title', 'foo')  
    →first();
```

Come il metodo **hasOne**, si possono sovrascrivere la foreign e la local keys passando argomenti aggiuntivi al metodo **hasMany**:

```
return $this→hasMany(Comment::class, 'foreign_key');  
  
return $this→hasMany(Comment::class, 'foreign_key', 'local_key');
```

Per accedere da un commento al post parent, si definisce il modello **belongsTo** (inversa di **hasMany**) nel model del figlio .

Una volta che la relazione è stata definita, possiamo recuperare il model **Post** per un **Comment** accedendo la proprietà dinamica **post**.

In questo esempio, Eloquent cercherà la corrispondenza tra **post_id** del **Comment** e l'**id** del modello **Post**.

```
use App\Models\Comment;  
  
$comment = Comment::find(1);  
  
return $comment→post→title;
```

```
namespace App\Models;  
  
use Illuminate\Database\Eloquent\Model;  
use Illuminate\Database\Eloquent\Relations\BelongsTo;  
  
class Comment extends Model  
{  
    /**  
     * Get the post that owns the comment.  
     */  
    public function post(): BelongsTo  
    {  
        return $this→belongsTo(Post::class);  
    }  
}
```

RELAZIONE UNO A MOLTI – One to Many

```
use App\Models\Comment;

$comment = Comment::find(1);

return $comment→post→title;
```

Come in precedenza, Eloquent determinerà il nome predefinito della foreign key partendo dal nome del metodo della relazione e aggiungendo il suffisso `_` seguito dal nome della colonna impostata come chiave primaria nel modello padre (id se non diversamente indicato).

Anche in questo caso, si può passare il nome della custom foreign key come secondo argomento al metodo `belongsTo`. Allo stesso modo, se si desidera che il join tra le tabelle avvenga su un'altra colonna, si può passare il nome della colonna come terzo argomento.

```
/**
 * Get the post that owns the comment.
 */
public function post(): BelongsTo
{
    return $this→belongsTo(Post::class, 'foreign_key', 'owner_key');
}
```

RELAZIONE MOLTI A MOLTI – Many to Many

Nel modello autore-libro considerato all'inizio dell'argomento, abbiamo dato per scontato che un libro potesse essere scritto da un solo autore. Nella realtà, però, è possibile che uno stesso libro sia stato scritto da più autori.

In questo caso si avrà una relazione many-to-many poichè **una riga della tabella A può essere legata a più righe della tabella B** e, allo stesso tempo, **una riga della tabella B può essere collegata a più righe della tabella A**.

Le relazioni many-to-many sono solitamente implementate in un RDBMS utilizzando una terza tabella chiamata «**tabella associativa**» (o **join table** oppure **cross-reference-table**). Questa tabella consente di separare la relazione many-to-many in due diverse relazioni one-to-many.



Anche in Eloquent, l'implementazione di questa relazione richiede la presenza di tre tabelle: books, authors e tabella associativa author_book.

Per convenzione Eloquent richiede che il nome della tabella associativa sia formato dal **nome dei rispettivi modelli al singolare uniti da un _ ed in ordine alfabetico**. Nella tabella associativa devono inoltre essere presenti **le colonne per le due foreign key** che puntano a ciascuna tabella (author_id e book_id)

RELAZIONE MOLTI A MOLTI – Many to Many

A differenza di quanto accade per la relazione one-to-many, in questo caso non saranno presenti le colonne `foreign_key` nelle due tabelle `books` e `authors` poiché non vi è una relazione diretta tra le due.

Si avrà dunque:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class Author extends Model
{
    /**
     * The roles that belong to the user.
     */
    public function books(): BelongsToMany
    {
        return $this->belongsToMany(Books::class);
    }
}
```

authors:	books:	author_book:
- id = int	- id = int	- id = int
- name = string	- title = string	- author_id = foreign_key(author.id)
		- book_id = foreign_key(book.id)

Una volta create le tabelle e le colonne con i giusti nomi tramite le migrations, si dovranno indicare le relazioni Eloquent nei relativi model. In questo caso avremo due relazioni `belongsToMany` per entrambi i model.

Il primo argomento passato al metodo è il nome del model collegato.

RELAZIONE MOLTI A MOLTI – Many to Many

Sarà poi possibile utilizzare le proprietà dinamiche di Laravel per accedere ai vari model e aggiungere condizioni alle query.

AUTHOR MODEL

```
use App\Models\Book;

$book = Book::find(1);

foreach ($book->authors as
$author) {
    // ...
}
```

BOOK MODEL

```
use App\Models\Author;

$author = Author::find(124);

foreach ($author->books as
$book) {
    // ...
}
```

CONDITIONS

```
$books = Author::find(141)->books()-
->orderBy('title')->get();
```

Abbiamo visto che esistono delle convenzioni per l'assegnazione del nome alla tabella intermedia e alle colonne delle chiavi. E' possibile però personalizzarli passando i nomi scelti come argomenti del metodo belongsToMany:

```
return $this->belongsToMany(Book::class, 'author_book',
'author_id', 'book_id');
```

SECONDO ARGOMENTO: nome personalizzato della tabella intermedia

TERZO ARGOMENTO: nome della foreign key del modello sul quale si sta definendo la relazione

QUARTO ARGOMENTO: nome della foreign key del modello collegato

COLLECTIONS

Tutti gli insiemi di risultati multipli sono istanze dell'oggetto

`Illuminate\Database\Eloquent\Collection`, compresi i risultati ottenuti con il metodo `get` o attraverso le relazioni.

L'oggetto `collection` di Eloquent estende la `collection` di base di Laravel, quindi eredita decine di metodi utilizzati per lavorare in modo fluido con l'array sottostante. Ovviamente tutte le `collections` servono come iteratori permettendo di eseguire cicli come se fossero normali arrays.

Le `collections` sono però molto più potenti degli array e dispongono di una serie di operazioni di mappatura e riduzione che possono essere concatenate tra loro.

Potremmo, ad esempio, rimuovere tutti gli users inattivi e raccogliere il nome di ogni utente rimasto.

```
use App\Models\User;

$users = User::where('active',
1)->get();

foreach ($users as $user) {
    echo $user->name;
}
```

Rotte protette con Sanctum

- Crea un utente da Tinker
use App\Models\User;
- User::create(['name' => 'Mario Rossi', 'email' => 'mario@example.com', 'password' => Hash::make('password')]);
- User::create(['name' => 'Admin', 'email' => 'admin@admin.it', 'password' => Hash::make('password')]);

Rotte protette con Sanctum

- `php artisan install:api`
- Questo comando:
- Configura il progetto per lavorare come API stateless
- Abilita Laravel Sanctum per gestire i token
- Genera controller e route API già pronti in `routes/api.php`
- Crea i controller nella cartella `App/http/Controllers`

Crea la route per il token

```
Route::post('/token', function (Request $request) {  
    $request->validate([  
        'email' => 'required|email',  
        'password' => 'required',  
    ]);  
  
    if (!Auth::attempt($request->only('email', 'password'))) {  
        return response()->json(['message' => 'Credenziali non valide'], 401);  
    }  
  
    $user = Auth::user();  
    $token = $user->createToken('api-token')->plainTextToken;  
  
    return response()->json(['token' => $token]);  
});
```

Estendi user

```
class User extends Authenticatable
{
    /** @use HasFactory<\Database\Factories\UserFactory> */
    use HasFactory, Notifiable, HasApiTokens;
```

Da postman richiesta del token

HTTP corso_api3_auth_sanctum / token

POST ▼ `{{base_url}}/token`

Params Authorization Headers (11) Body ● Scripts Settings

Headers 9 hidden

	Key	Value
☒	Accept	application/json
☒	Content-Type	application/json
	Key	Value

Body Cookies Headers (8) Test Results ↺

`{}` JSON ▼ ▶ Preview 🔍 Visualize ▼

```
1 {
2   |   "token": "1|R7iNhzjsKbuhu7cbaSr2wPK6k7UWUUusYzV1Myybb7a5bca5"
3 }
```

Da postman url protetta

HTTP corso_api3_auth_sanctum / user

GET ▼ `{{base_url}}/user`

Params Authorization **Headers (10)** Body Scripts Settings

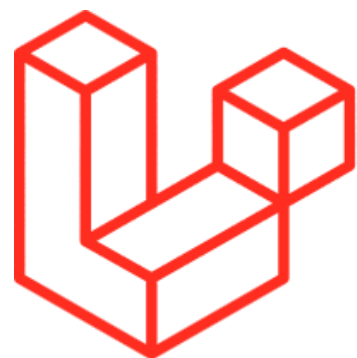
Headers 8 hidden

	Key	Value
☒	Accept	application/json
☒	Content-Type	application/json
	Key	Value

Body Cookies Headers (8) Test Results ↺

`{}` JSON ▼ ▶ Preview 🔗 Visualize ▼

```
1 {
2   "id": 2,
3   "name": "Admin",
4   "email": "admin@admin.it",
5   "email_verified_at": null,
6   "created_at": "2025-09-06T16:09:23.000000Z",
7   "updated_at": "2025-09-06T16:09:23.000000Z"
8 }
```



Laravel

QUERY BUILDER

- Il query builder di Laravel fornisce una comoda interfaccia per creare e lanciare queries. Funziona con tutti i database supportati da Laravel.
- Il query builder utilizza il binding dei parametri PDO. Non serve sanificare le stringhe passate al query builder come query bindings.

LANCIARE QUERIES

RECUPERARE TUTTE LE RIGHE DA UNA TABELLA

- Il metodo **table** della facade **DB** serve per iniziare una query. Ritorna un'istanza per la tabella indicata consentendo di concatenare più vincoli ed infine recuperare il risultato della query usando il metodo **get**.
- Il metodo **get** ritorna una **collection di risultati** dove ogni risultato è un oggetto **stdClass**. Si può accedere al valore di ogni colonna accedendo alla colonna stessa come proprietà dell'oggetto:

```
namespace App\Http\Controllers;

use Illuminate\Support\Facades\DB;
use Illuminate\View\View;

class UserController extends Controller
{
    /**
     * Show a list of all of the application's users.
     */
    public function index(): View
    {
        $users = DB::table('users')->get();

        return view('user.index', ['users' => $users]);
    }
}
```

```
use Illuminate\Support\Facades\DB;

$users = DB::table('users')->get();

foreach ($users as $user) {
    echo $user->name;
}
```

RECUPERARE UNA SINGOLA RIGA/COLONNA

- Si usa il metodo **first** della facade **DB** che ritorna un singolo oggetto **stdClass**. Per lanciare un exception se non viene trovato alcun record, si usa il metodo **firstOrFail()**
- Il metodo **value** serve ad estrarre un singolo valore da un record. Ritorna direttamente il valore della colonna
- Con il metodo **find** si recupera una singola riga **attraverso il valore della sua colonna id**

```
$user = DB::table('users')->where('name', 'John')->first();  
return $user->email;
```

```
$email = DB::table('users')->where('name', 'John')->value('email');
```

```
$user = DB::table('users')->find(3);
```

RECUPERARE UNA LISTA DI VALORI DA UNA COLONNA

- Il metodo **pluck** ritorna una **Collection** contenente i **valori di una singola colonna**.
- Come secondo argomento si può specificare la colonna che la collection dovrà usare come chiave.

```
use Illuminate\Support\Facades\DB;  
$titles = DB::table('users')->pluck('title');  
foreach ($titles as $title) {  
    echo $title;  
}
```

Si sta recuperando una collection di titles

```
$titles = DB::table('users')->pluck('title', 'name');  
foreach ($titles as $name => $title) {  
    echo $title;  
}
```

CHUNKING DEI RISULTATI

- Il metodo **chunk** della facade DB permette di «spezzettare» i risultati in piccole porzioni che, una alla volta, vengono inviate ad una closure per essere processate.

```
use Illuminate\Support\Collection;
use Illuminate\Support\Facades\DB;

DB::table('users')->orderBy('id')->chunk(100, function (Collection $users) {
    foreach ($users as $user) {
        // ...
    }
});
```

Viene recuperata la tabella users in tranches di 100 records alla volta

- Per fermare altre porzioni di risultati dall'essere processate, si ritorna **false** dalla closure

```
DB::table('users')->orderBy('id')->chunk(100, function (Collection $users)
{
    // Process the records...

    return false;
});
```

```
DB::table('users')->where('active', false)
->chunkById(100, function (Collection $users) {
    foreach ($users as $user) {
        DB::table('users')
            ->where('id', $user->id)
            ->update(['active' =>
true]);
    }
});
```

- Se si vuole eseguire **l'update dei records recuperati** mentre questi vengono suddivisi, è meglio usare il metodo **chunkById** che **paginerà i risultati** in base alla chiave primaria dei record

SELECT

SPECIFICARE UNA CLAUSOLA SELECT

Con il metodo **select** si può specificare una select clause per la query

```
use Illuminate\Support\Facades\DB;

$users = DB::table('users')
    ->select('name', 'email as user_email')
    ->get();
```

Il metodo **distinct** permette di forzare la query a riportare solo risultati distinti

```
$users = DB::table('users')->distinct()->get();
```

Per aggiungere una colonna alla select clause di una query già esistente, si usa il metodo **addSelect**

```
$query = DB::table('users')->select('name');
$users = $query->addSelect('age')->get();
```

JOINS

- **INNER JOIN**

Per una inner join basica, si usa il metodo **join** su un'istanza del query builder. **Primo argomento** è il **nome della tabella alla quale ci si vuole unire**, gli **argomenti rimanenti** specificano i **vincoli delle colonne** per la join. Si possono anche unire più tabelle in una singola query.

- **LEFT JOIN / RIGHT JOIN**

Si utilizzano i metodi **leftJoin** o **rightJoin**. Stessa signature della inner join

- **CROSS JOIN**

crossJoin genera un prodotto cartesiano tra la prima tabella e la tabella unita

- **CLAUSOLE JOIN AVANZATE**

Passando una closure come secondo argomento del metodo join si possono specificare clausole di join avanzate.

```
use Illuminate\Support\Facades\DB;

$users = DB::table('users')
    ->join('contacts', 'users.id',
        '=', 'contacts.user_id')
    ->join('orders', 'users.id', '=',
        'orders.user_id')
    ->select('users.*', 'contacts.phone',
```

```
        'orders.amount');

$users = DB::table('users')
    ->leftJoin('posts', 'users.id', '=',
        'posts.user_id')
    ->get();

$users = DB::table('users')
    ->rightJoin('posts', 'users.id', '=',
        'posts.user_id')
```

```
$sizes = DB::table('sizes')
    ->crossJoin('colors')
    ->get();
```

```
DB::table('users')
    ->join('contacts', function (JoinClause $join) {
        $join->on('users.id', '=', 'contacts.user_id')-
    ->orOn(/* ... */);
    })
    ->get();
```

Si possono usare i metodi **where** e **orWhere** forniti dall'istanza della JoinClause per usare una clausola **where** sulle joins. Invece di confrontare due colonne, questi metodi confronteranno una colonna ed un valore

- SUBQUERY DELLE JOINS**

I metodi **joinSub**, **leftJoinSub** e **rightJoinSub** servono per unire una query ad una sottoquery. Ognuno di questi metodi riceve **tre argomenti**: la **sottoquery**, l'**alias** della sua tabella ed una **closure** che definisce le colonne correlate.

UNIONI

Il metodo **union** permette di unire due o più query insieme.

Il metodo **unionAll** non rimuoverà i risultati duplicati formati dall'unione di due queries. Stessa signature di union.

```
DB::table('users')
    ->join('contacts', function (JoinClause $join) {
        $join->on('users.id', '=', 'contacts.user_id')
        ->where('contacts.user_id', '>',
5);
    })
    ->get();
```

```
$latestPosts = DB::table('posts')
    ->select('user_id', DB::raw('MAX(created_at) as
last_post_created_at'))
    ->where('is_published', true)
    ->groupBy('user_id');

$users = DB::table('users')
    ->joinSub($latestPosts, 'latest_posts', function (JoinClause $join)
{
    $join->on('users.id', '=', 'latest_posts.user_id');
})->get();
```

Recuperiamo una collection di utenti dove ogni record contiene anche un timestamp created_at del post più recente

```
use Illuminate\Support\Facades\DB;

$first = DB::table('users')
    ->whereNull('first_name');

$users = DB::table('users')
    ->whereNull('last_name')
    ->union($first)
    ->get();
```

CLAUSOLE WHERE DI BASE

```
$users = DB::table('users')
->where('votes', '=', 100)
->where('age', '>', 35)
->get();
```

La query recupera gli utenti che hanno voto uguale a 100 ed età superiore a 35 anni

La più basica chiamata al metodo del query builder **where** richiede tre argomenti:

- Il nome della colonna
- Un operatore (se omissso, Laravel userà l'operatore =)
- Il valore da confrontare con il valore della colonna

Si può usare qualsiasi operatore supportato dal database

È anche possibile passare al metodo **where** un array di condizioni. Ogni elemento deve essere un array contenente i tre argomenti passati al metodo **where**.

```
$users = DB::table('users')->where([
    ['status', '=', '1'],
    ['subscribed', '<>', '1'],
])->get();
```

```
$users = DB::table('users')
->where('votes', '>=', 100)
->get();
```

```
$users = DB::table('users')
->where('votes', '<>', 100)
->get();
```

```
$users = DB::table('users')
->where('name', 'like',
'T%')
->get();
```

```
$users = DB::table('users')
->where('votes', '>',
100)
->orWhere('name',
'John')
->get();
```

CLAUSOLA ORWHERE

Il metodo **orWhere** permette di unire una clausola alla query usando l'operatore or. Accetta gli stessi argomenti del metodo **where**.

Si può raggruppare una condizione **or** tra parentesi passando una closure come primo argomento del metodo **orWhere**.

Questo esempio produrrà il seguente SQL:

```
select * from users where votes > 100 or (name = 'Abigail' and votes > 50)
```

```
$users = DB::table('users')
->where('votes', '>', 100)
->orWhere(function (Builder $query) {
    $query->where('name',
'Abigail')
->where('votes',
'>', 50);
})
->get();
```

CLAUSOLA WHERE NOT

I metodi **whereNot** e **orWhereNot** negano un dato gruppo di condizioni. Ad esempio, la query a fianco esclude i prodotti che sono in liquidazione e che hanno un prezzo inferiore a 10

```
$products = DB::table('products')
    ->whereNot(function (Builder $query) {
        $query->where('clearance',
            true)
            ->orWhere('price',
                '<', 10);
    })
    ->get();
```

```
$users = DB::table('users')
    ->where('preferences->dining->meal',
        'salad')
    ->get();
```

LE CLAUSOLE WHERE JSON

Laravel supporta le query verso colonne di tipo JSON su database che le supportano. Al momento sono MariaDB 10.3, MySQL 8.0+, PostgreSQL 12.0+, SQL Server 2017+, and SQLite 3.39.0+.

Per rivolgere una query ad una Colonna JSON **si usa l'operatore ->**

Per gli array JSON si usa **whereJsonContains**

```
$users = DB::table('users')
    ->whereJsonContains('options->languages', 'en')
    ->get();
```

Oppure il metodo **whereJsonLength** per query di JSON array a seconda della loro lunghezza.

```
$users = DB::table('users')
    ->whereJsonLength('options->languages', 0)
    ->get();

$users = DB::table('users')
    ->whereJsonLength('options->languages',
        '>', 1)
    ->get();
```

CLAUSOLE WHERE ANY/ ALL/ NONE

- **whereAny:** Per applicare gli stessi vincoli di query a più colonne.
Si vogliono ad esempio recuperare tutti i record dove una o più delle colonne della lista hanno valore LIKE un dato valore

```
$users = DB::table('users')  
    →where('active', true)  
    →whereAny([  
        'name',  
        'email',  
        'phone',  
    ], 'like', 'Example%')  
    →get();
```

SQL

```
SELECT *  
FROM users  
WHERE active = true AND (  
    name LIKE 'Example%' OR  
    email LIKE 'Example%' OR  
    phone LIKE 'Example%'  
)
```

- **whereAll:** recupera tutti i record dove **TUTTE** le colonne della lista hanno valore corrispondente alla condizione

```
$posts = DB::table('posts')  
    →where('published', true)  
    →whereAll([  
        'title',  
        'content',  
    ], 'like', '%Laravel%')  
    →get();
```

SQL

```
SELECT *  
FROM posts  
WHERE published = true AND (  
    title LIKE '%Laravel%' AND  
    content LIKE '%Laravel%'  
)
```

- **whereNone:** recupera tutti i record dove **NESSUNA** delle colonne della lista ha valore corrispondente alla condizione

```
$posts = DB::table('albums')  
    →where('published', true)  
    →whereNone([  
        'title',  
        'lyrics',  
        'tags',  
    ], 'like', '%explicit%')  
    →get();
```

SQL

```
SELECT *  
FROM albums  
WHERE published = true AND NOT (  
    title LIKE '%explicit%' OR  
    lyrics LIKE '%explicit%' OR  
    tags LIKE '%explicit%'  
)
```

CLAUSOLE WHERE AGGIUNTIVE

- **whereBetween / orWhereBetween** verificano che il valore di una colonna sia compreso tra due valori
- **whereNotBetween / orWhereNotBetween**: valore della colonna non compreso tra due valori
- **whereBetweenColumns / whereNotBetweenColumns / orWhereBetweenColumns / orWhereNotBetweenColumns**: il valore della colonna è o non è tra due valori di due colonne nella stessa tabella
- **whereIn / whereNotIn / orWhereIn / orWhereNotIn**: il valore di una colonna è (o non è) contenuto nell'array dato
- **whereNull / whereNotNull / orWhereNull / orWhereNotNull**: il valore della colonna è (o non è) null.
- **whereDate / whereMonth / whereDay / whereYear / whereTime**: confronta il valore di una Colonna con una data, mese, un giorno, un anno o un orario
- **whereColumn / orWhereColumn**: verifica se due colonne sono uguali. Si può specificare un operatore come secondo parametro oppure passare un array di comparatori.

```
$users = DB::table('users')
    ->whereBetween('votes', [1, 100])
    ->get();
```

```
$patients = DB::table('patients')
    ->whereBetweenColumns('weight',
    ['minimum_allowed_weight', 'maximum_allowed_weight'])
    ->get();
```

```
$users = DB::table('users')
    ->whereIn('id', [1, 2, 3])
    ->get();
```

```
$users = DB::table('users')
    ->whereNull('updated_at')
    ->get();
```

```
$users = DB::table('users')
    ->whereDate('created_at', '2016-12-31')
    ->get();
```

```
$users = DB::table('users')
    ->whereColumn('first_name', 'last_name')
    ->get();
```

```
$users = DB::table('users')
    ->whereColumn([
        ['first_name', '=', 'last_name'],
        ['updated_at', '>', 'created_at'],
    ])->get();
```

- **whereLike / orwhereLike / whereNotLike / orWhereNotLike** permettono di aggiungere clausole "LIKE" o "NOT LIKE". Possibilità di scegliere case-sensitivity, di default è case-insensitive.

```
$users = DB::table('users')
    ->whereLike('name', '%John%')
    ->get();
```

```
$users = DB::table('users')
    ->where ('votes', '>', 100)
    ->orWhereNotLike('name', '%John%', caseSensitive: true)
    ->get();
```

- **wherePast/ whereFuture / whereToday / whereBeforeToday /whereAfterToday**: per determinare se il valore della colonna è nel passato o nel futuro, è oggi o prima o dopo oggi. Esistono anche i metodi **whereNowOrPast**, **whereNowOrFuture**, **whereTodayOrBefore**, **whereTodayOrAfter**

```
$invoices = DB::table('invoices')
    ->wherePast('due_at')
    ->get();

$invoices = DB::table('invoices')
    ->whereFuture('due_at')
    ->get();
```

```
$invoices = DB::table('invoices')
    ->whereToday('due_at')
    ->get();

$invoices = DB::table('invoices')
    ->whereBeforeToday('due_at')
    ->get();

$invoices = DB::table('invoices')
    ->whereAfterToday('due_at')
    ->get();
```

RAGGRUPPAMENTO LOGICO

Si consiglia di raggruppare sempre le chiamate al metodo **orWhere** tra parentesi in modo da evitare comportamenti inaspettati. Per farlo si passa una closure al metodo **where**.

```
$users = DB::table('users')
    ->where('name', '=', 'John')
    ->where(function (Builder $query) {
        $query->where('votes', '>', 100)
            ->orWhere('title', '=', 'Admin');
    })
    ->get();
```


CLAUSOLE WHERE AVANZATE

- **whereExists**: accetta una closure che riceve un'istanza del query builder dove definire la query che deve essere inserita nella clausola **exists**
- In alternativa, si può passare al metodo **whereExists** un oggetto query invece di una closure

```
$users = DB::table('users')
    →whereExists(function (Builder $query) {
        $query→select(DB::raw(1))
            →from('orders')
            →whereColumn('orders.user_id', 'users.id');
    })
    →get();
```

Il risultato di entrambi gli esempi riportati è il seguente SQL:

```
select * from users
where exists (
    select 1
    from orders
    where orders.user_id = users.id
)
```

```
$orders = DB::table('orders')
    →select(DB::raw(1))
    →whereColumn('orders.user_id', 'users.id');

$users = DB::table('users')
    →whereExists($orders)
    →get();
```

- **Where clause di una subquery**

Per **confrontare i risultati di una subquery con un dato valore**, si possono passare una closure ed un value al metodo **where**.

```
use App\Models\User;
use Illuminate\Database\Query\Builder;

$users = User::where(function (Builder $query) {
    $query→select('type')
        →from('membership')
        →whereColumn('membership.user_id', 'users.id')
        →orderByDesc('membership.start_date')
        →limit(1);
}, 'Pro')→get();
```

Per confrontare **una colonna con i risultati di una subquery**, si possono passare una colonna, un operatore ed una closure al metodo **where**.

```
use App\Models\Income;
use Illuminate\Database\Query\Builder;

$incomes = Income::where('amount', '<', function (Builder $query) {
    $query→selectRaw('avg(i.amount)')→from('incomes as i');
})→get();
```

ORDINARE, RAGGRUPPARE, LIMIT E OFFSET

ORDINARE

- **Metodo orderBy**: ordina i risultati in base ad una data colonna. Primo argomento è la colonna, secondo è la direzione dell'ordinamento che può essere **asc** o **desc**
- Per ordinare più colonne, si può semplicemente richiamare **orderBy** tante volte quante ne occorrono
- **Metodi latest e oldest**: ordina i risultati in base alla data. Di default il risultato sarà ordinato in base alla colonna **created_at**. Oppure si passa il nome della colonna che si desidera usare per l'ordinamento.
- **Ordine casuale**: Si usa il metodo **inRandomOrder**
- **Metodo reorder**: rimuove tutti gli **orderBy** che sono stati precedentemente applicati alla query.

Passando una colonna ed una direzione al metodo **reorder**, si rimuovono tutte gli **orderBy** precedenti e si applica un nuovo ordinamento alla query.

```
$users = DB::table('users')
    ->orderBy('name', 'desc')
    ->get();
```

```
$users = DB::table('users')
    ->orderBy('name', 'desc')
    ->orderBy('email', 'asc')
    ->get();
```

```
$user = DB::table('users')
    ->latest()
    ->first();
```

```
$randomUser = DB::table('users')
    ->inRandomOrder()
    ->first();
```

```
$query = DB::table('users')->orderBy('name');
$unorderedUsers = $query->reorder()->get();
```

```
$query = DB::table('users')->orderBy('name');
$usersOrderedByEmail = $query->reorder('email', 'desc')->get();
```

RAGGRUPPARE I RISULTATI

- **Metodi `groupBy` e `having`**: il metodo **`having`** ha una signature simile al metodo **`where`**
- **Metodo `havingBetween`** Per filtrare i risultati all'interno di un certo range
- Passando più argomenti al metodo **`groupBy`**, si possono raggruppare i risultati per colonne multiple
- Per costruire istruzioni **`having`** più avanzate, vedere il metodo **`havingRaw`**

```
$users = DB::table('users')
    ->groupBy('account_id')
    ->having('account_id', '>', 100)
    ->get();
```

```
$report = DB::table('orders')
    ->selectRaw('count(id) as number_of_orders, customer_id')
    ->groupBy('customer_id')
    ->havingBetween('number_of_orders', [5, 15])
    ->get();
```

```
$users = DB::table('users')
    ->groupBy('first_name', 'status')
    ->having('account_id', '>', 100)
    ->get();
```

LIMIT E OFFSET

- **Metodi `skip` e `take`**: questi due metodi vengono usati per limitare il numero dei risultati di una query o per saltare un dato numero di risultati
- **Metodi `limit` e `offset`**: si possono usare in alternativa ai metodi **`skip`** e **`take`**.

```
$users = DB::table('users')->skip(10)->take(5)->get();
```

```
$users = DB::table('users')
    ->offset(10)
    ->limit(5)
    ->get();
```

CLAUSOLE CONDIZIONALI – metodo when

- Per applicare certe clausole alla query in base ad altre condizioni. Ad esempio, si potrebbe voler applicare una **where** se il valore di un dato input è presente nella richiesta HTTP in entrata.

```
$role = $request->input('role');

$users = DB::table('users')
    ->when($role, function (Builder $query, string $role) {
        $query->where('role_id', $role);
    })
    ->get();
```

- La closure eventualmente passata come **terzo argomento**, verrà eseguita se il primo argomento risulta false.

- Il metodo **when** esegue la closure solo quando il primo argomento è **true**. Se il primo argomento è **false**, la closure non viene eseguita.

```
$sortByVotes = $request->boolean('sort_by_votes');

$users = DB::table('users')
    ->when($sortByVotes, function (Builder $query, bool $sortByVotes) {
        $query->orderBy('votes');
    }, function (Builder $query) {
        $query->orderBy('name');
    })
    ->get();
```

ISTRUZIONE INSERT

- Il metodo **insert** è usato per inserire record nelle tabelle del database. Accetta un array di colonne e valori.

Si possono inserire **più records alla volta** passando un array di array. **Ogni array rappresenta un record che deve essere inserito in tabella.**

- Il metodo **insertOrIgnore** ignorerà gli errori che si potrebbero verificare inserendo il record nel database. Saranno ignorati anche gli errori per record duplicato. Gli errori ignorati dipendono dal tipo di database in uso.
- Il metodo **insertUsing** inserirà **nuovi records nella tabella** mentre una subquery determina i dati che dovrebbero essere inseriti

```
DB::table('users')->insert([
    'email' => 'kayla@example.com',
    'votes' => 0
]);
```

```
DB::table('users')->insert([
    ['email' => 'picard@example.com', 'votes' => 0],
    ['email' => 'janeway@example.com', 'votes' => 0],
]);
```

```
DB::table('users')->insertOrIgnore([
    ['id' => 1, 'email' => 'sisko@example.com'],
    ['id' => 2, 'email' => 'archer@example.com'],
]);
```

```
DB::table('pruned_users')->insertUsing([
    'id', 'name', 'email', 'email_verified_at'
], DB::table('users')->select(
    'id', 'name', 'email', 'email_verified_at'
)->where('updated_at', '<=', now()->subMonth()));
```

```
$id = DB::table('users')->insertGetId(
    ['email' => 'john@example.com', 'votes' => 0]
);
```

ID AUTO-INCREMENTALI

Se la tabella ha id auto-incrementali, il metodo **insertGetId** permette di inserire un record e recuperare l'ID

UPDATE

- Il metodo **update** è usato per aggiornare records già presenti nelle tabelle del database. Accetta un array di coppie colonne / valori che indicano le colonne da aggiornare. Il metodo update **ritorna il numero di righe modificate**.
- Si possono **aggiungere vincoli** con delle **where clauses**.
- Il metodo **updateOrCreate** consente di aggiornare un record già esistente o crearlo se non viene trovato. Accetta due argomenti: un **array di condizioni** in base alle quali trovare il record ed un **array di coppie colonne/valori** per indicare quali colonne aggiornare.
Se il record viene trovato, i suoi valori vengono aggiornati in base a quanto indicato nel secondo argomento della funzione.
Se il record non viene trovato, ne viene creato uno con i valori delle colonne risultanti dall'unione dei due argomenti della funzione.
- Si può passare una closure al metodo **updateOrCreate** per personalizzare gli attributi che vengono aggiornati o inseriti a seconda dell'esistenza o meno di un record corrispondente

```
$affected = DB::table('users')  
    →where('id', 1)  
    →update(['votes' => 1]);
```

```
DB::table('users')  
    →updateOrCreate(  
        ['email' => 'john@example.com', 'name' => 'John'],  
        ['votes' => '2']  
    );
```

```
DB::table('users')→updateOrCreate(  
    ['user_id' => $user_id],  
    fn ($exists) => $exists ? [  
        'name' => $data['name'],  
        'email' => $data['email'],  
    ] : [  
        'name' => $data['name'],  
        'email' => $data['email'],  
        'marketable' => true,  
    ],  
);
```

INCREMENT E DECREMENT

- I metodi **increment** e **decrement** servono per **aumentare o diminuire il valore di una data colonna**.
- Entrambi accettano almeno un argomento: la colonna da modificare. Un **secondo argomento** opzionale può essere specificato **per indicare di quanto** il valore della colonna dovrebbe essere aumentato o diminuito. Un **terzo argomento** può essere fornito per indicare **altre colonne** da aggiornare.
- Si possono anche incrementare o decrementare **più colonne alla volta** con i metodi **incrementEach** ed **decrementEach**

```
DB::table('users')->incrementEach([  
    'votes' => 5,  
    'balance' => 100,  
]);
```

```
DB::table('users')->increment('votes');  
  
DB::table('users')->increment('votes', 5);  
  
DB::table('users')->decrement('votes');  
  
DB::table('users')->decrement('votes', 5);
```

```
DB::table('users')->increment('votes', 1, ['name' => 'John']);
```

ISTRUZIONE DELETE

Il metodo **delete** elimina i record dal database. Si può vincolare il delete usando la clausola **where** prima di chiamare il metodo **delete**. Viene **ritornato il numero di righe interessate dalla cancellazione**

```
$deleted = DB::table('users')->delete();  
  
$deleted = DB::table('users')->where('votes', '>', 100)->delete();
```

PESSIMISTIC LOCKING

Il pessimistic locking evita che la riga selezionata venga modificata prima che la transazione con il database sia completata. Si usano i metodi **sharedLock** o **lockForUpdate**

```
DB::table('users')  
->where('votes', '>', 100)  
->sharedLock()  
->get();
```

```
DB::table('users')  
->where('votes', '>', 100)  
->lockForUpdate()  
->get();
```

Il lock «for update» evita che i record selezionati siano modificati o selezionati con un altro shared lock

DEBUGGING

Metodi dd e dump

Utilizzati per eseguire il dump dei binding della query e dell'SQL corrente. Il metodo **dd** visualizza le informazioni di debug e interrompe l'esecuzione della richiesta.

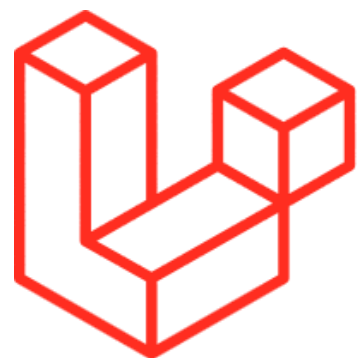
Il metodo **dump** visualizza le informazioni di debug ma continua l'esecuzione della richiesta

```
DB::table('users')->where('votes', '>', 100)->dd();  
DB::table('users')->where('votes', '>', 100)->dump();
```

Metodi dumpRawSql e ddRawSql

Utilizzati per eseguire il dump dell'SQL con tutti i bindings opportunamente sostituiti.

```
DB::table('users')->where('votes', '>', 100)->dumpRawSql();  
DB::table('users')->where('votes', '>', 100)->ddRawSql();
```



Laravel

LE BASI

SESSION

• Poichè le **applicazioni HTTP driven** sono **stateless**, le **sessioni rappresentano un modo per immagazzinare informazioni sull'utente attraverso più richieste.**

• Queste informazioni sono tipicamente memorizzate in un **archivio persistente**/backend al quale si può accedere con richieste successive

• Laravel offre una serie di backend per le sessioni a cui si accede tramite un'API unificata.

CONFIGURAZIONE

Il file di configurazione della sessione dell'applicazione si trova in **config/session.php**.

Di default, Laravel è configurato per utilizzare il driver delle sessioni basato sul database.

L'opzione di **configurazione del driver di sessione** definisce **dove saranno salvati i dati di sessione per ogni richiesta.**

Laravel include **diversi drivers**:

- **file** – le sessioni sono salvate in `storage/framework/sessions`
- **cookie** – sessioni salvate in cookies sicuri e criptati
- **database** – sessioni salvate in database relazionali
- **memcached /redis** – sessioni salvate in uno di questi archivi veloci e basati sulla cache
- **dynamodb** – sessioni salvate in AWS DynamoDB
- **array** – sessioni memorizzate in un array PHP e non saranno conservate. Usato principalmente durante i test.

PREREQUISITI DEL DRIVER

DATABASE

Quando si usa il driver di sessione database, si dovrà **creare una tabella per contenere i dati della sessione**. Solitamente, questa è inclusa nella migrazione di default `0001_01_01_000000_create_users_table.php`

Si può usare il comando Artisan **make:session-table** per generare questa migration qualora la tabella non sia già presente.

```
php artisan make:session-table
```

```
php artisan migrate
```

```
Schema::create('sessions', function (Blueprint $table) {  
    $table->string('id')->primary();  
    $table->foreignId('user_id')->nullable()->index();  
    $table->string('ip_address', 45)->nullable();  
    $table->text('user_agent')->nullable();  
    $table->longText('payload');  
    $table->integer('last_activity')->index();  
});
```

1- INTERAGIRE CON LA SESSION

1.1 – RECUPERARE DATI

Esistono principalmente due modi per lavorare con i dati della session:

- L'helper globale `session`
- Attraverso un'istanza della classe `Request`

1.1.1 ACCEDERE LA SESSIONE VIA ISTANZA DELLA REQUEST

L'istanza della classe `Request` può essere type hinted nella closure di una route o nel metodo del controller. Si ricorda che le dipendenze del metodo del controller sono automaticamente iniettate attraverso il service container.

Quando si recupera un elemento dalla session, si può **passare un valore di default come secondo argomento del metodo `get`**.

Questo valore di default sarà ritornato **se la chiave specificata non esiste** nella session.

Se si passa una **closure come secondo argomento** del metodo `get` e la chiave richiesta non esiste, la **closure sarà eseguita ed il risultato ritornato**

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\View\View;

class UserController extends Controller
{
    /**
     * Show the profile for the given user.
     */
    public function show(Request $request, string $id): View
    {
        $value = $request->session()->get('key');

        // ...

        $user = $this->users->find($id);

        return view('user.profile', ['user' => $user]);
    }
}
```

```
$value = $request->session()->get('key', 'default');

$value = $request->session()->get('key', function () {
    return 'default';
});
```

1.1.2 L'HELPER GLOBALE SESSION

Per recuperare e memorizzare i dati nella session, si può anche usare la **funzione globale session**.

Quando l'helper **session** è chiamato **con un singolo argomento**, ritorna il **valore di quella chiave** di sessione.

Quando è chiamato con un **array di coppie chiave/valore**, questi **valori saranno salvati nella session**

```
Route::get('/home', function () {  
    // Retrieve a piece of data from the session...  
    $value = session('key');  
  
    // Specifying a default value...  
    $value = session('key', 'default');  
  
    // Store a piece of data in the session...  
    session(['key' => 'value']);  
});
```

1.1.3 – RECUPERARE TUTTI I DATI DELLA SESSION

Per recuperare **TUTTI** i dati della session, si usa il metodo **all**

```
$data = $request->session()->all();
```

1.1.4 – RECUPERARE PARTE DEI DATI DELLA SESSION

I metody **only** e **except** permettono di recuperare un subset di dati

```
$data = $request->session()->only(['username', 'email']);  
  
$data = $request->session()->except(['username', 'email']);
```

1.1.5 – DETERMINARE SE UN ELEMENTO ESISTE NELLA SESSION

- Si usa il metodo **has** che ritorna un booleano: **true se l'elemento esiste e non è null**.
- Per determinare **se un elemento esiste** nella session, **anche se il suo valore è null**, si usa il metodo **exists**
- Al contrario, per **verificare se un elemento è mancante nella session**, si usa il metodo **missing** che ritorna **true se l'elemento non è presente**

```
if ($request->session()->has('users')) {  
    // ...  
}
```

```
if ($request->session()->exists('users')) {  
    // ...  
}
```

```
if ($request->session()->missing('users')) {  
    // ...  
}
```

1.2 – MEMORIZZARE DATI

Per memorizzare i dati nella session, si usa tipicamente il metodo **put** dell'istanza request, oppure si usa l'helper globale **session**.

```
// Via a request instance...  
$request->session()->put('key', 'value');  
  
// Via the global "session" helper...  
session(['key' => 'value']);
```

1.2.1 Pushing nell'array dei valori della Request

Usato per **inserire un nuovo valore** in un array che è un valore di sessione.

Ad esempio, se la chiave **user.teams** contiene un array di nomi di team, si può inserire un nuovo valore nell'array.

```
$request->session()->push('user.teams', 'developers');
```

1.2.2 Recuperare ed eliminare un elemento

Il metodo **pull** **recupera ed elimina** un elemento dalla sessione con una singola istruzione

```
$value = $request->session()->pull('key', 'default');
```

1.2.3 Incrementare e decrementare valori di sessione

Si usano i metodi **increment** e **decrement** per **aumentare o diminuire** un intero

```
$request->session()->increment('count');  
$request->session()->increment('count', $incrementBy = 2);  
$request->session()->decrement('count');  
$request->session()->decrement('count', $decrementBy = 2);
```


1.3 – FLASH DATA

Per **memorizzare** gli elementi **nella session per la richiesta successiva**, si può usare il metodo **flash**.

I dati memorizzati nella session con questo metodo saranno disponibili immediatamente e durante la successiva richiesta HTTP.

In seguito alla richiesta successiva, i dati saranno cancellati.

I dati flash sono **utili principalmente per messaggi di stato di breve durata**

Per **conservare i dati flash per più requests**, si può utilizzare il metodo **reflash** che terrà i dati per un'altra request.

Se si ha bisogno **solo di specifici dati**, si può utilizzare il metodo **keep**.

Per conservare i dati **solo per la request corrente**, esiste il metodo **now**.

```
$request->session()->flash('status', 'Task was successful!');
```

```
$request->session()->reflash();  
$request->session()->keep(['username', 'email']);
```

```
$request->session()->now('status', 'Task was successful!');
```

1.4 – ELIMINARE DATI

Il metodo `forget` rimuove parte dei dati dalla session

Per eliminare **TUTTI I DATI** si usa il metodo `flush`

```
// Forget a single key...
$request->session()->forget('name');

// Forget multiple keys...
$request->session()->forget(['name', 'status']);

$request->session()->flush();
```

1.5 – RIGENERARE L'ID DELLA SESSION

Rigenerare l'ID della session è spesso fatto per prevenire un attacco di tipo **session fixation**.

Se si utilizza Laravel Fortify o uno dei starter kits di Laravel, l'ID della session sarà automaticamente rigenerato.

Per farlo manualmente, invece, serve il metodo `regenerate`.

Se si vuole **cancellare l'ID della session e rimuovere tutti i dati dalla session in un'unica istruzione**, si usa il metodo `invalidate`

```
$request->session()->regenerate();

$request->session()->invalidate();
```

HTTP CLIENT

Laravel fornisce un'API espressiva e minimale intorno al client HTTP Guzzle, che consente di effettuare rapidamente **richieste HTTP in uscita** per **comunicare con altre applicazioni web**.

FARE REQUESTS

Per fare richieste ad applicazioni esterne, si usano i metodi **head**, **get**, **post**, **put**, **patch** e **delete** forniti dalla facade http.

```
use Illuminate\Support\Facades\Http;

$response = Http::get('http://example.com');
```

Il metodo get ritorna un'istanza di **Illuminate\Http\Client\Response** che prevede diversi metodi che possono essere utilizzati per esaminare le risposte

```
1 $response->body() : string;
2 $response->json($key = null, $default = null) : mixed;
3 $response->object() : object;
4 $response->collect($key = null) : Illuminate\Support\Collection;
5 $response->resource() : resource;
6 $response->status() : int;
7 $response->successful() : bool;
8 $response->redirect() : bool;
9 $response->failed() : bool;
10 $response->clientError() : bool;
11 $response->header($header) : string;
12 $response->headers() : array;
```

- L'oggetto **Response** implementa anche l'interfaccia **ArrayAccess** che consente di accedere ai dati **Json** direttamente dalla risposta stessa

```
return Http::get('http://example.com/users/1')['name'];
```

- Esistono inoltre dei metodi per determinare se la risposta ha una determinato status

```
$response->ok() : bool;           // 200 OK
$response->created() : bool;      // 201 Created
$response->accepted() : bool;     // 202 Accepted
$response->noContent() : bool;    // 204 No Content
$response->movedPermanently() : bool; // 301 Moved Permanently
$response->found() : bool;        // 302 Found
$response->badRequest() : bool;   // 400 Bad Request
$response->unauthorized() : bool; // 401 Unauthorized
$response->paymentRequired() : bool; // 402 Payment Required
$response->forbidden() : bool;    // 403 Forbidden
$response->notFound() : bool;     // 404 Not Found
$response->requestTimeout() : bool; // 408 Request Timeout
$response->conflict() : bool;     // 409 Conflict
$response->unprocessableEntity() : bool; // 422 Unprocessable Entity
$response->tooManyRequests() : bool; // 429 Too Many Requests
$response->serverError() : bool;   // 500 Internal Server Error
```

- **URI TEMPLATES** - Con il metodo **withUrlParameters**, il client **HTTP** consente anche di costruire gli URLs delle richieste usando il template URI.

```
Http::withUrlParameters([
    'endpoint' => 'https://laravel.com',
    'page' => 'docs',
    'version' => '11.x',
    'topic' => 'validation',
])->get('{+endpoint}/{page}/{version}/{topic}');
```

- **DUMPO DELLE RICHIESTE** – per visualizzare l'istanza della richiesta in uscita prima che sia inviata e terminare l'esecuzione dello script, si aggiunge il metodo **dd** all'inizio della definizione della request.

```
return Http::dd()->get('http://example.com');
```

ESEMPIO DI DUMP DELLA REQUEST

```
use Illuminate\Support\Facades\Http;

Http::dd() // 👉 Mostra tutte le informazioni della
           // richiesta e blocca l'esecuzione
->withHeaders([
    'Authorization' => 'Bearer token123',
])
->post('https://api.example.com/users', [
    'name' => 'Mario Rossi',
    'email' => 'mario@example.com',
]);
```

Cosa fa Http::dd()

- Blocca esecuzione prima che la richiesta venga inviata
- Stampa sullo schermo **l'URL della richiesta**, il **metodo HTTP** usato (POST nell'esempio), gli **Headers** (ad es. l'Authorization) ed il **body** della richiesta.

DATI DELLA REQUEST

- Con le richieste **POST**, **PUT** e **PATCH** è comune inviare dati aggiuntivi. Per questo il metodo accetta un array di dati come secondo argomento. Di default, i dati sono inviati usando il content type **application/json**

```
use Illuminate\Support\Facades\Http;

$response = Http::post('http://example.com/users', [
    'name' => 'Steve',
    'role' => 'Network Administrator',
]);
```

- **QUERY PARAMETERS PER RICHIESTE GET**

Si può **aggiungere una stringa di query** all'URL oppure passare un **array di coppie chiave/valore** come secondo argomento del metodo **get**.

```
$response = Http::get('http://example.com/users', [
    'name' => 'Taylor',
    'page' => 1,
]);
```

Oppure si può usare il metodo **withQueryParameters**.

```
Http::retry(3, 100)→withQueryParameters([
    'name' ⇒ 'Taylor',
    'page' ⇒ 1,
])→get('http://example.com/users')
```

- **INVIARE RICHIESTE FORM URL ENCODED**

Per inviare dati usando il conten type **application/x-www-form-urlencoded**, si usa il metodo **asForm** prima di creare la request

```
$response = Http::asForm()→post('http://example.com/users', [
    'name' ⇒ 'Sara',
    'role' ⇒ 'Privacy Consultant',
]);
```

- **RICHIESTE MULTI-PART**

per inviare file come request multi-part, si usa metodo **attach** prima di creare la request.

Accetta il nome del file ed il suo contenuto. Il terzo e quarto argomento (opzionali) sono, rispettivamente, il filename del file e gli headers associati al file.

```
$response = Http::attach(
    'attachment', file_get_contents('photo.jpg'), 'photo.jpg', ['Content-Type' ⇒ 'image/jpeg']
)→post('http://example.com/attachments');
```

Si può anche passare una risorsa di tipo stream

```
$photo = fopen('photo.jpg', 'r');

$response = Http::attach(
    'attachment', $photo, 'photo.jpg'
)→post('http://example.com/attachments');
```

HEADERS

Aggiunti usando il metodo **withHeaders** che accetta un array di coppie chiave/valore.

Si usa il metodo **accept** per specificare il content type che la nostra applicazione si aspetta come risposta.

Per velocizzare, il metodo **acceptJson** specifica che ci si aspetta un **application/json**.

```
$response = Http::withHeaders([
    'X-First' => 'foo',
    'X-Second' => 'bar'
])->post('http://example.com/users', [
    'name' => 'Taylor',
]);
```

```
$response = Http::accept('application/json')->get('http://example.com/users');
```

```
$response = Http::acceptJson()->get('http://example.com/users');
```

Il metodo **withHeader** unisce nuovi headers nell'header esistente.

Per sostituirlo interamente si usa il metodo **replaceHeaders**

```
$response = Http::withHeaders([
    'X-Original' => 'foo',
])->replaceHeaders([
    'X-Replacement' => 'bar',
])->post('http://example.com/users', [
    'name' => 'Taylor',
]);
```

AUTHENTICATION

Specificare credenziali di autenticazione basic o digest usando, rispettivamente, i metodi **withBasicAuth** o **withDigestAuth**

```
// Basic authentication...
$response = Http::withBasicAuth('taylor@laravel.com', 'secret')->post(/* ... */);

// Digest authentication...
$response = Http::withDigestAuth('taylor@laravel.com', 'secret')->post(/* ... */);
```

BEARER TOKEN

Metodo **withToken** per aggiungere rapidamente un bearer token alla header **Authorization**

```
$response = Http::withToken('token')->post(/* ... */);
```

TIMEOUT

Metodo **timeout** per specificare il numero massimo di secondi di attesa di una risposta (default: 30 secondi).

```
$response = Http::timeout(3)->get(/* ... */);
```

Metodo **connectTimeout** per impostare il tempo di attesa massimo di collegamento al server (default: 10 secondi).

```
$response = Http::connectTimeout(3)->get(/* ... */);
```


Integrazione AI in Laravel

Integrare Gemini

GEMINI

GEMINI

Gemini è un avanzato modello di intelligenza artificiale di Google, costruito su precedenti modelli di linguaggio come Meena e PaLM.

FUNZIONALITÀ PRINCIPALI DI GEMINI

Può comprendere e processare diversi tipi di dati:

- testo
- Codice
- Immagini
- Audio
- Video

Permette ad esempio di:

- Generare contenuti creativi (storie, traduzioni, articoli, musica, immagini, ecc.)
- Capire e scrivere codice
- Rispondere a domande complesse

L'integrazione di Gemini in Laravel può avvenire sia tramite API rest che SDK. Al momento non esiste un SDK PHP ufficiale per Gemini, ma online se ne trovano alcuni ben fatti.

`google-gemini-php/client`

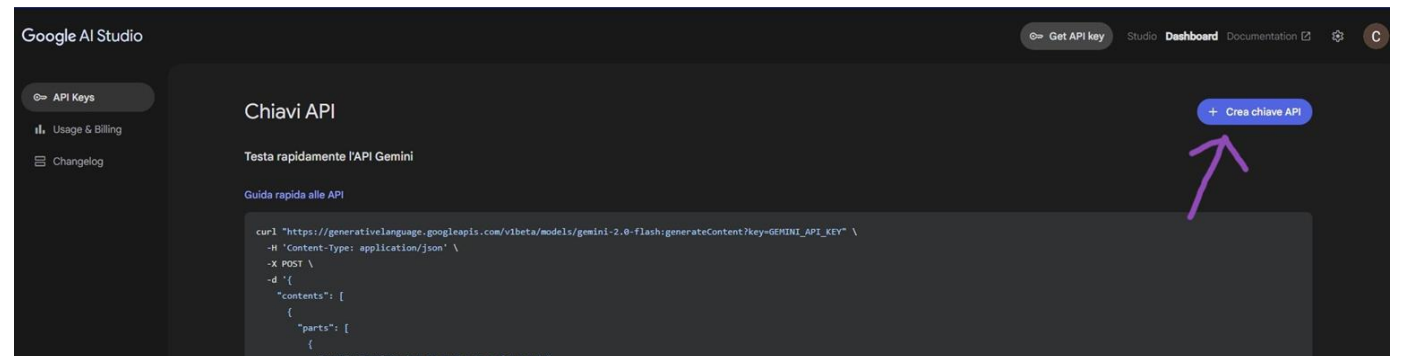
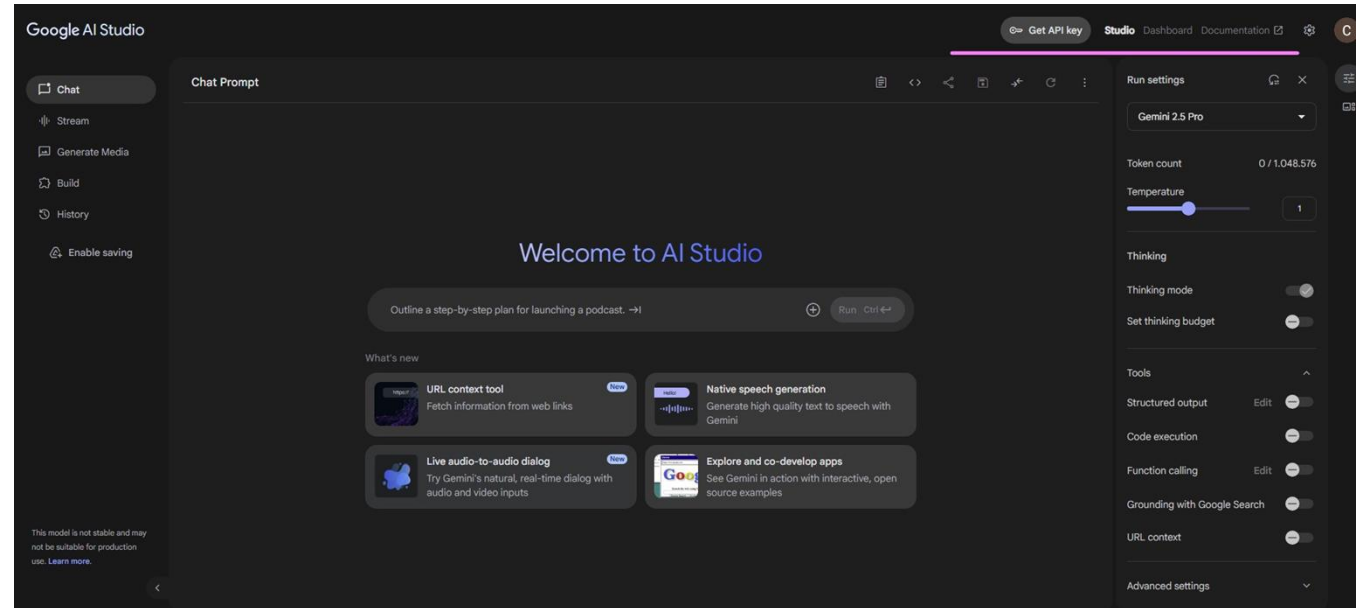
COME USARE GEMINI IN LARAVEL 12

1. INSTALLARE LARAVEL
Se non si ha un progetto già avviato, crearne uno nuovo

```
laravel new laravel-gemini-ai
```

1. OTTENERE LA CHIAVE API DI GEMINI

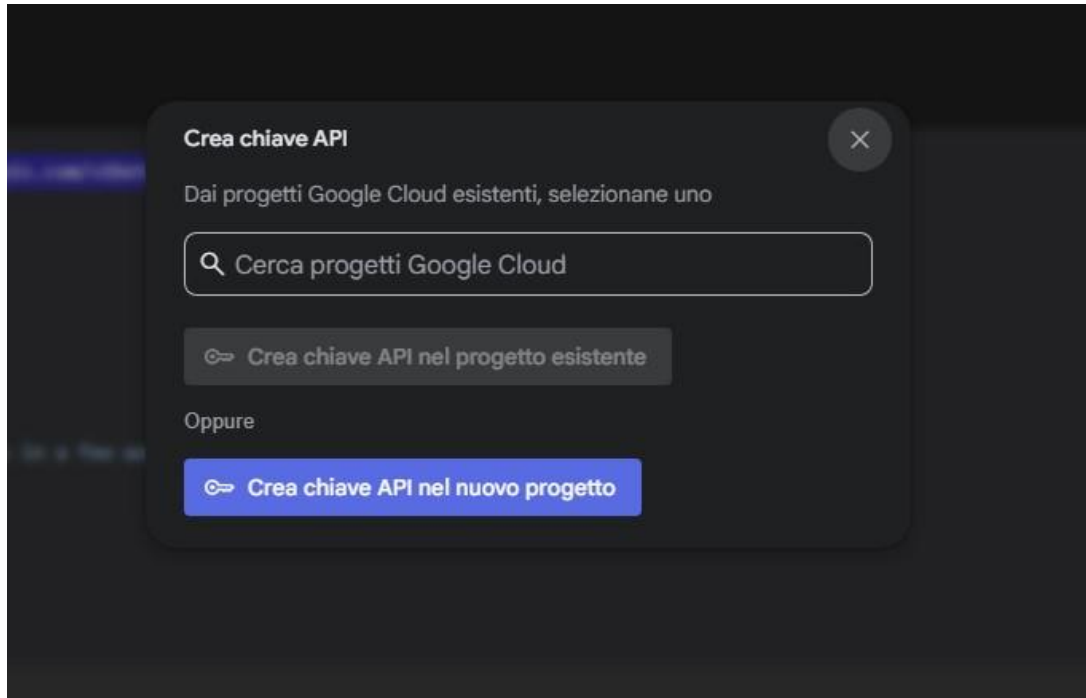
- Andare su [Google AI Studio](https://ai.google.dev/gemini-api/docs?hl=it)
-
- <https://ai.google.dev/gemini-api/docs?hl=it>
- Effettuare l'accesso
- Generare una chiave API e salvarla per un utilizzo futuro



Per visualizzare API

- <https://console.cloud.google.com/apis/credentials>
- Per visualizzare modelli disponibili in API:
- `https://generativelanguage.googleapis.com/v1/models?key=*****`
**

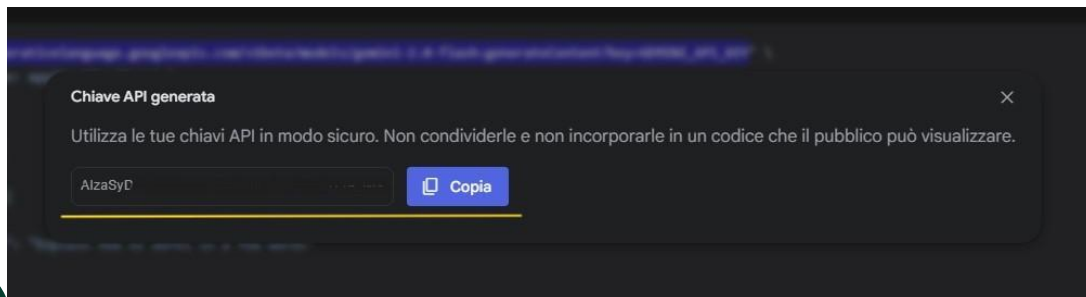
CONFIGURARE API KEY IN LARAVEL



.env FILE

Aprire il file .env in Laravel e salvare la chiave API

```
GEMINI_API_KEY=*****  
GEMINI_BASE_URL=https://generativelanguage.googleapis.com/v1/models/
```



Aggiungere la configurazione in config/services.php

- File utilizzato per memorizzare le credenziali dei servizi di terze parti. Posizione predefinita per questo tipo di informazioni.
- Aprire il file config/services.php

```
'gemini' => [  
    'api_key' => env('GEMINI_API_KEY'),  
    'base_url' => env('GEMINI_BASE_URL', 'https://api.gemini.com/v1/'),  
],
```

Creazione di una Classe Service dedicata (Best Practice)

- Invece di effettuare chiamate HTTP direttamente da un controller, creare una classe di servizio che incapsula tutta la logica di comunicazione con Gemini
- Se non presente, creare cartella app/Services
- Al suo interno creare il file GeminiService.php
- Aprire il file e creare la logica

```
class GeminiService
{
    protected string $apiKey;
    protected string $endpoint;
    protected string $model;

    public function __construct()
    {
        $this->apiKey = config('services.gemini.api_key');
        $this->model = 'gemini-1.5-flash-002'; // usa un modello
        effettivamente disponibile
        $this->endpoint = config('services.gemini.base_url') .
        "{$this->model}:generateContent";
    }
}
```

Il metodo HTTP::post() invia la richiesta. Secondo argomento è il body della richiesta, nel formato richiesto da API Gemini



```
public function generateText(string $prompt): ?string
{
    $response = Http::post($this->endpoint . '?key=' . $this->apiKey,
    [
        'contents' => [
            [
                'parts' => [
                    ['text' => $prompt]
                ]
            ]
        ]
    ]);

    if ($response->successful()) {
        return $response->json('candidates.0.content.parts.0.text');
    }

    return null;
}
```


Inserire la chiave API in un Header

```
$response = Http::withHeaders([
    'x-goog-api-key' => $this->apiKey,
    'Content-Type'   => 'application/json',
])->post($this->endpoint, [
    'contents' => [
        [
            'parts' => [
                ['text' => $prompt]
            ]
        ]
    ]
]);
```

- L'approccio visto ora prevede il passaggio della chiave API tramite query string. Metodo supportato da API di Google.
- Il metodo più sicuro è però l'inserimento della chiave in un header HTTP.
- Più sicuro perché:
 - Gli url completi vengono spesso registrati in chiaro in molti punti, come il log del server web, cronologia del browser. Questo aumenta superficie di attacco. Gli header HTTP sono visibili durante transazione ma sono registrati meno frequentemente.
 - CACHING: url messi in cache da browser o proxy.
- API di Google accetta, per il metodo generateContent, la chiave in un header specifico: x-goog-api-key
- Aprire app/Services/GeminiService.php e modificare tutti i metodi che fanno chiamate HTTP

Utilizzare il service in un controller

- `php artisan make:controller GeminiController`

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Services\GeminiService; // Importa il nostro servizio
use Illuminate\Http\Client\RequestException; // Importa l'eccezione del client HTTP

class GeminiController extends Controller
{
    protected GeminiService $geminiService;

    // Iniezione del servizio tramite il costruttore
    public function __construct(GeminiService $geminiService)
    {
        $this->geminiService = $geminiService;
    }

    public function ask(Request $request)
    {
        $request->validate([
            'question' => 'required|string|max:1000',
        ]);

        $prompt = $request->input('question');

        try {
            $responseText = $this->geminiService->generateText($prompt);

            return response()->json([
                'status' => 'success',
                'answer' => $responseText,
            ]);
        }
```

```
        } catch (RequestException $e) {
            // Errore specifico della richiesta HTTP (es. 400 Bad Request, 429 Too Many Requests)
            return response()->json([
                'status' => 'error',
                'message' => 'Error communicating with the API: ' . $e->getMessage(),
                'api_response' => $e->response->json() // Utile per il debug
            ], $e->response->status());
        } catch (\Exception $e) {
            // Altri errori (es. chiave API mancante, risposta malformata)
            return response()->json([
                'status' => 'error',
                'message' => 'An unexpected error occurred: ' . $e->getMessage(),
            ], 500);
        }
    }
}
```

Definire la rotta

VERSIONI LARAVEL < 11

- In routes/api.php
- `use App\Http\Controllers\GeminiController;`
- `Route::post('/gemini/ask', [GeminiController::class, 'ask']);`

VERSIONI LARAVEL > 11

- Il file routes/api.php non è presente di default
- Lanciare il comando `php artisan install:api`
- Questo comando, automaticamente, crea il file api.php, configura il routing nel file bootstrap/app.php, consente l'applicazione del corretto gruppo di middleware alla rotta.

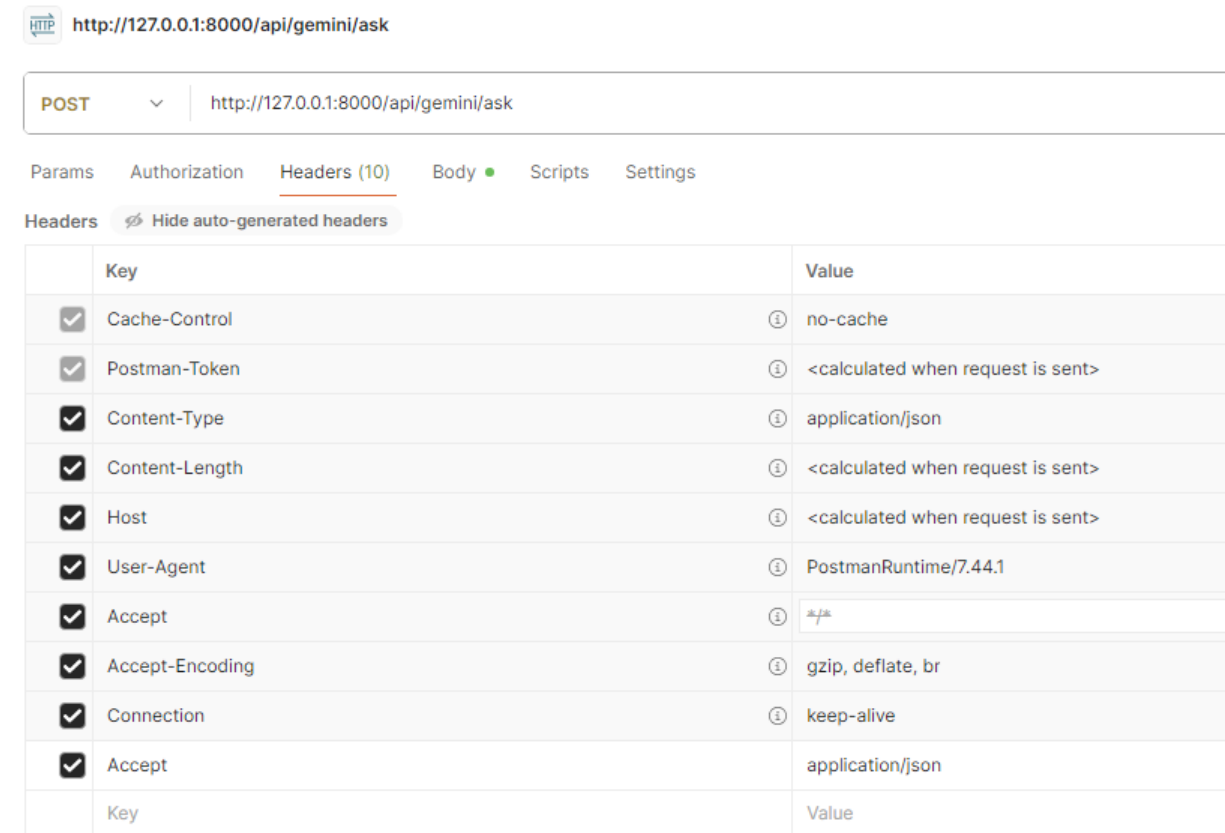
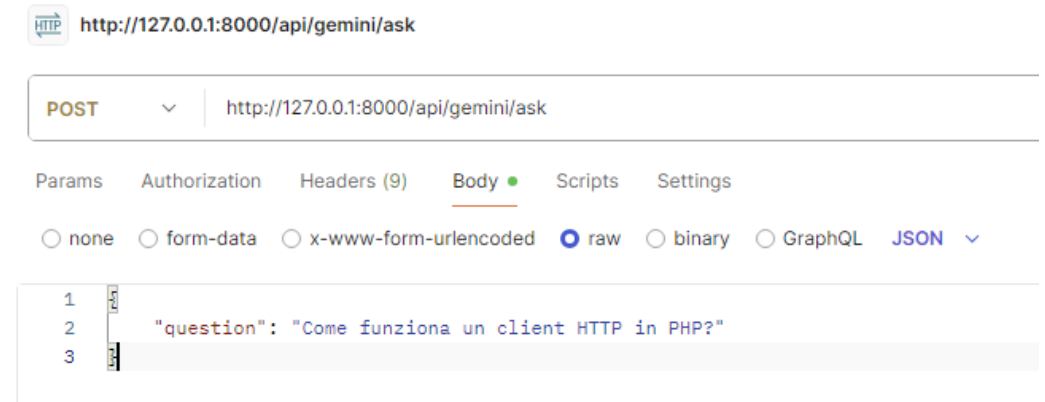
```
use App\Http\Controllers\GeminiController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');

Route::post('/gemini/ask', [GeminiController::class, 'ask']);
```

Testare l'API - POSTMAN

1. Avviare il server di Laravel
2. Aprire Postman e creare nuova richiesta
3. Selezionare metodo POST ed inserire Url dell'endpoint :
<http://127.0.0.1:8000/api/gemini/ask>
4. Configurare il body della richiesta
5. Impostare gli Header (importante). Il Content-Type dovrebbe già essere impostato su application/json perché si è scelto JSON nel body.
 1. Se non già impostato, impostare la chiave Accept a application/json
6. Inviare la richiesta



Generated bash

```
curl --location --request POST 'http://127.0.0.1:8000/api/gemini/ask' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--data '{
  "question": "Come funziona un client HTTP in PHP?"
}'
```

COMANDO IN UNA SOLA RIGA

```
curl --location --request POST
"http://127.0.0.1:8000/api/gemini/ask" --header
"Content-Type: application/json" --header
"Accept: application/json" --data "{\"question\":
\"Come funziona un client HTTP in PHP?\"}" | jq
```

CONTROLLARE L'API CON CURL

- Aprire il terminale
- Inserire il seguente comando:

```
curl --location --request POST
'http://127.0.0.1:8000/api/gemini/ask' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--data '{
  "question": "Come funziona un client HTTP in PHP?"
}'
```

- Windows non consente di andare a capo con \. Ogni parte del comando deve essere scritta su una sola riga, oppure bisogna concatenare le righe usando ^ (solo su cmd.exe)

```
C:\WINDOWS\system32>curl --location --request POST "http://127.0.0.1:8000/api/gemini/ask" --
ation/json" --header "Accept: application/json" --data "{\"question\": \"Come funziona un cl
{\"status\":\"success\",\"answer\":\"Un client HTTP in PHP \u00e8 un pezzo di codice che ti permett
a un server web (ad esempio, un'API, un altro sito web) e di ricevere e processare le sue ri
per la comunicazione tra applicazioni web, l'integrazione con servizi esterni, il fetching d
o altro.\n\nVediamo come funziona a un livello pi\u00f9 profondo e quali sono le principali
rlo in PHP.\n\n---\n\n### Come Funziona un Client HTTP (Principi Base)\n\nImmagina un dialog
client) e un server web:\n\n1. **Costruzione della Richiesta:** Il client prepara tutti i d
vuole inviare:\n
* **URL:** L'indirizzo della risorsa desiderata (es. `https://api.example.com/users/123`
* **Metodo HTTP:** L'azione che si vuole compiere (GET per recuperare, POST per inviare dati, PUT per aggiornare, DE
LETE per eliminare, ecc.).\n
* **Header:** Metadati aggiuntivi sulla richiesta (es. `Content-Type: application/json`
n` per indicare il formato dei dati inviati, `Authorization: Bearer <token>` per l'autenticazione, `User-Agent` per iden
tificare il client).\n
* **Corpo (Body):** I dati effettivi da inviare, tipicamente per richieste POST o PUT (es. u
n JSON con i dati di un nuovo utente).\n\n2. **Invio della Richiesta:** Il client apre una connessione di rete (solitam
ente TCP/IP) verso il server specificato dall'URL e invia la richiesta HTTP formattata.\n\n3. **Elaborazione della Ris
posta:** Il server riceve la richiesta, la processa, e invia una risposta al client. Anche la risposta \u00e8 composta d
a:\n
* **Status Code:** Un numero di 3 cifre che indica l'esito dell'operazione (es. `200 OK` per successo, `404 No
```


FONTI

- [Gestore di dipendenze Composer \(aulab.it\)](#)
 - [Laravel e il ciclo di vita delle richieste HTTP \(aulab.it\)](#)
 - [PHP Laravel Project Example for Beginners – Phppot](#)
 - [PHP Laravel Project Example for Beginners – Phppot](#)
 - [Laravel: risorse gratuite per sviluppatori Web e non solo \(laramind.com\)](#)
 - [Laravel - Middleware | Tutorialspoint](#)
-
- [Laravel Tutorial: Step by Step Guide to Building Your First Laravel Application | Laravel News \(laravel-news.com\)](#)
 - [The Best Laravel Tutorials !\[\]\(1207edb9a08751d3d55970560645ed23_img.jpg\) For Beginners 2023 - DEV Community](#)
 - [Laravel Tutorial for Beginners \(guru99.com\)](#)
 - [Laravel View Composers Vs View Creators | Medium](#)

TERMINI UTILI

- **HELPERS:** gli helpers in Laravel sono funzioni built-in che possono essere chiamate in qualsiasi punto dell'applicazione. Se l'helper di cui si ha bisogno non esiste, si può creare il proprio helper.